

Static analysis and software verification

Lecture 13 - The finite state automata abstract domain

JavaScript is a scripting language
used to interact with HTML

```
<!DOCTYPE html>
<title>Example</title>

<p>
  <span id="timestamp"></span>
  milliseconds since midnight, January 1, 1970.
</p>

<script>
  document.addEventListener("DOMContentLoaded", function(event) {
    window.setTimeout(function(){
      document.location.reload(true);
    }, 3000);

    document.getElementById("timestamp").innerText = new Date().getTime
      ();
  });
</script>
```

```
<!DOCTYPE html>
<title>Example</title>
```

```
<p>
  <span id="timestamp"></span>
  milliseconds since midnight, January 1, 1970.
</p>
```

```
<script>
  document.addEventListener("DOMContentLoaded", function(event) {
    window.setTimeout(function(){
      document.location.reload(true);
    }, 3000);

    document.getElementById("timestamp").innerText = new Date().getTime
      ();
  });
</script>
```

HTML

JS

```
<!DOCTYPE html>
<title>Example</title>
```

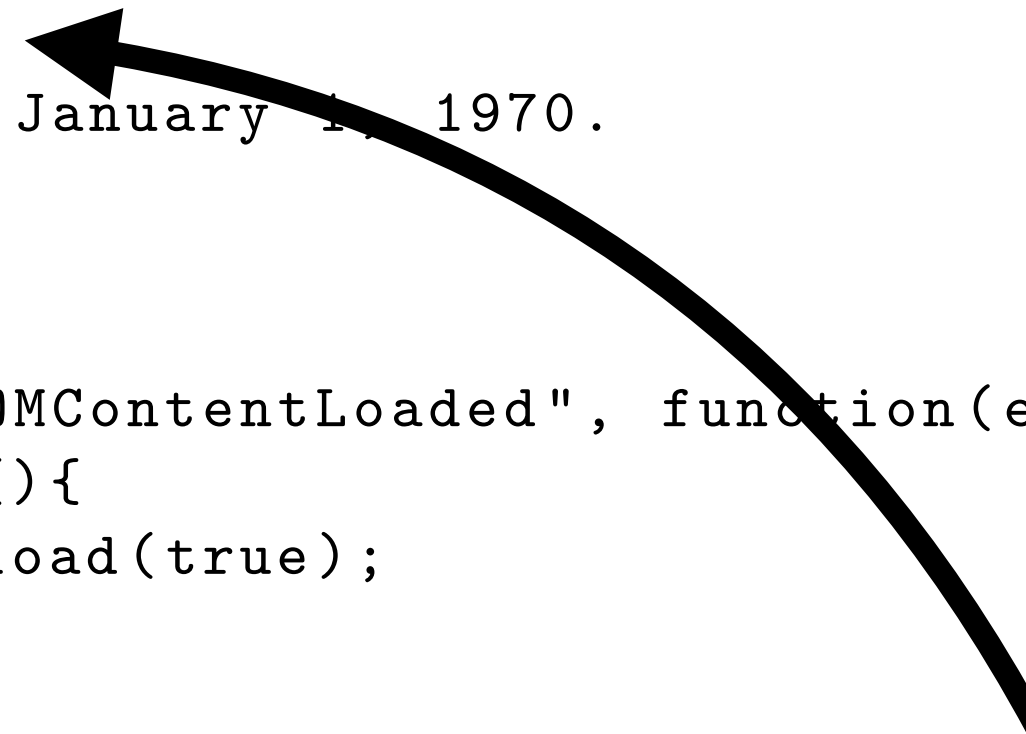
```
<p>
  <span id="timestamp"></span>
  milliseconds since midnight, January 1, 1970.
</p>
```

```
<script>
  document.addEventListener("DOMContentLoaded", function(event) {
    window.setTimeout(function(){
      document.location.reload(true);
    }, 3000);

    document.getElementById("timestamp").innerText = new Date().getTime
      ();
  });
</script>
```

HTML

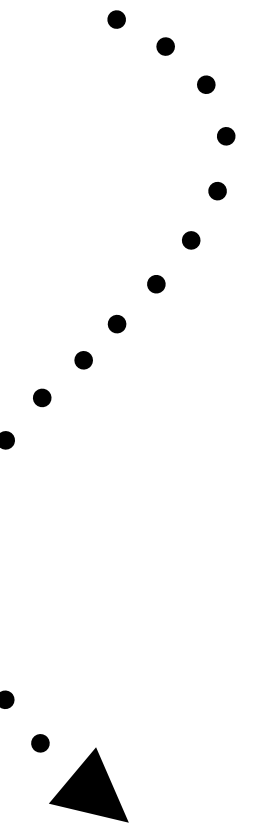
JS



JavaScript is not strongly typed

$$x = 5$$

`x = 5`



`x = "str"`

JavaScript has implicit type conversion

5 + true

5 + true

⋮

Type conversion

⋮



5 + 1

JavaScript has prototypal inheritance

Java: object-class relation

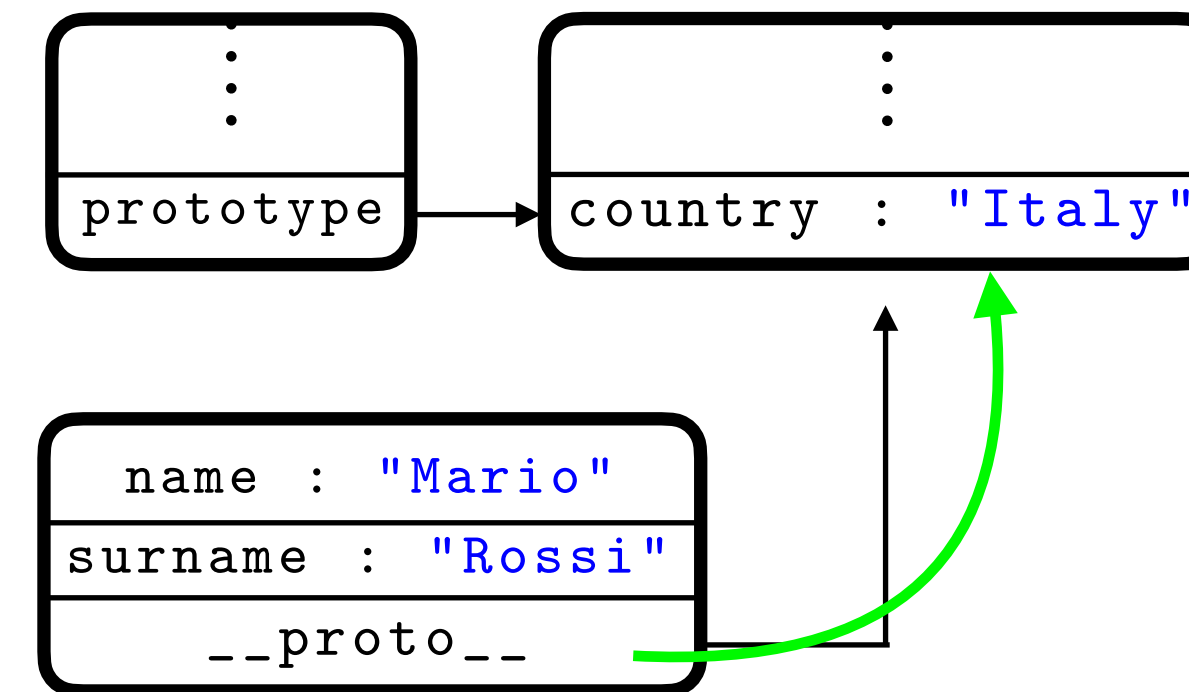
JavaScript: object-object relation

```
function Person(name, surname) {  
  this.name = name;  
  this.surname = surname;  
}
```

```
Person.prototype = {country : "Italy"}
```

```
var p = new Person("Mario", "Rossi");
```

```
console.log(p.country)
```



Java: object-class relation

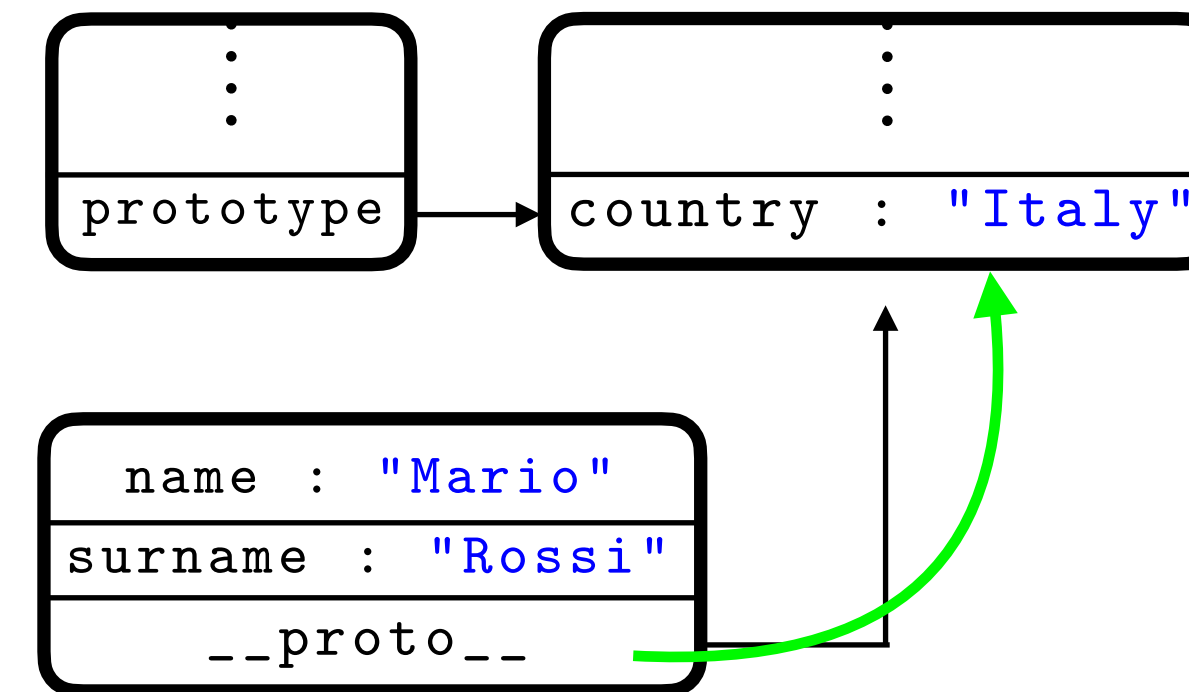
JavaScript: object-object relation

```
function Person(name, surname) {  
  this.name = name;  
  this.surname = surname;  
}
```

```
Person.prototype = {country : "Italy"}
```

```
var p = new Person("Mario", "Rossi");
```

```
console.log(p.country)
```



JavaScript allows to rewrite standard built-in methods

```
Number.prototype.toString = ...
```

JavaScript has eval

or: “the goto of the web”

```
eval ("a = a + 1")
```

String

`eval ("a = a + 1")`

⋮

Parsing of "a = a + 1"

String

`eval ("a = a + 1")`

⋮

Parsing of "a = a + 1"

⋮

a=a+1

String



Code!

Assembling code at run-time is, in general, dangerous

```
eval('alert("Your query was ' + document.location.search + '");');
```

Assembling code at run-time is, in general, dangerous



```
eval('alert("Your query was ' + document.location.search + '");');
```

XSS, SSJS, CSRF, Code injection...

How much eval is popular?

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

Benevolent code

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

Benevolent code

G. Richards et al. *“The eval that men do”*

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

Benevolent code

G. Richards et al. *“The eval that men do”*

Malicious code

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

Benevolent code

G. Richards et al. *“The eval that men do”*

Malicious code

We have analyzed a data-set of JavaScript malware samples:

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

Benevolent code

G. Richards et al. *“The eval that men do”*

Malicious code

We have analyzed a data-set of JavaScript malware samples:

48% have at least
one *explicit* eval
call

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

Benevolent code

G. Richards et al. *“The eval that men do”*

Malicious code

We have analyzed a data-set of JavaScript malware samples:

48% have at least
one *explicit* eval
call

57% have at least
one eval call
(implicit/explicit)

How much eval is popular?

Best practice:

“eval is the most misused feature of JavaScript. Avoid it.”

Benevolent code

G. Richards et al. *“The eval that men do”*

Malicious code

We have analyzed a data-set of JavaScript malware samples:

48% have at least
one *explicit* eval
call

57% have at least
one eval call
(implicit/explicit)

10% have nested
eval calls

The problem (in static analysis)

Static analysis assumption: the program code is static

```
x = 0;
while (x >= 0) {
    if (x % 2 == 0)
        x = x + 1;
    else
        x = x + 2;
}
```

The problem (in static analysis)

Static analysis assumption: the program code is static

The problem (in static analysis)

Static analysis assumption: the program code is static
but this assumption is broken by eval!

The problem (in static analysis)

Static analysis assumption: the program code is static
but this assumption is broken by eval!

```
while (x++ < 3)
  os = os + "x=x+1;y=10;x=x+1;";

if (x > 10)
  os = "while(x>5){x=x+1;y=x;}";

if (x == 5)
  os = "hello";

if (x == 8)
  os = "while(x;";

eval(os);
```

The problem (in static analysis)

Static analysis assumption: the program code is static
but this assumption is broken by eval!

```
while (x++ < 3)
  os = os + "x=x+1;y=10;x=x+1;";

if (x > 10)
  os = "while(x>5){x=x+1;y=x;}";

if (x == 5)
  os = "hello";

if (x == 8)
  os = "while(x;";

eval(os);
```



x = x + 1; y = 10; x = x + 1;

The problem (in static analysis)

Static analysis assumption: the program code is static
but this assumption is broken by eval!

```
while (x++ < 3)
  os = os + "x=x+1;y=10;x=x+1;";

if (x > 10)
  os = "while(x>5){x=x+1;y=x;}";

if (x == 5)
  os = "hello";

if (x == 8)
  os = "while(x;";

eval(os);
```



`x = x + 1; y = 10; x = x + 1;`

Derived at run-time
May contain other eval calls

The problem (in static analysis)

Static analysis assumption: the program code is static
but this assumption is broken by eval!

```
while (x++ < 3)
  os = os + "x=x+1;y=10;x=x+1;";

if (x > 10)
  os = "while(x>5){x=x+1;y=x;}";

if (x == 5)
  os = "hello";

if (x == 8)
  os = "while(x;";

eval(os);
```



`x = x + 1; y = 10; x = x + 1;`

Derived at run-time

May contain other eval calls

“You can’t check the code you don’t see”

State-of-art of static analysis for JavaScript

- Several static analyzers have been proposed for dynamic languages (PHP, Python, Ruby...)
- String analyses fails to precisely analyze string manipulation programs
- Almost none of them faced the problem of analyzing string-to-code statements

State-of-art of static analysis for JavaScript

- Several static analyzers have been proposed for dynamic languages (PHP, Python, Ruby...)
- String analyses fails to precisely analyze string manipulation programs
- Almost none of them faced the problem of analyzing string-to-code statements

Most common approaches:

- Ignoring the eval effects (making the tool unsound)
- Forbidding the eval usage

State-of-art of static analysis for JavaScript

- Several static analyzers have been proposed for dynamic languages (PHP, Python, Ruby...)
- String analyses fails to precisely analyze string manipulation programs
- Almost none of them faced the problem of analyzing string-to-code statements

Most common approaches:

- Ignoring the eval effects (making the tool unsound)
- Forbidding the eval usage

TAJS Type Analysis for JavaScript

SAFE: Scalable Analysis Framework for ECMAScript

JSAl: A Static Analysis Platform for JavaScript

Goal

Make static analysis *dynamic*

Goal

Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed
by the *concrete* string

Goal

Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed by the *concrete* string

Abstract

eval recursively calls the *abstract* interpreter on the programs expressed by the *abstract* string value

Goal

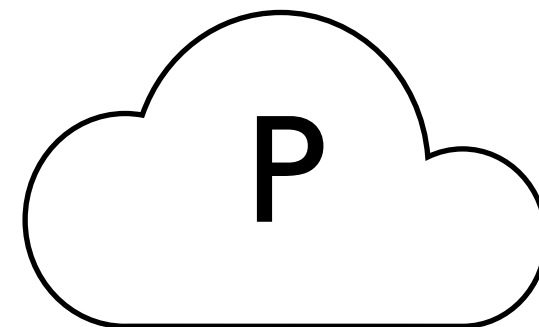
Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed by the *concrete* string

Abstract

eval recursively calls the *abstract* interpreter on the programs expressed by the *abstract* string value



Goal

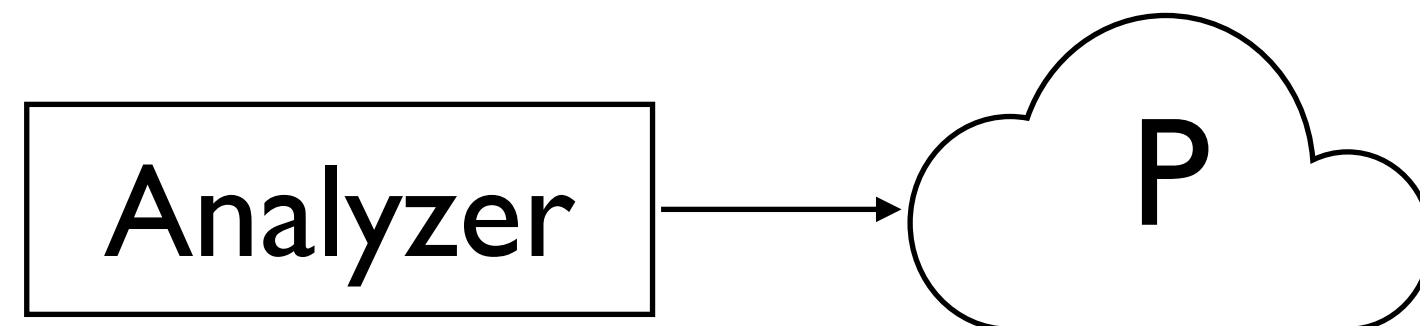
Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed by the *concrete* string

Abstract

eval recursively calls the *abstract* interpreter on the programs expressed by the *abstract* string value



Goal

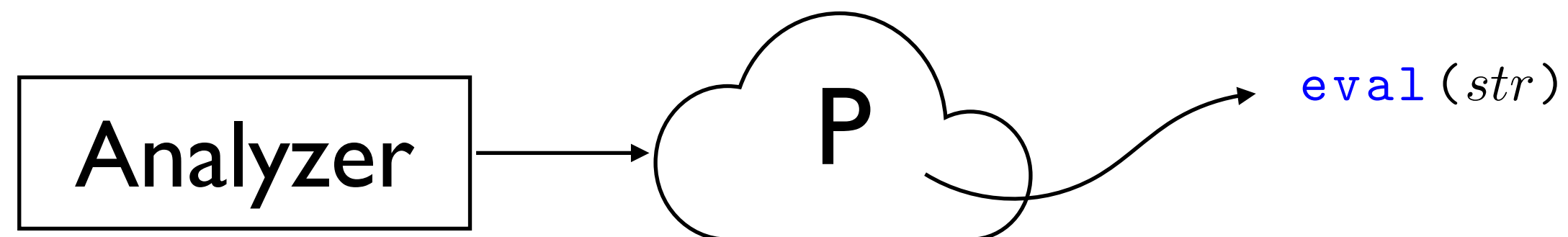
Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed by the *concrete* string

Abstract

eval recursively calls the *abstract* interpreter on the programs expressed by the *abstract* string value



Goal

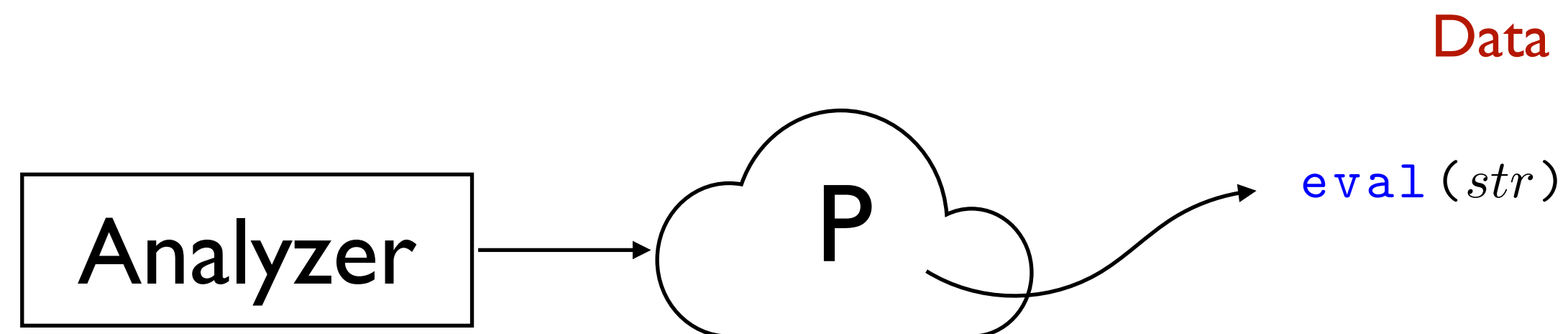
Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed by the *concrete* string

Abstract

eval recursively calls the *abstract* interpreter on the programs expressed by the *abstract* string value



Goal

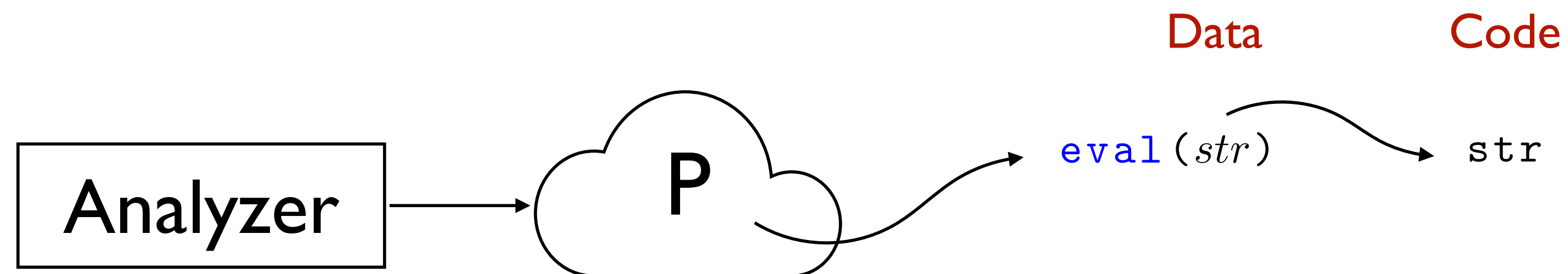
Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed by the *concrete* string

Abstract

eval recursively calls the *abstract* interpreter on the programs expressed by the *abstract* string value



Goal

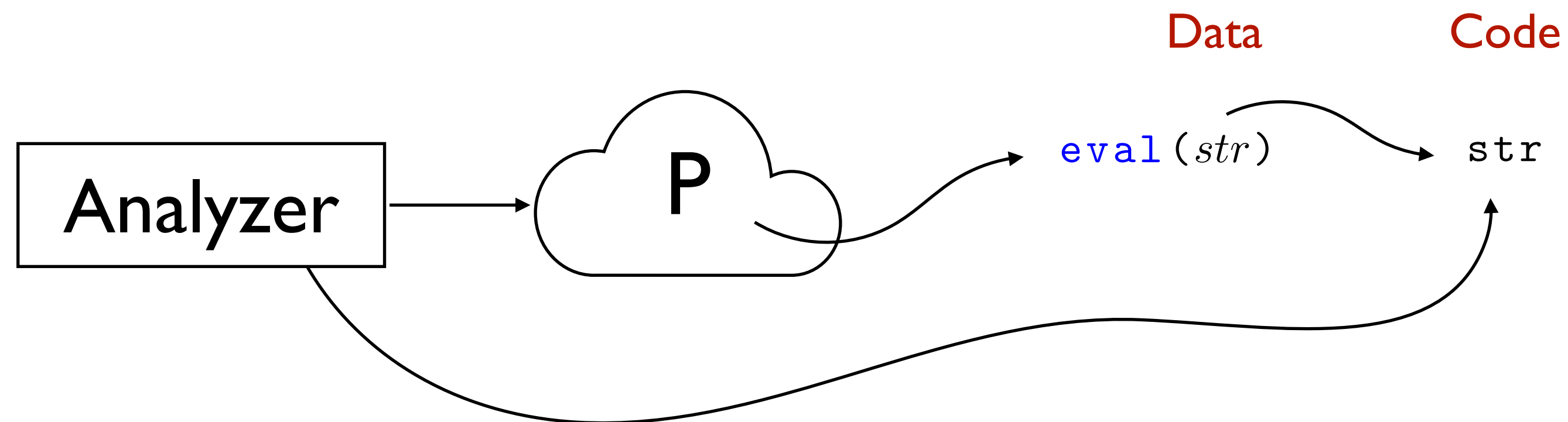
Make static analysis *dynamic*

Concrete

eval recursively calls the interpreter on the code expressed by the *concrete* string

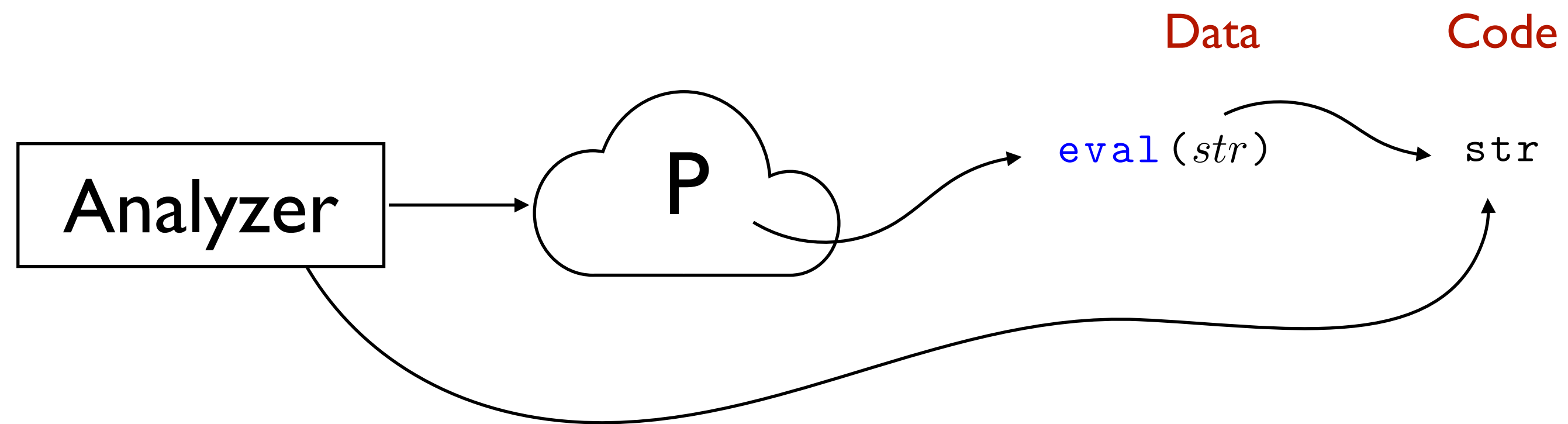
Abstract

eval recursively calls the *abstract* interpreter on the programs expressed by the *abstract* string value



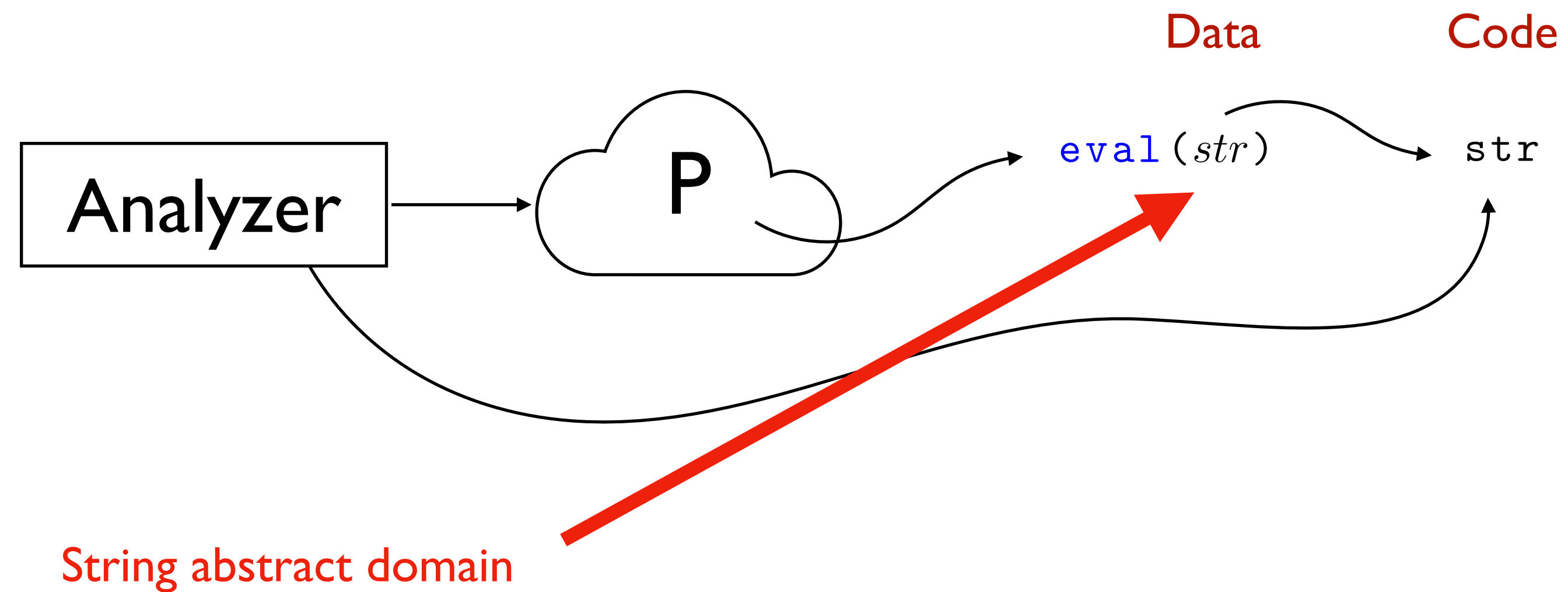
What we need?

Make static analysis *dynamic*



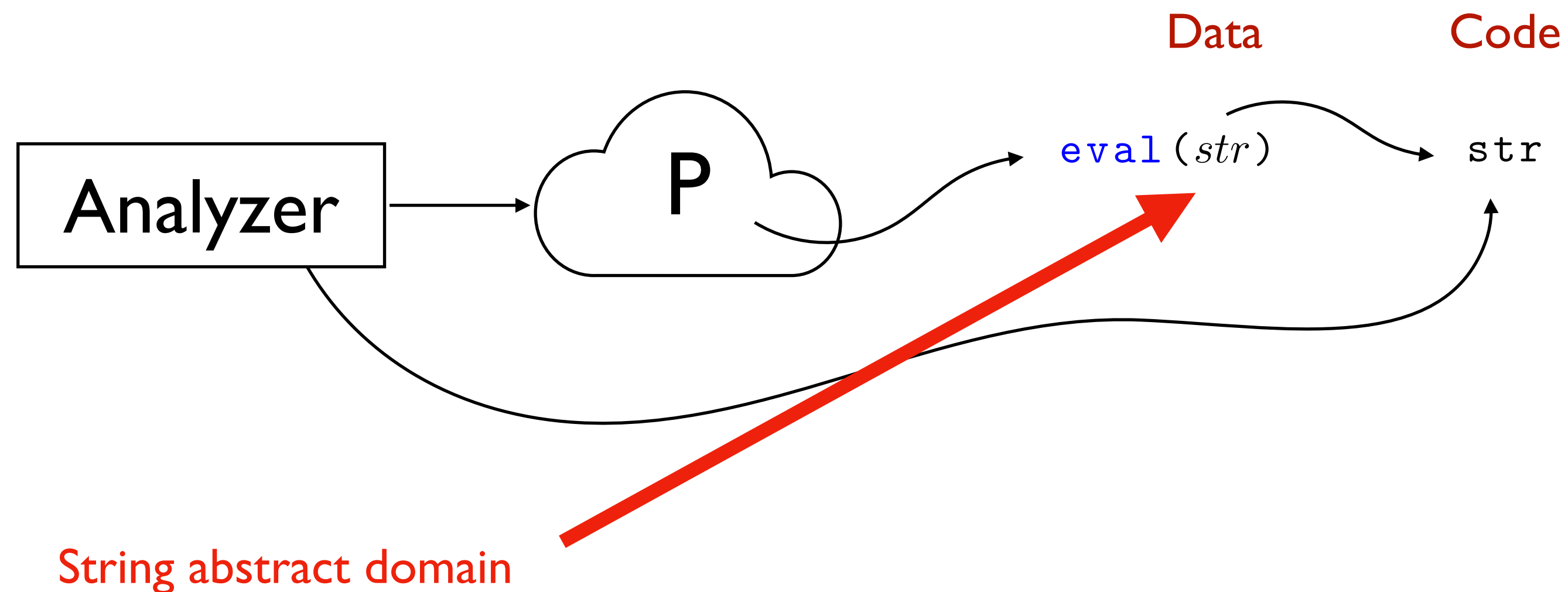
What we need?

Make static analysis *dynamic*



What we need?

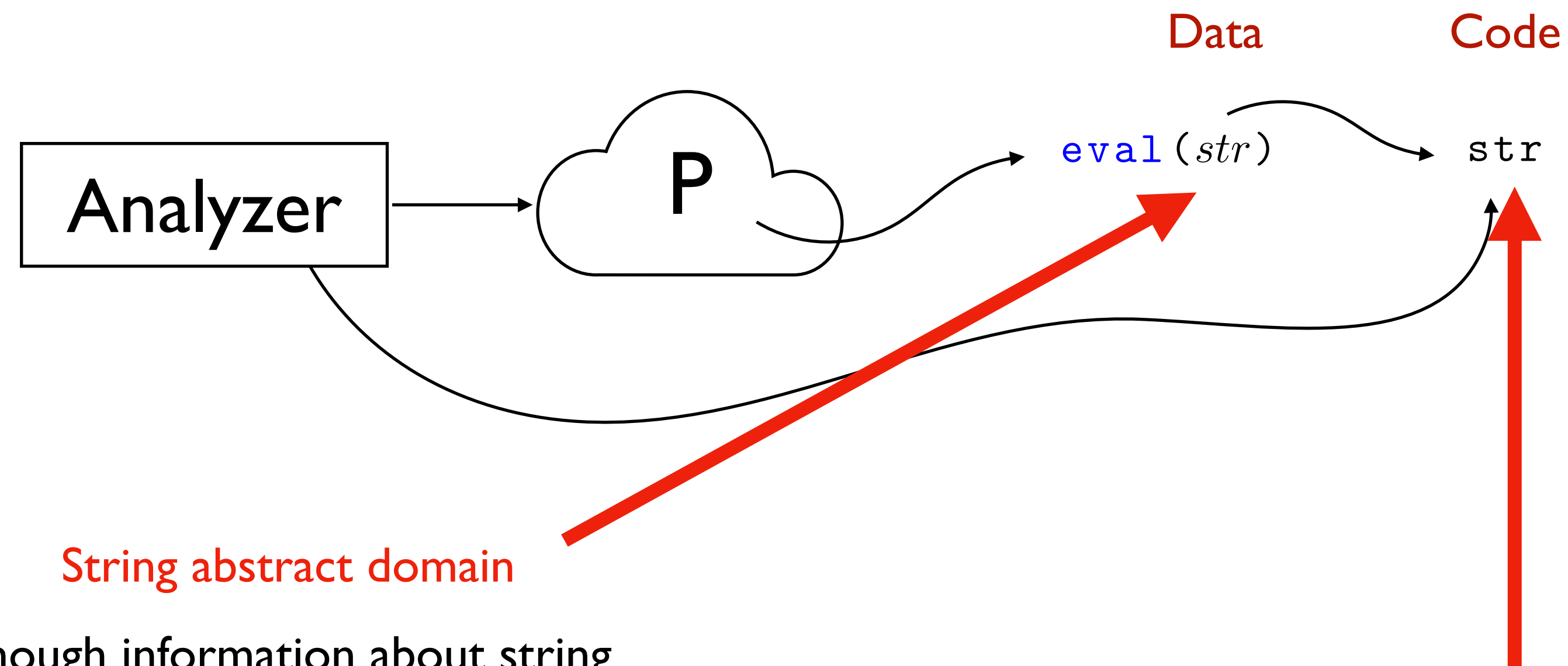
Make static analysis *dynamic*



- keep enough information about string properties in general
- keep enough information about the code expressed by strings abstractions

What we need?

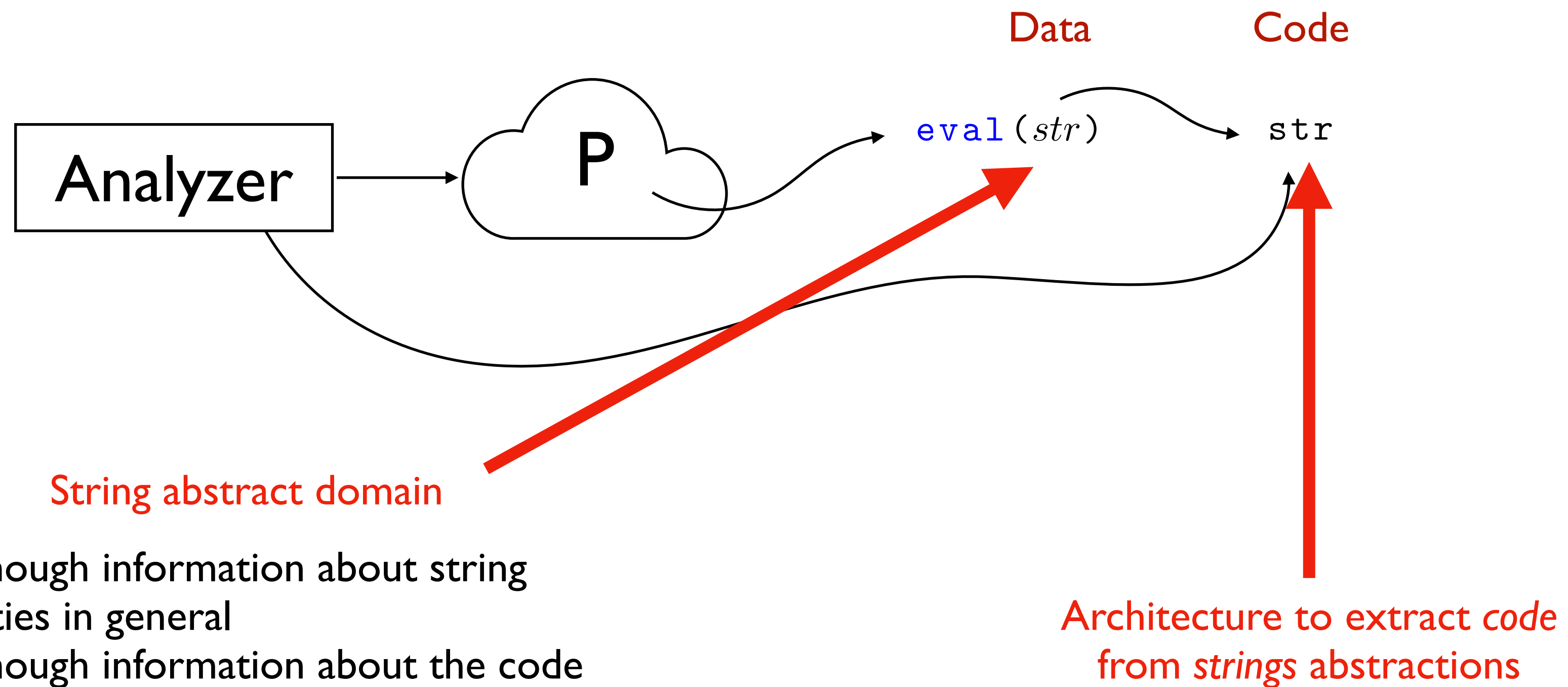
Make static analysis *dynamic*



- keep enough information about string properties in general
- keep enough information about the code expressed by strings abstractions

What we need?

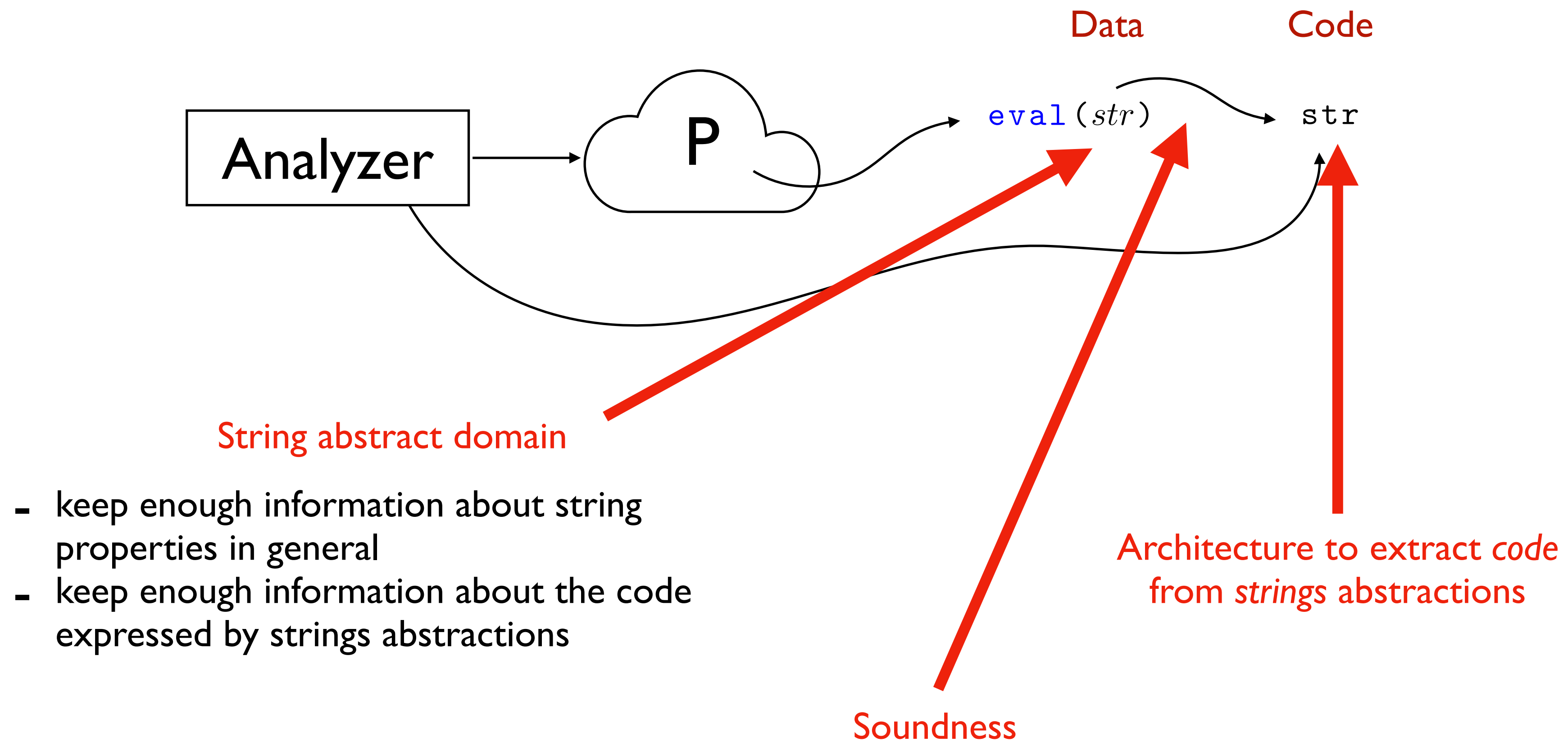
Make static analysis *dynamic*



- keep enough information about string properties in general
- keep enough information about the code expressed by strings abstractions

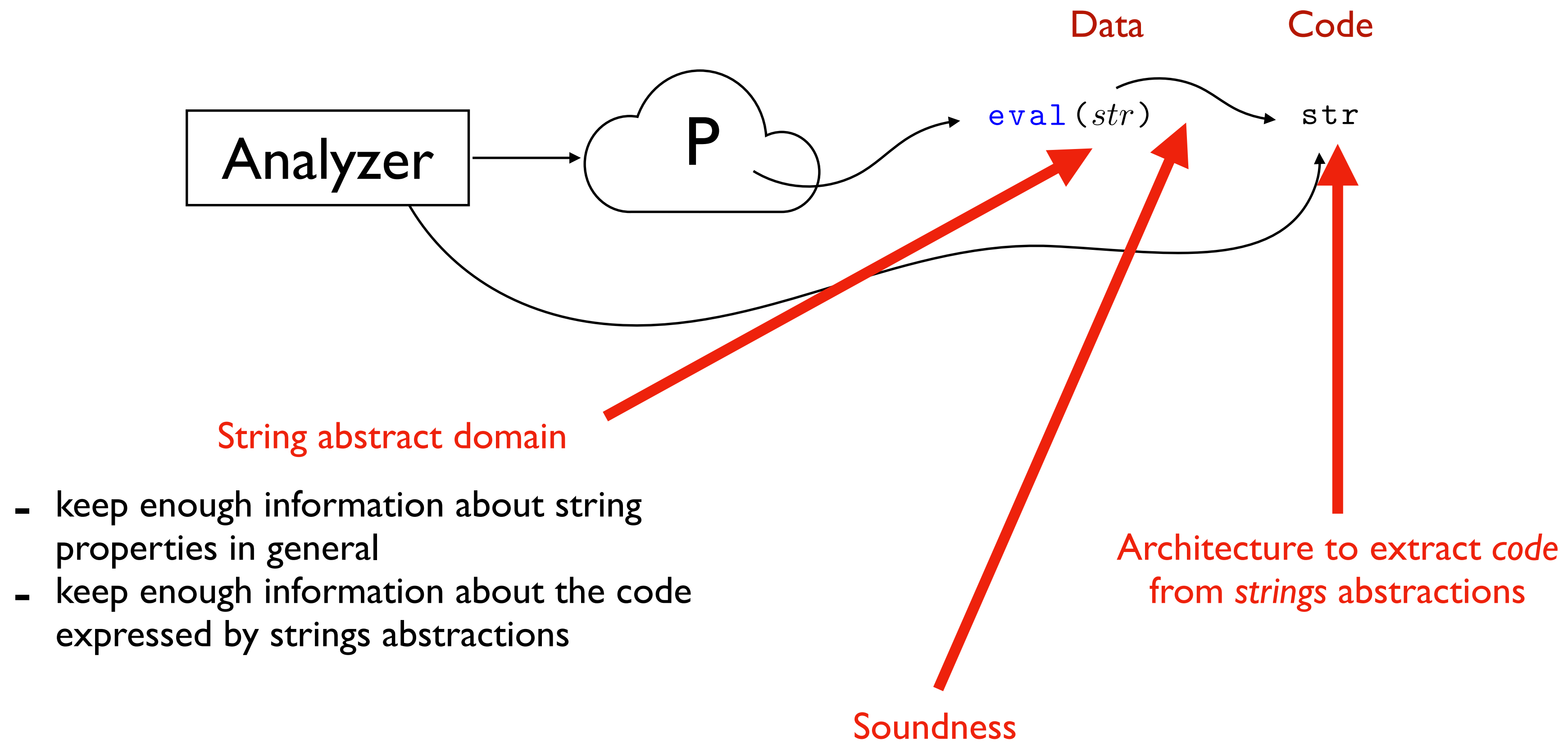
What we need?

Make static analysis *dynamic*



What we need?

Make static analysis *dynamic*



Any concrete execution of `eval(str)` must be taken into account by `str`

Finite state automata abstract domain

A sound static analyzer for eval

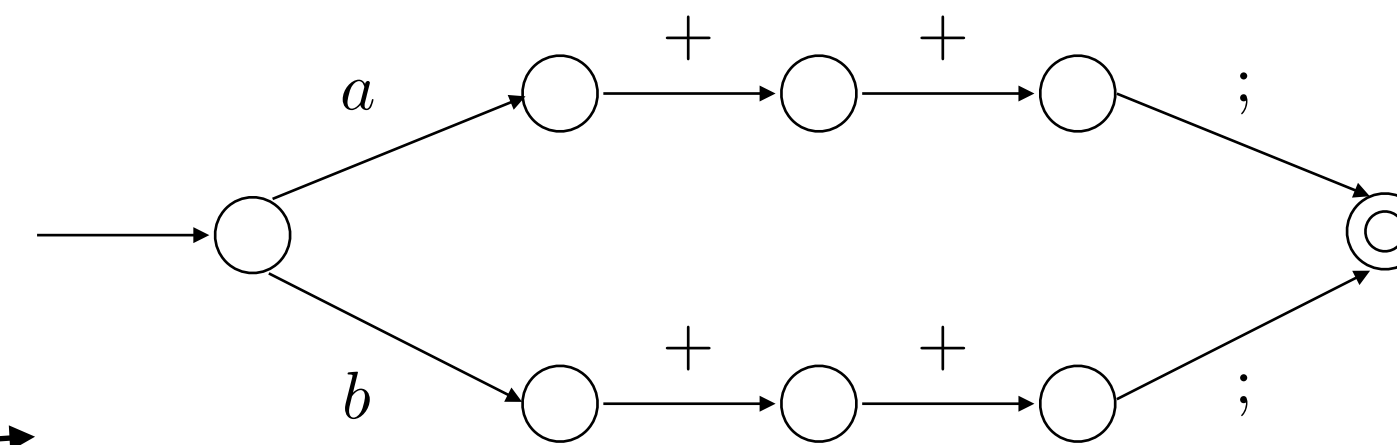
Static analysis of eval

```
if (...)
    x = "a++;"
else
    x = "b++;"

eval(x);
```

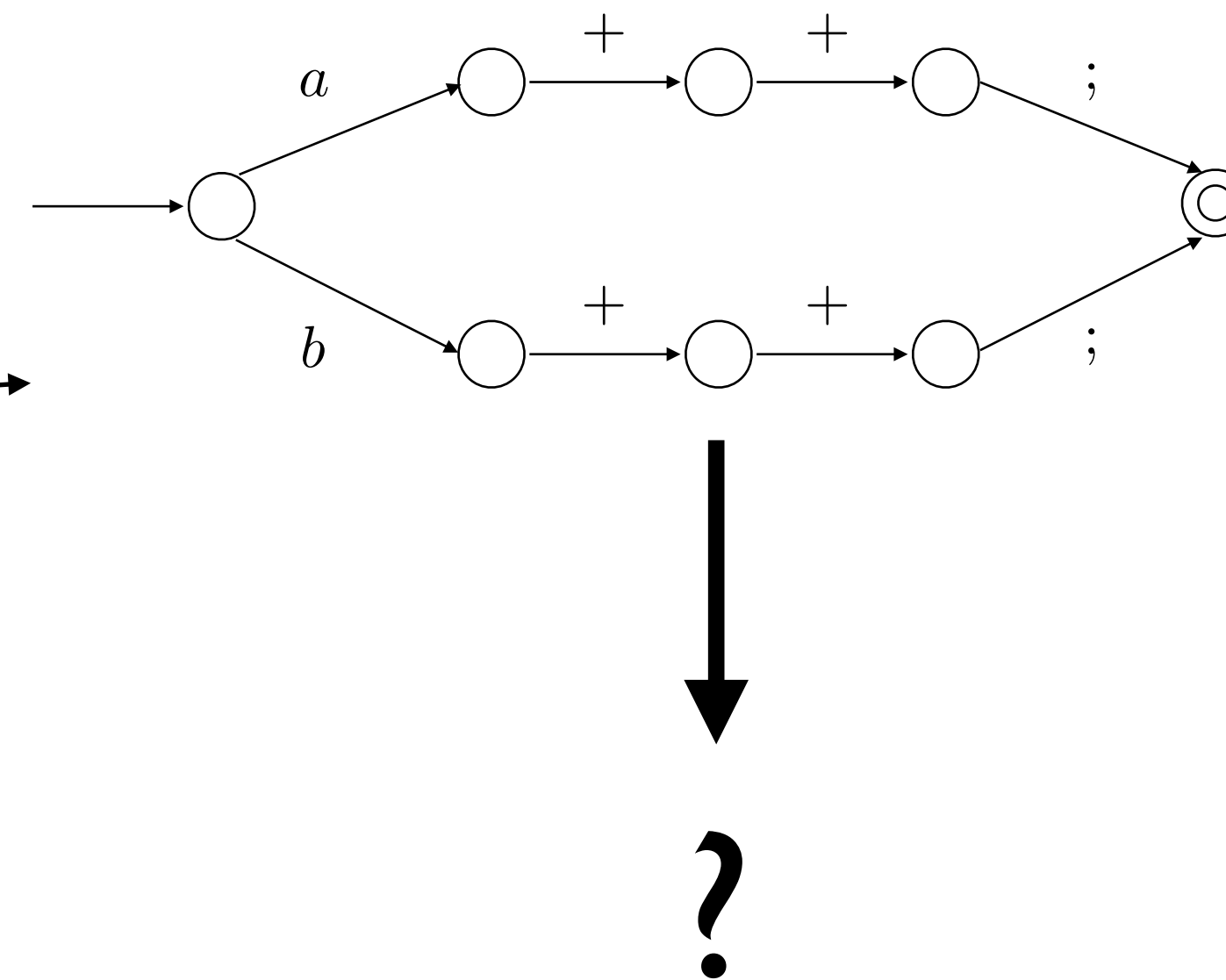
Static analysis of eval

```
if (...)  
    x = "a++;"  
else  
    x = "b++;"  
  
eval(x);
```



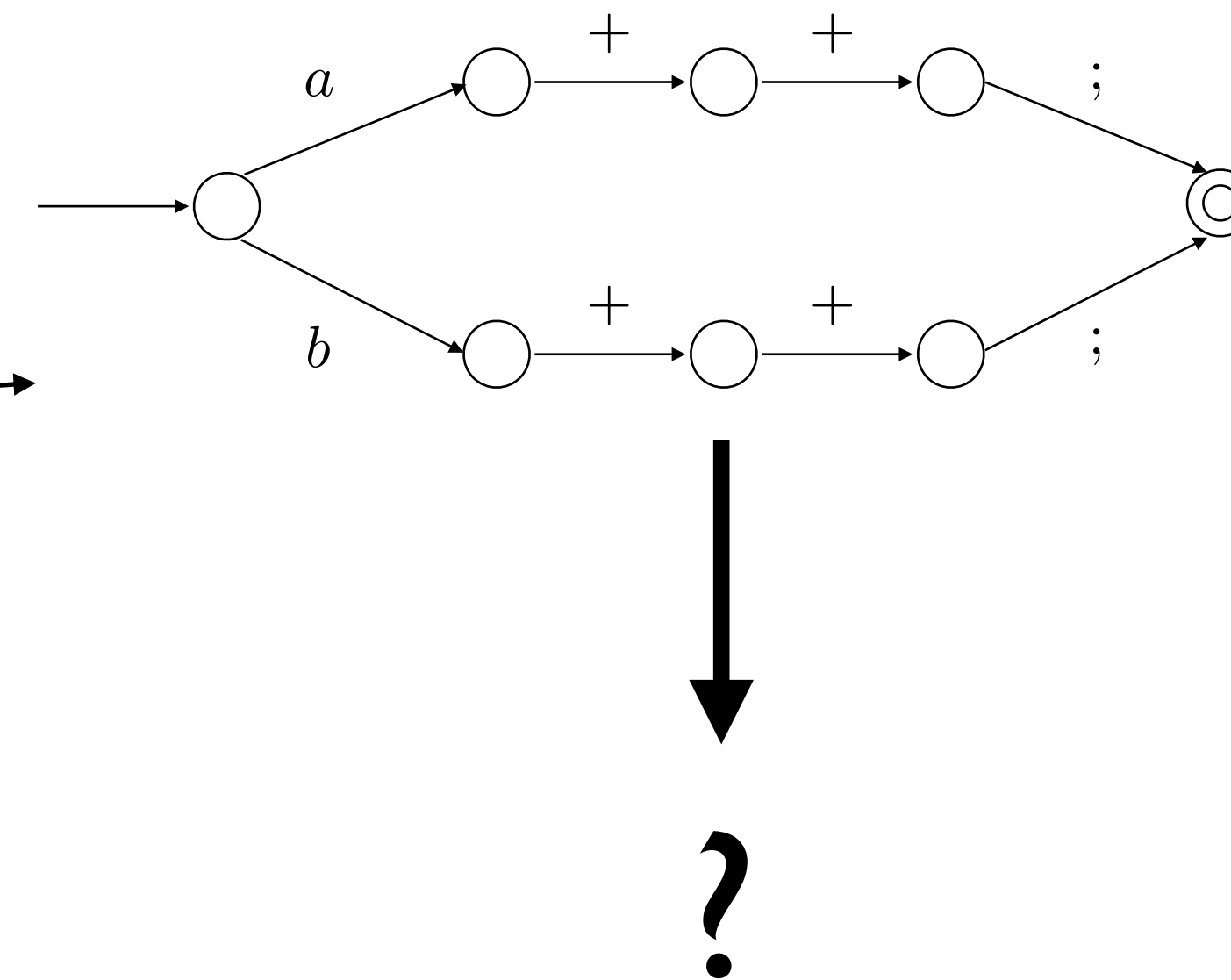
Static analysis of eval

```
if (...)  
    x = "a++;"  
else  
    x = "b++;"  
  
eval(x);
```



Static analysis of eval

```
if (...)  
    x = "a++;"  
else  
    x = "b++;"  
  
eval(x);
```



How to exploit automata for analyzing eval?

Static analysis of eval

```
while (x++ < 3)
    os = os + "x=x+1;y=10;x=x+1;";

if (x > 10)
    os = "while(x>5){x=x+1;y=x;}";

if (x == 5)
    os = "hello";

if (x == 8)
    os = "while(x;";

eval(os);
```

The value of x is statically unknown

Static analysis of eval

```
while (x++ < 3)
  os = os + "x=x+1;y=10;x=x+1;";

if (x > 10)
  os = "while(x>5){x=x+1;y=x;}";

if (x == 5)
  os = "hello";

if (x == 8)
  os = "while(x;";

eval(os);
```

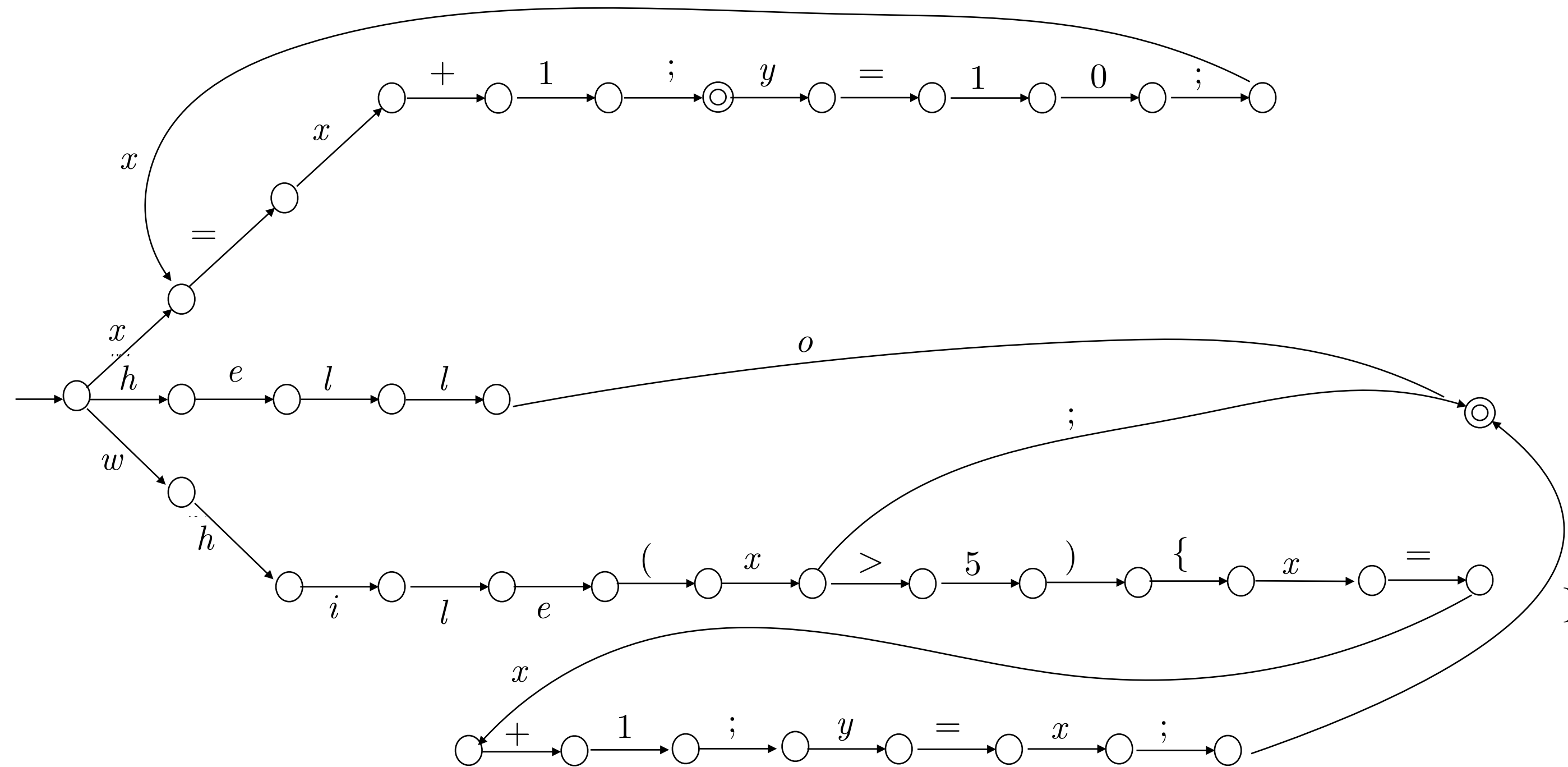


Not executable

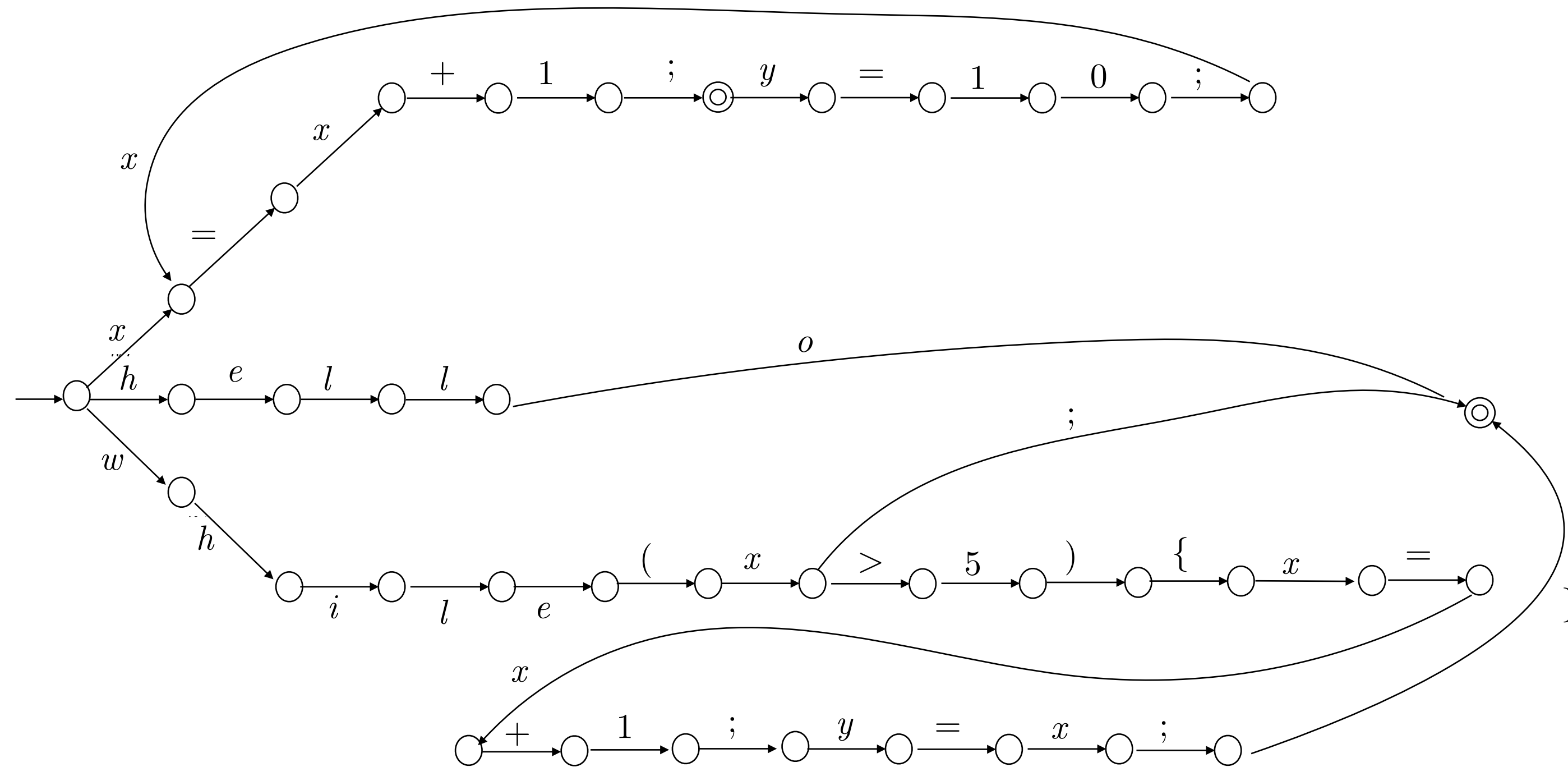


The value of x is statically unknown

Static analysis of eval

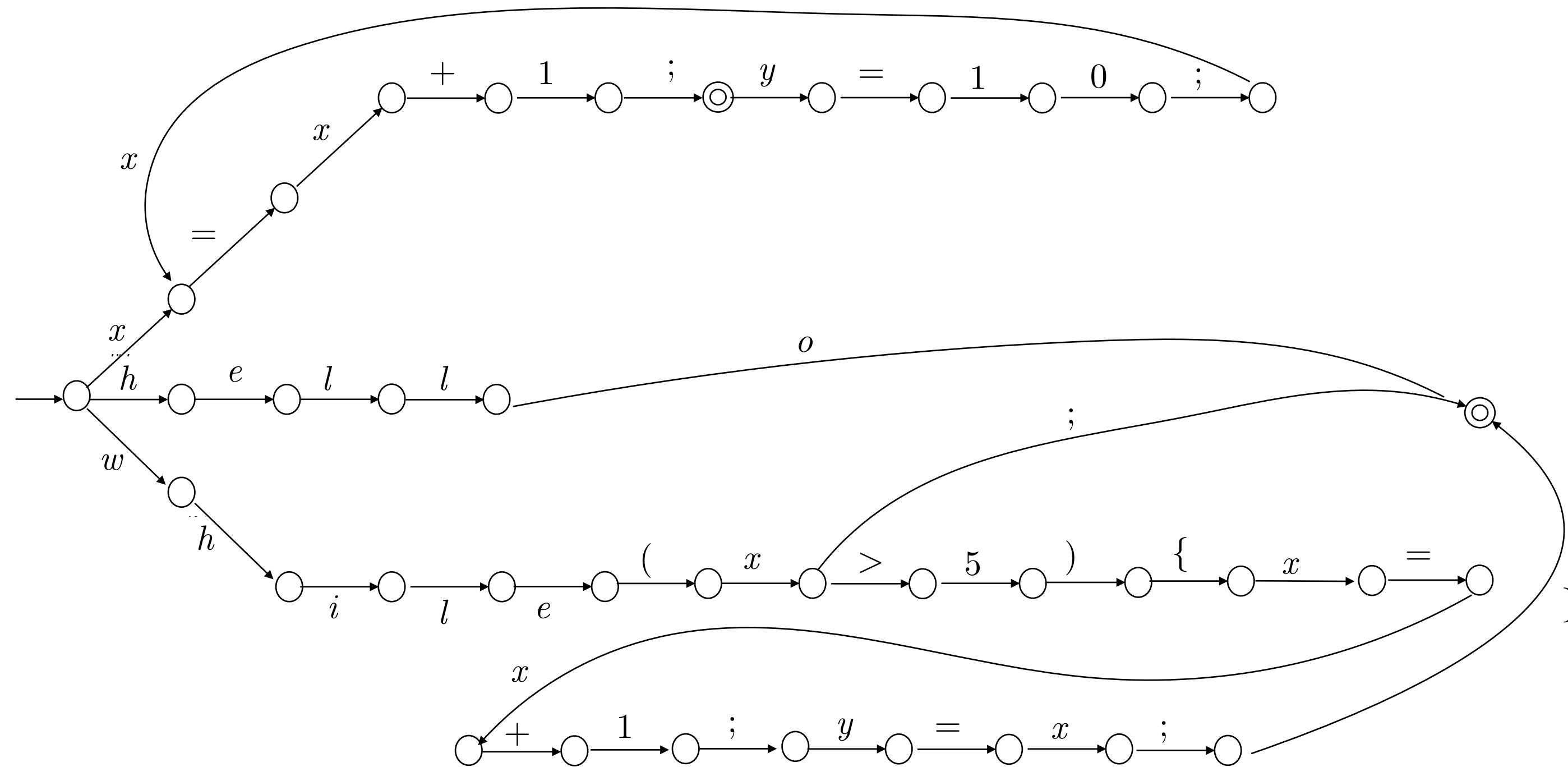


Static analysis of eval



We transform an automaton over the alphabet of characters into an automaton over the alphabet of partial partial statement is a prefix statement ending with a punctuation symbol or a punctuation symbol

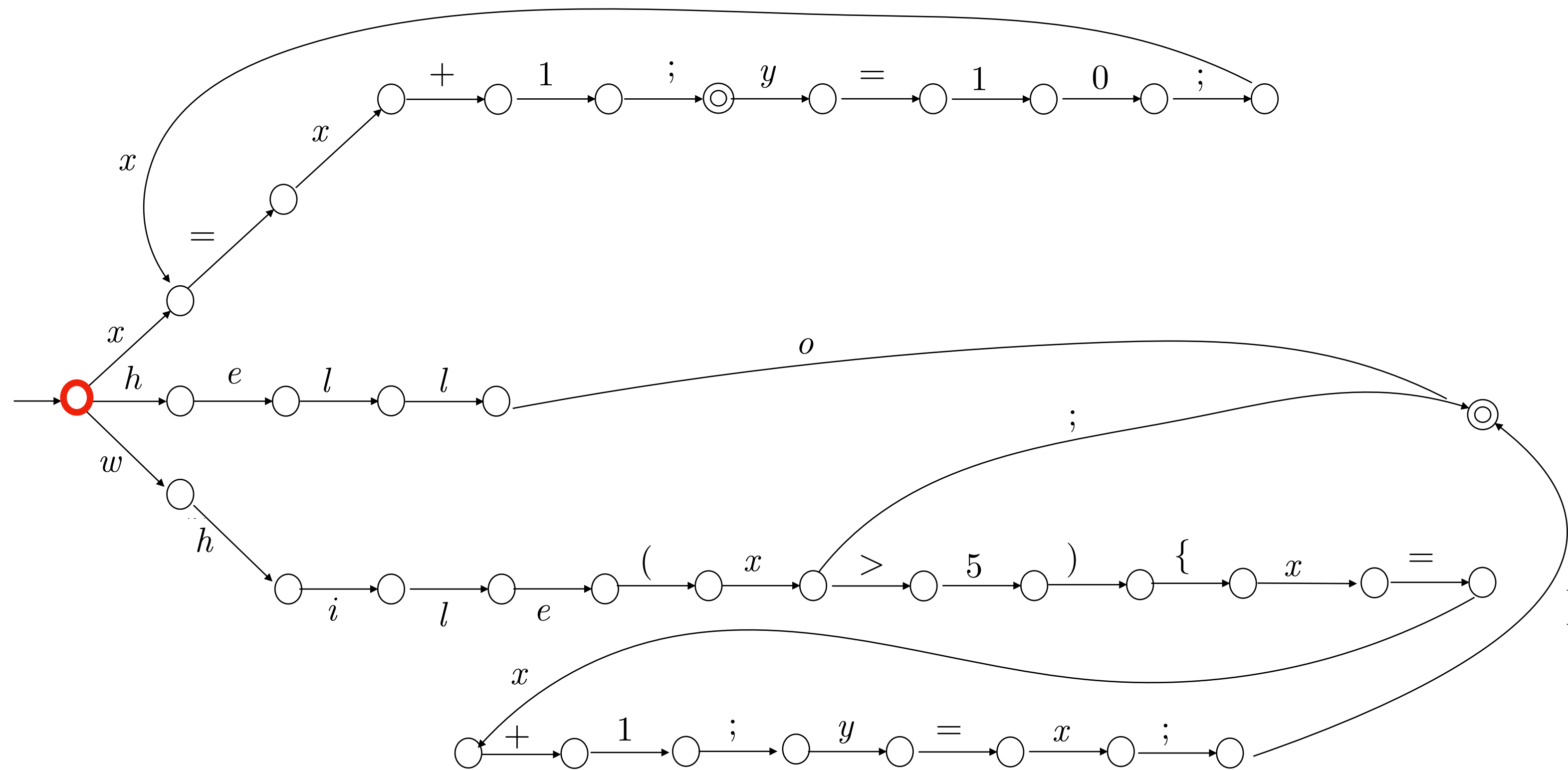
Static analysis of eval



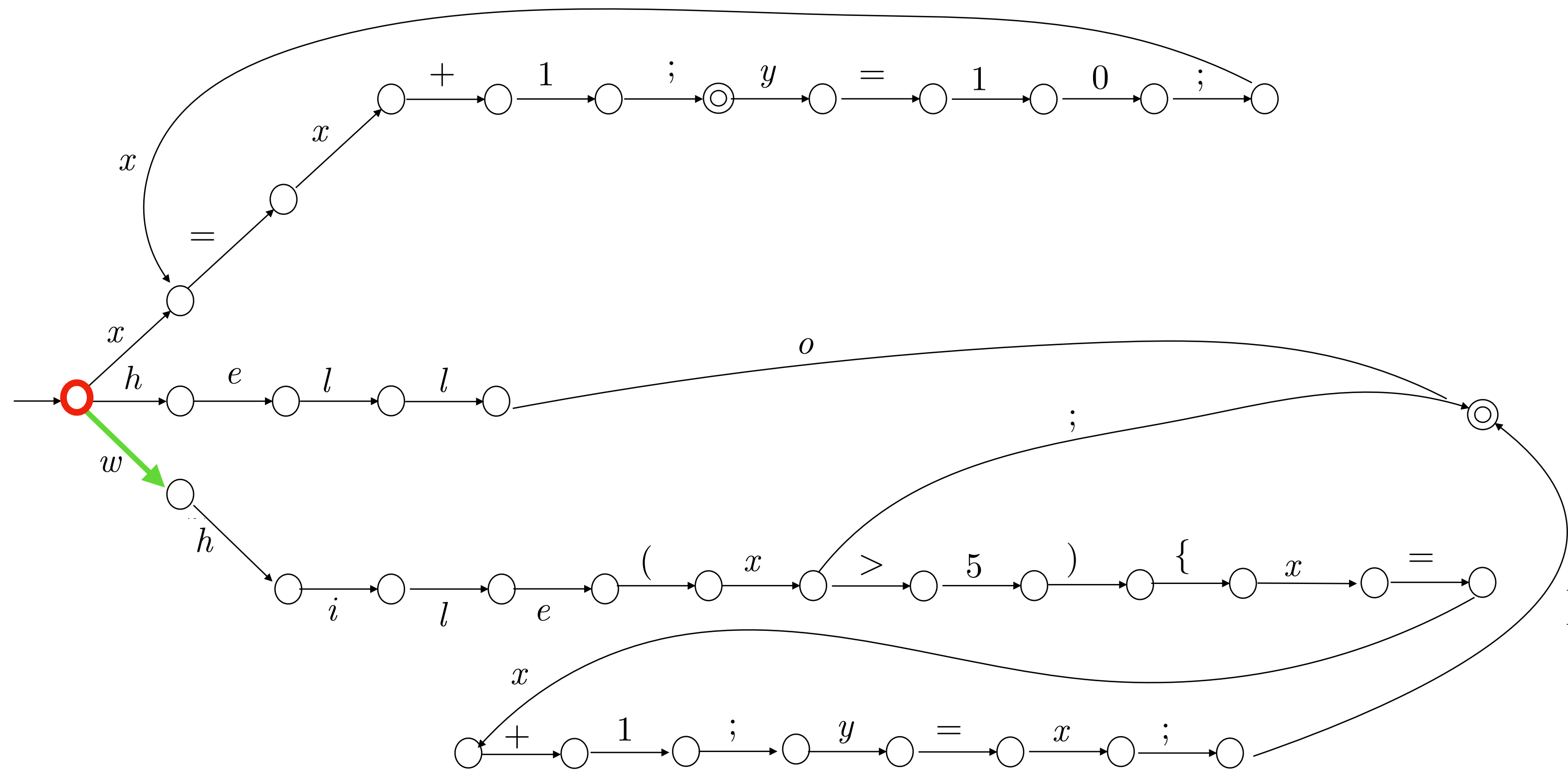
We transform an automaton over the alphabet of characters into an automaton over the alphabet of partial partial statement is a prefix statement ending with a punctuation symbol or a punctuation symbol

`x=x+1;` `while (x<5) {` `}`

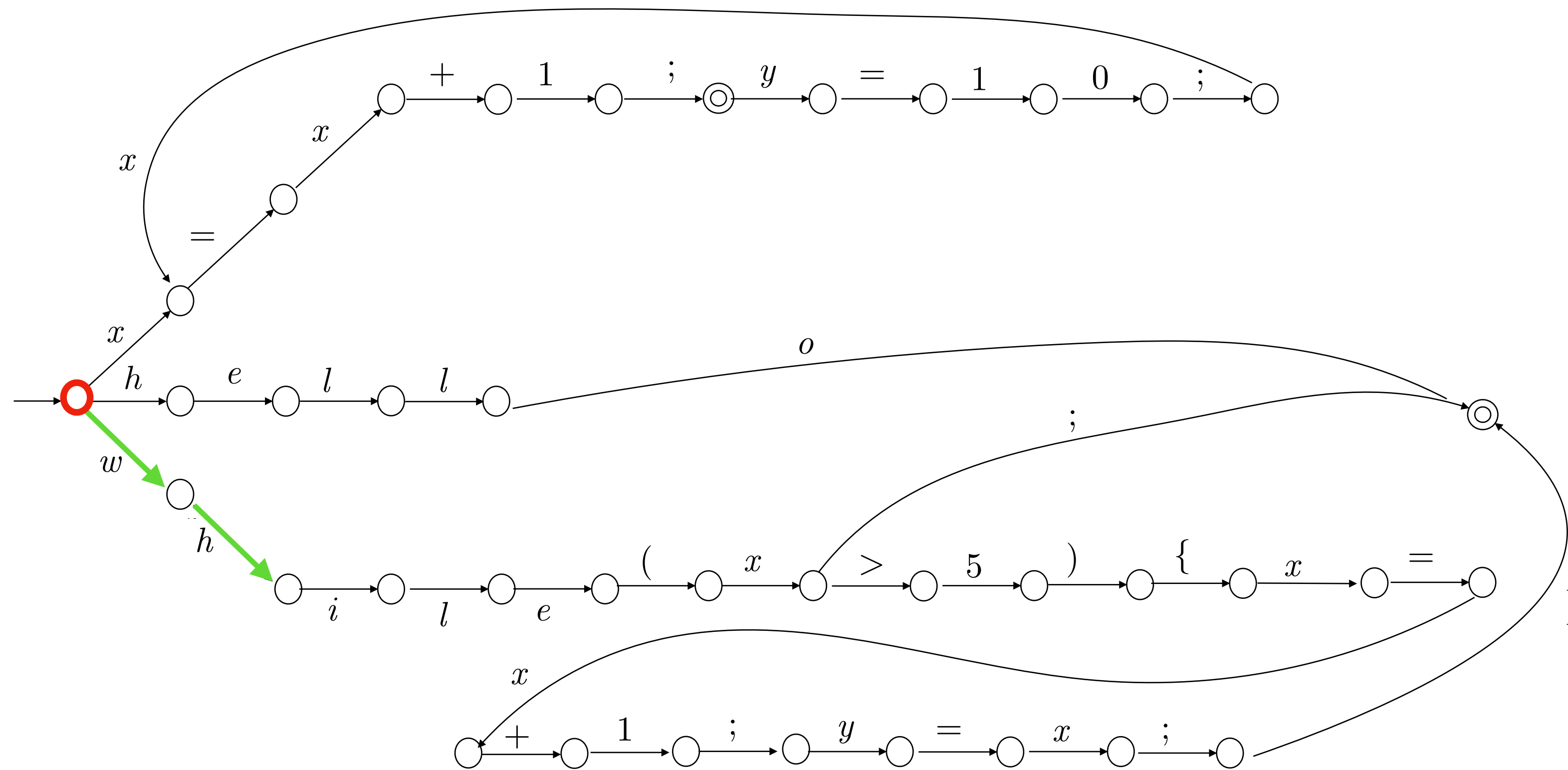
Bonus: Static analysis of eval



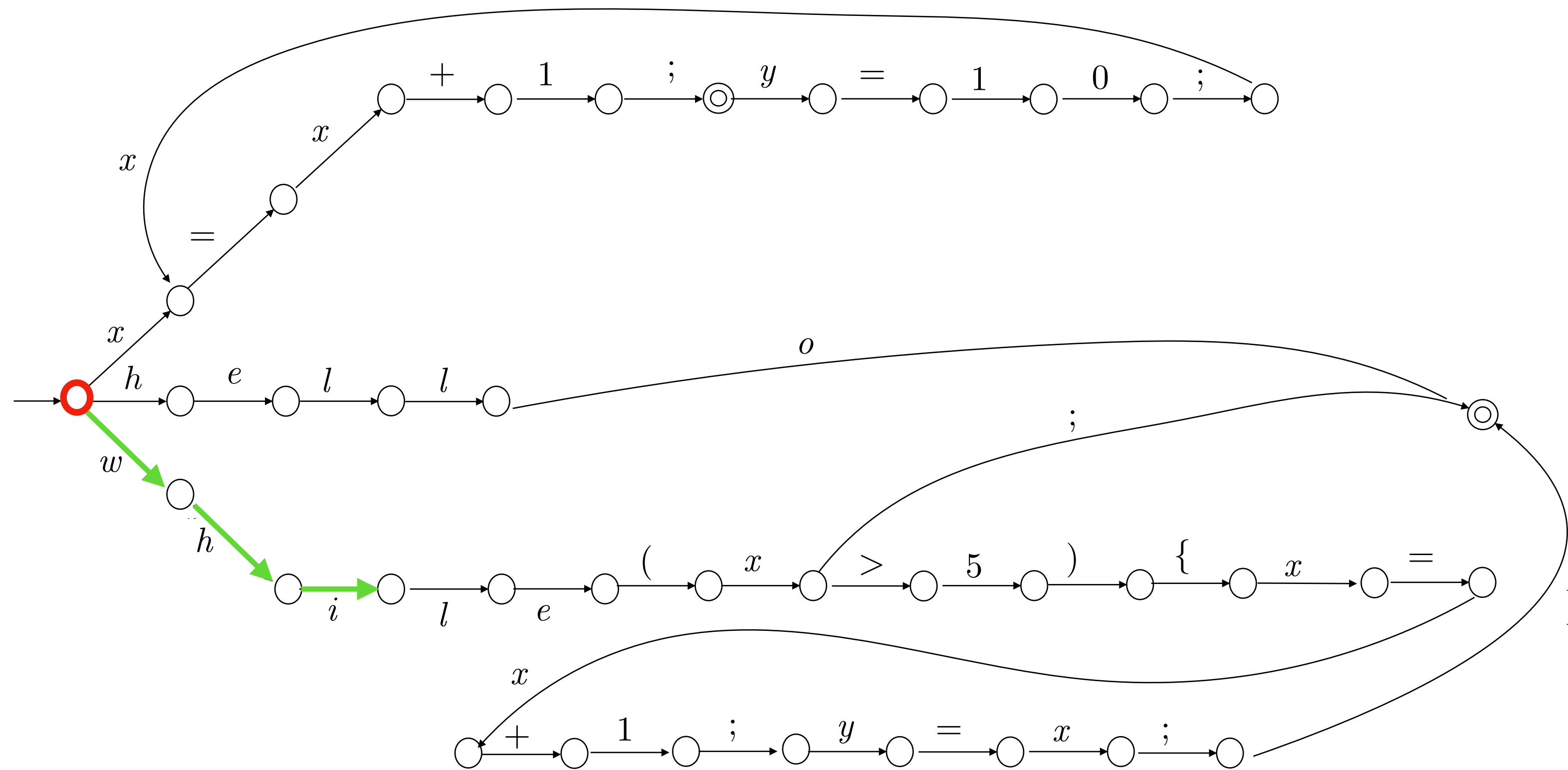
Bonus: Static analysis of eval



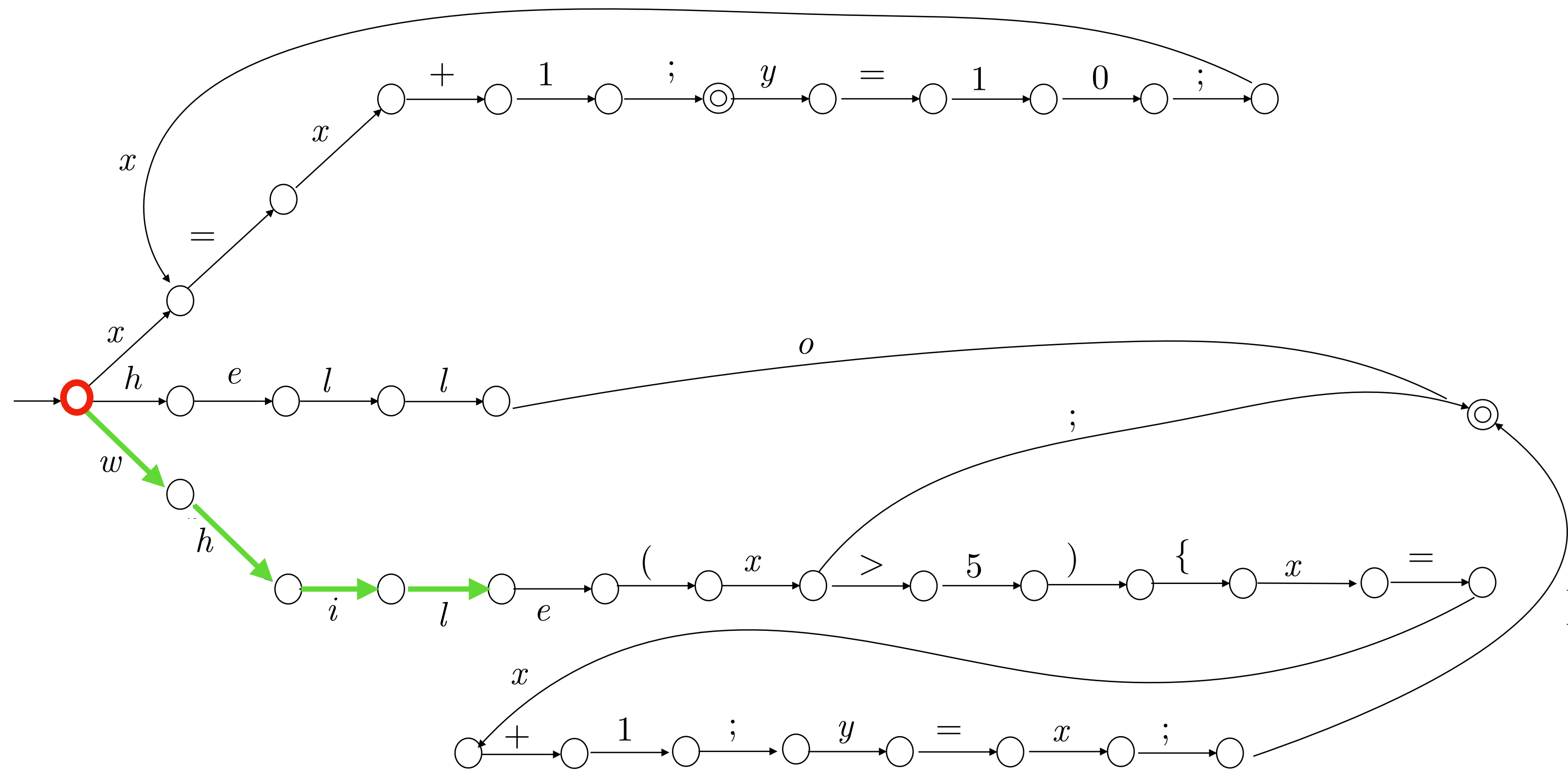
Bonus: Static analysis of eval



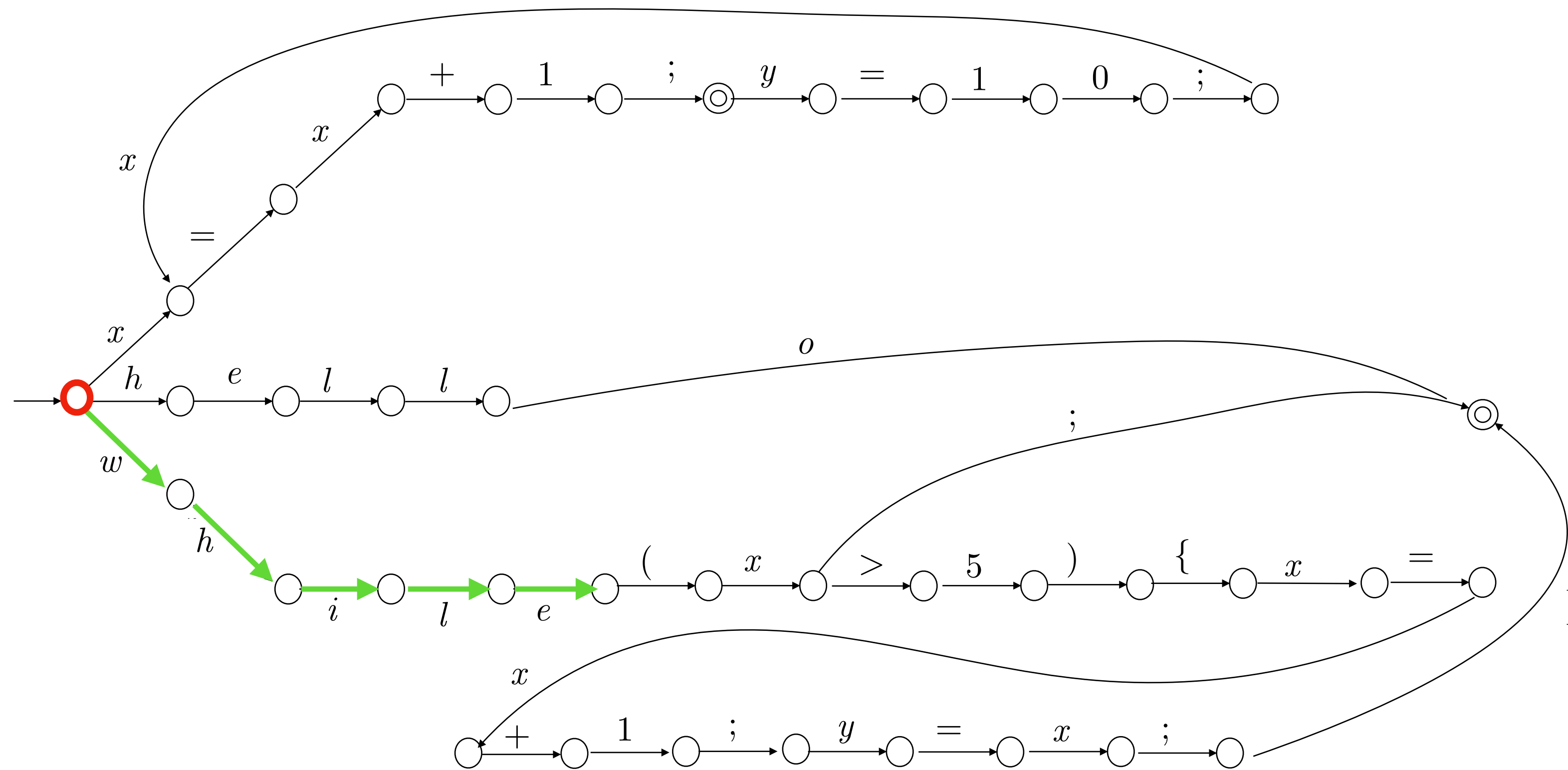
Bonus: Static analysis of eval



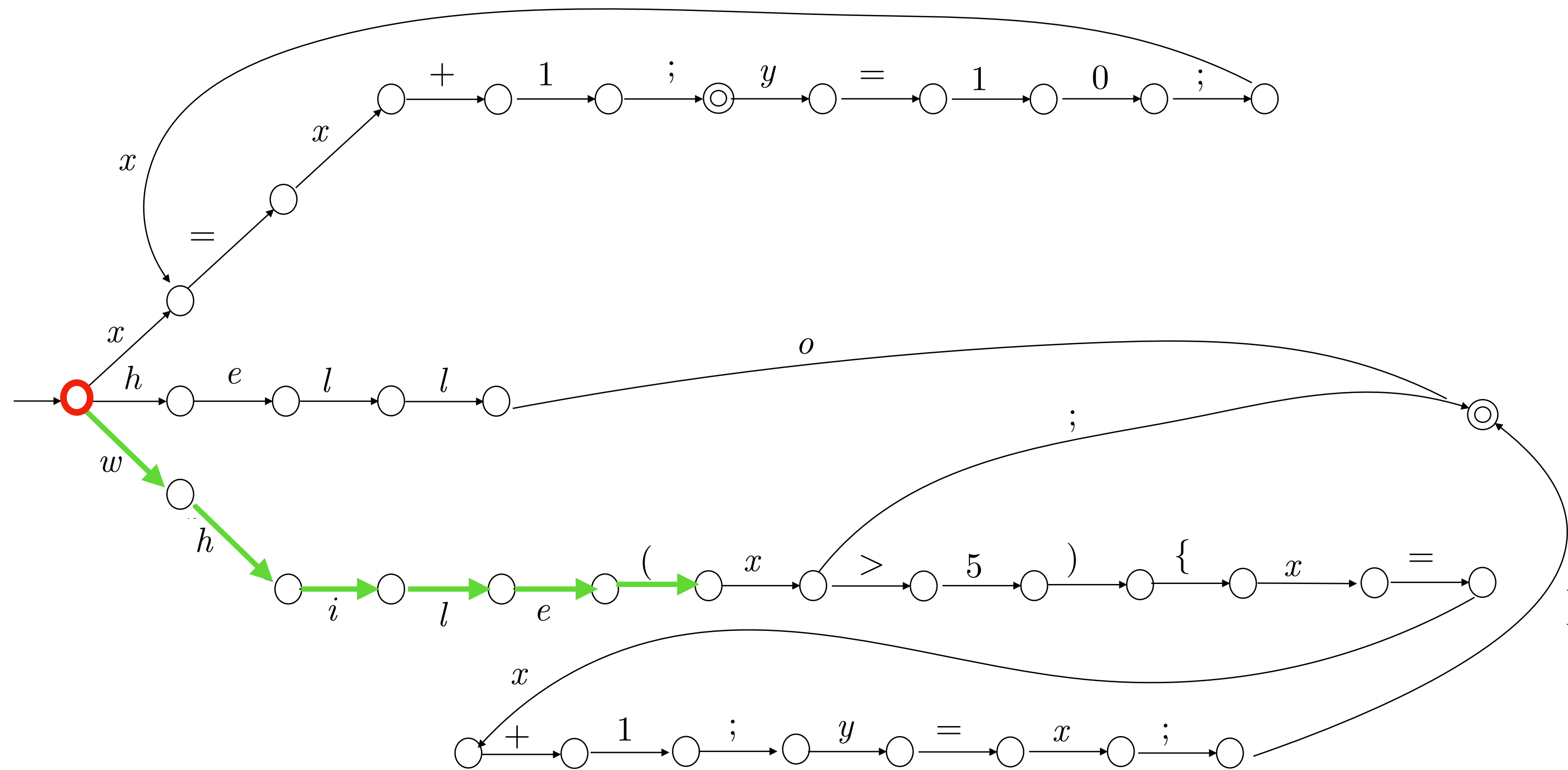
Bonus: Static analysis of eval



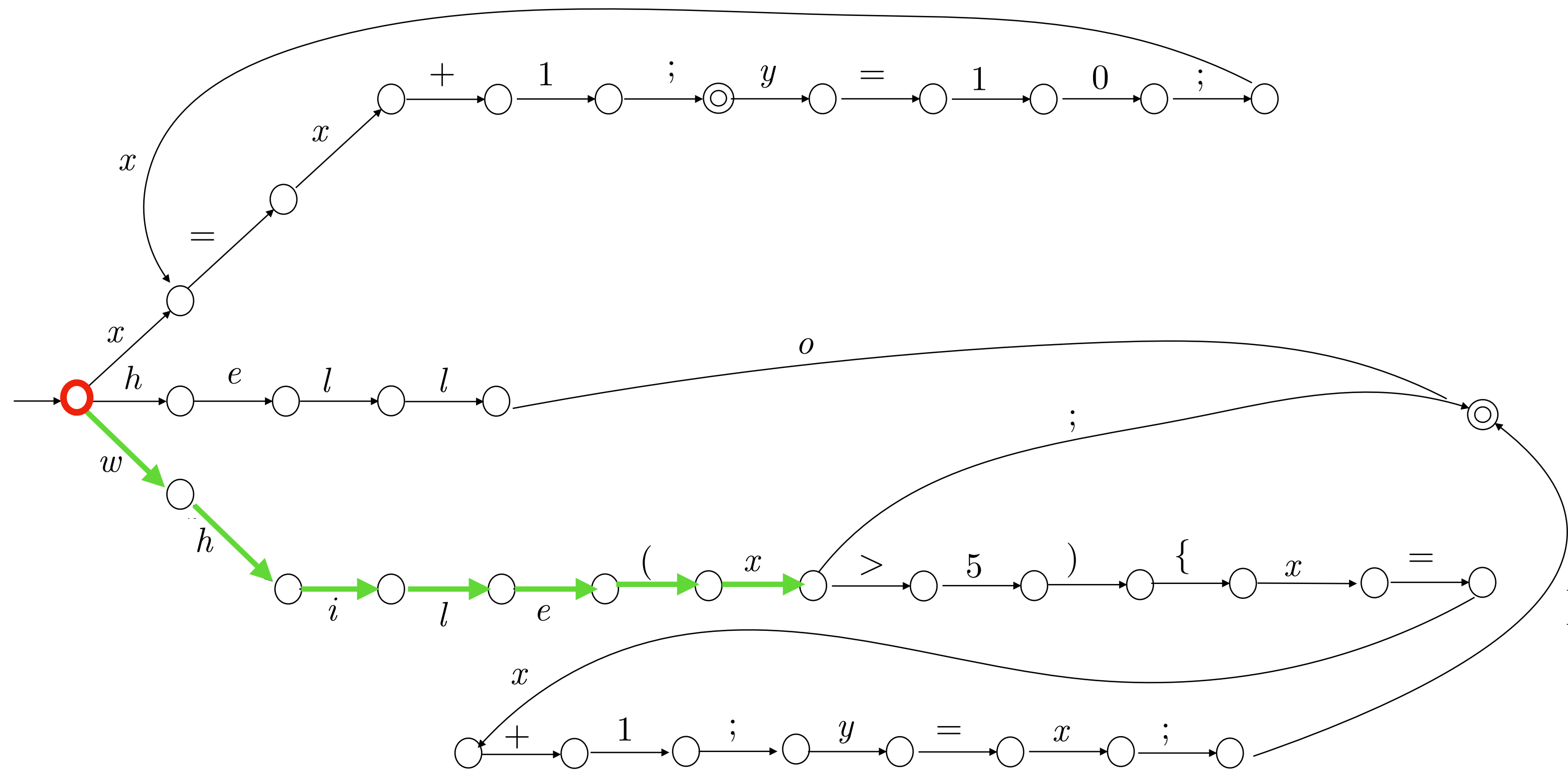
Bonus: Static analysis of eval



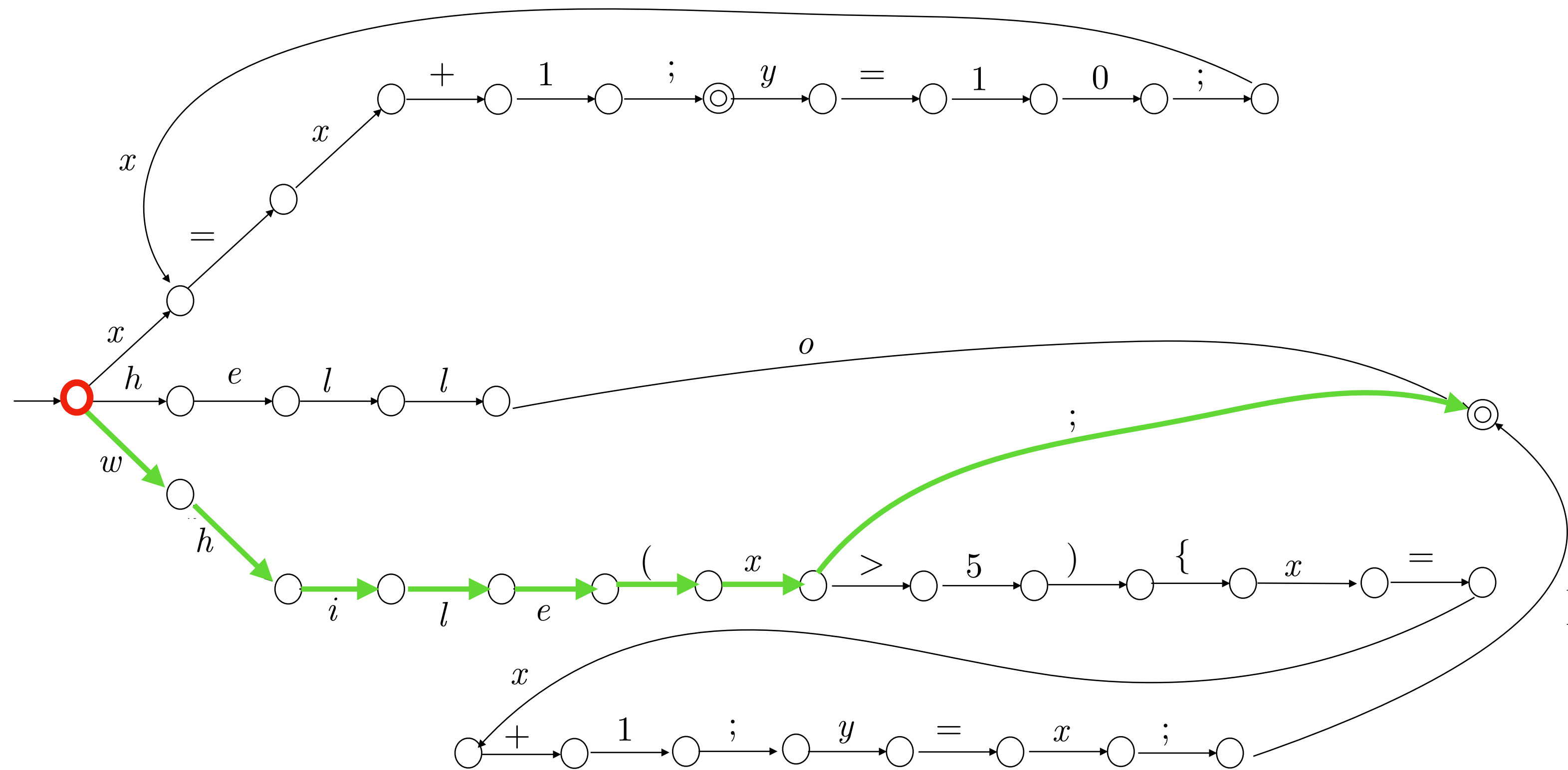
Bonus: Static analysis of eval



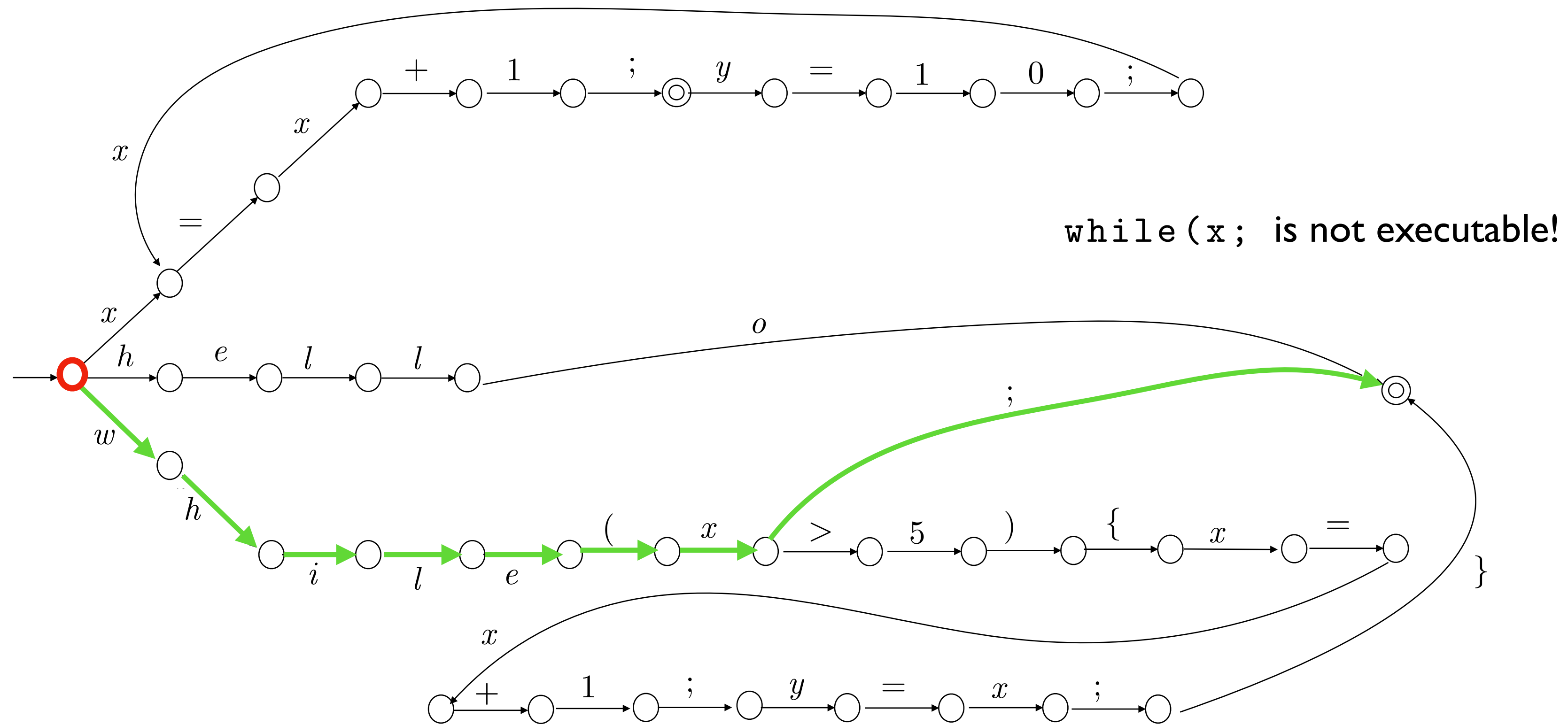
Bonus: Static analysis of eval



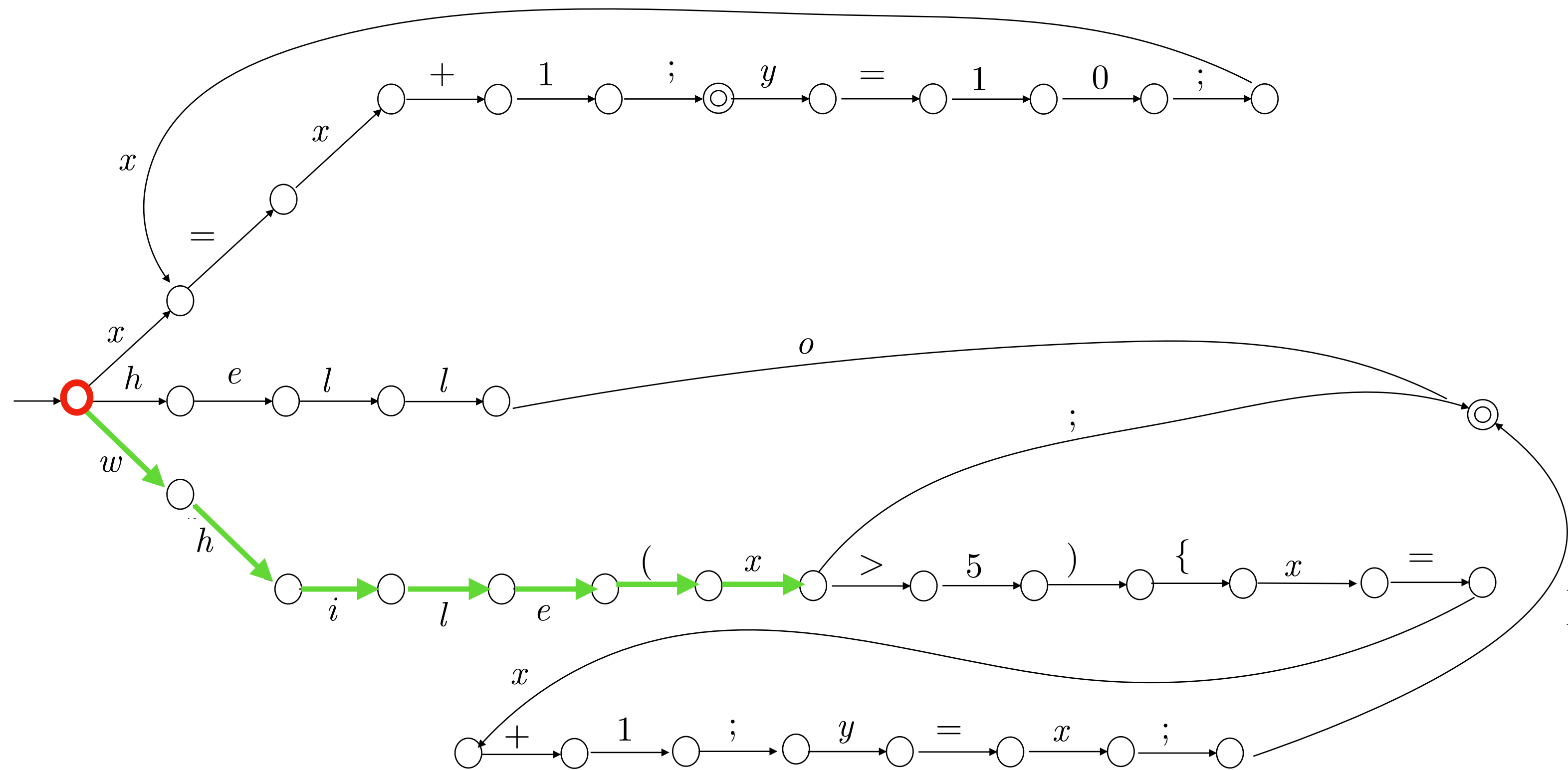
Bonus: Static analysis of eval



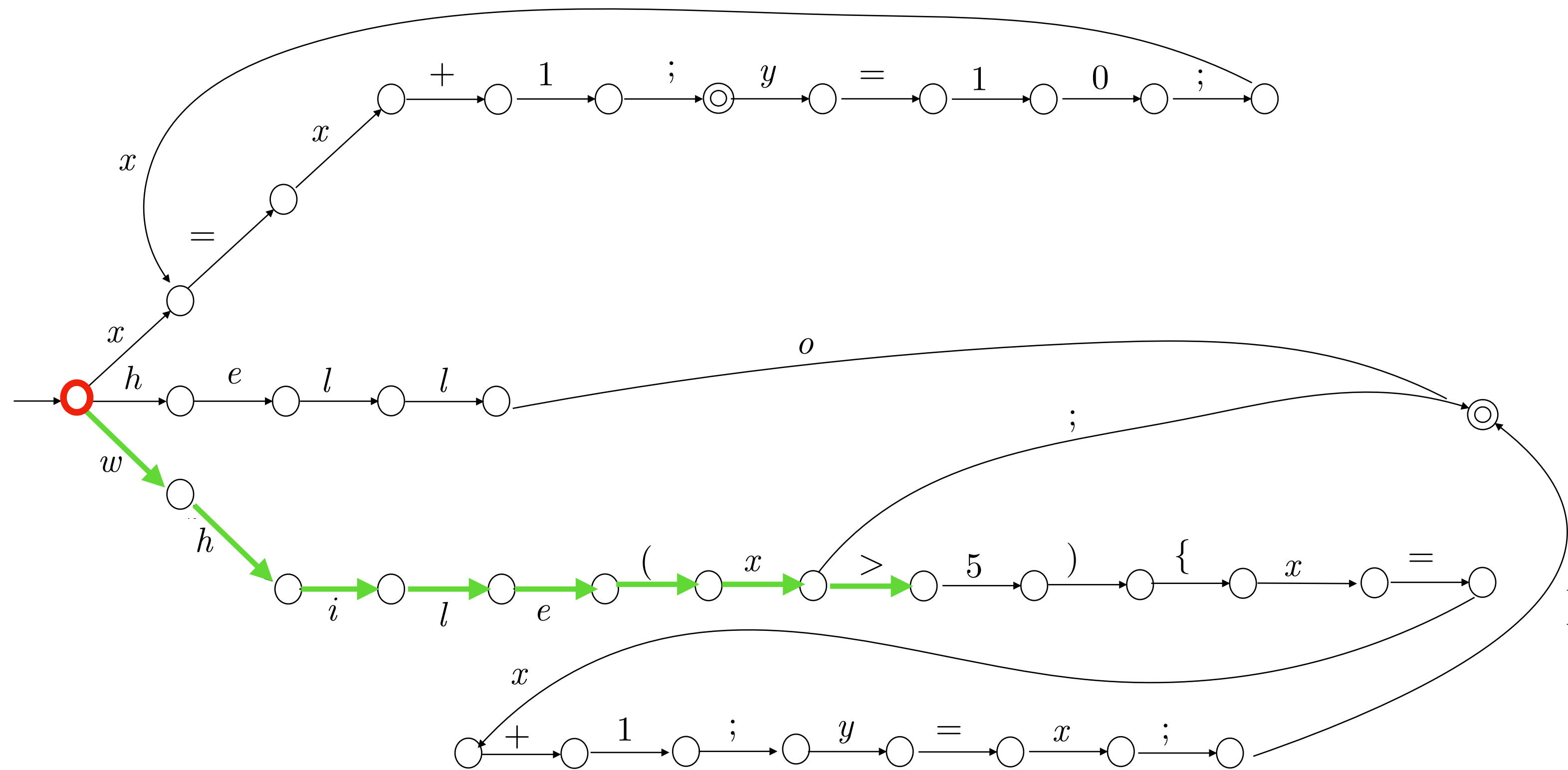
Bonus: Static analysis of eval



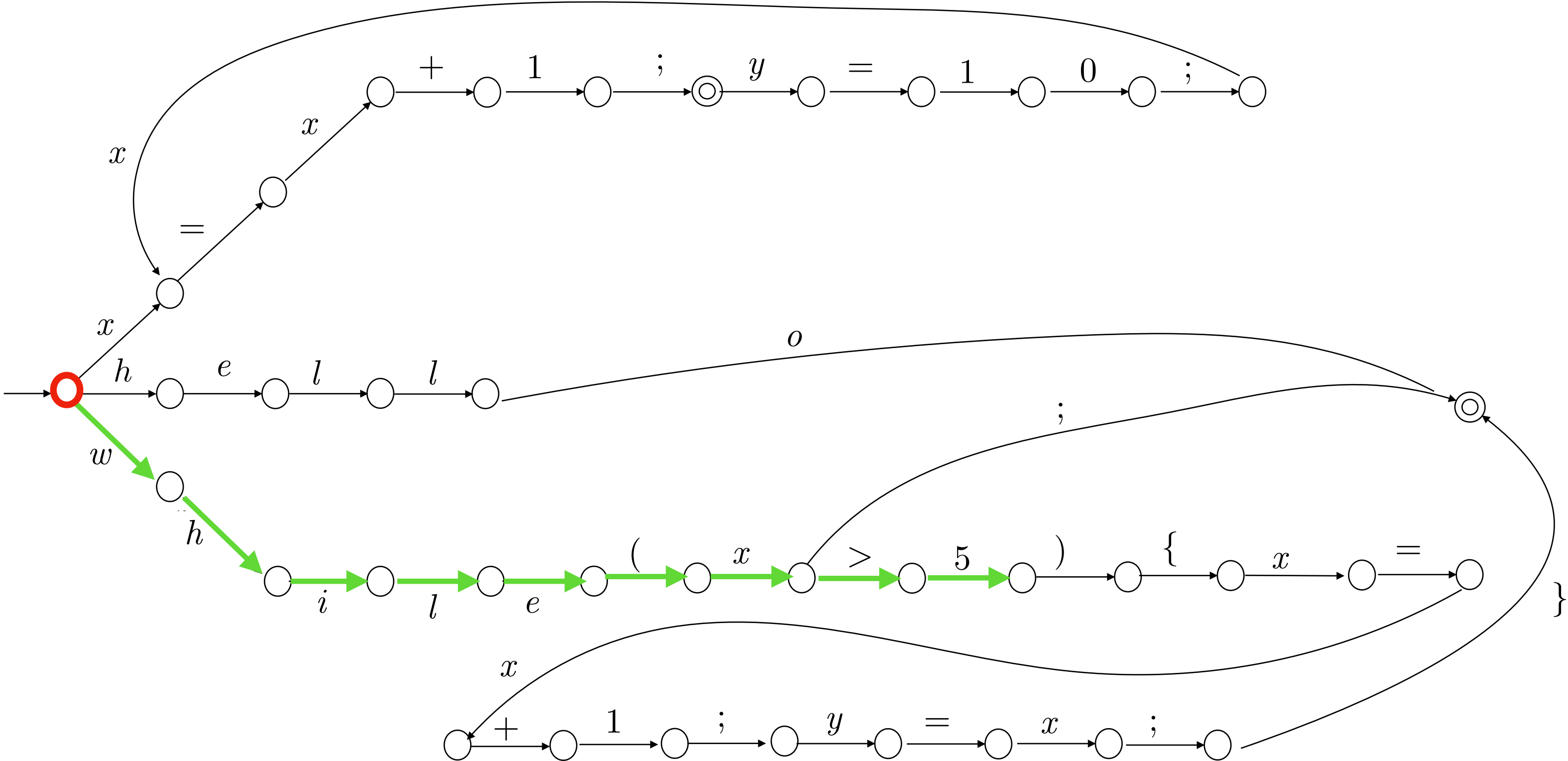
Bonus: Static analysis of eval



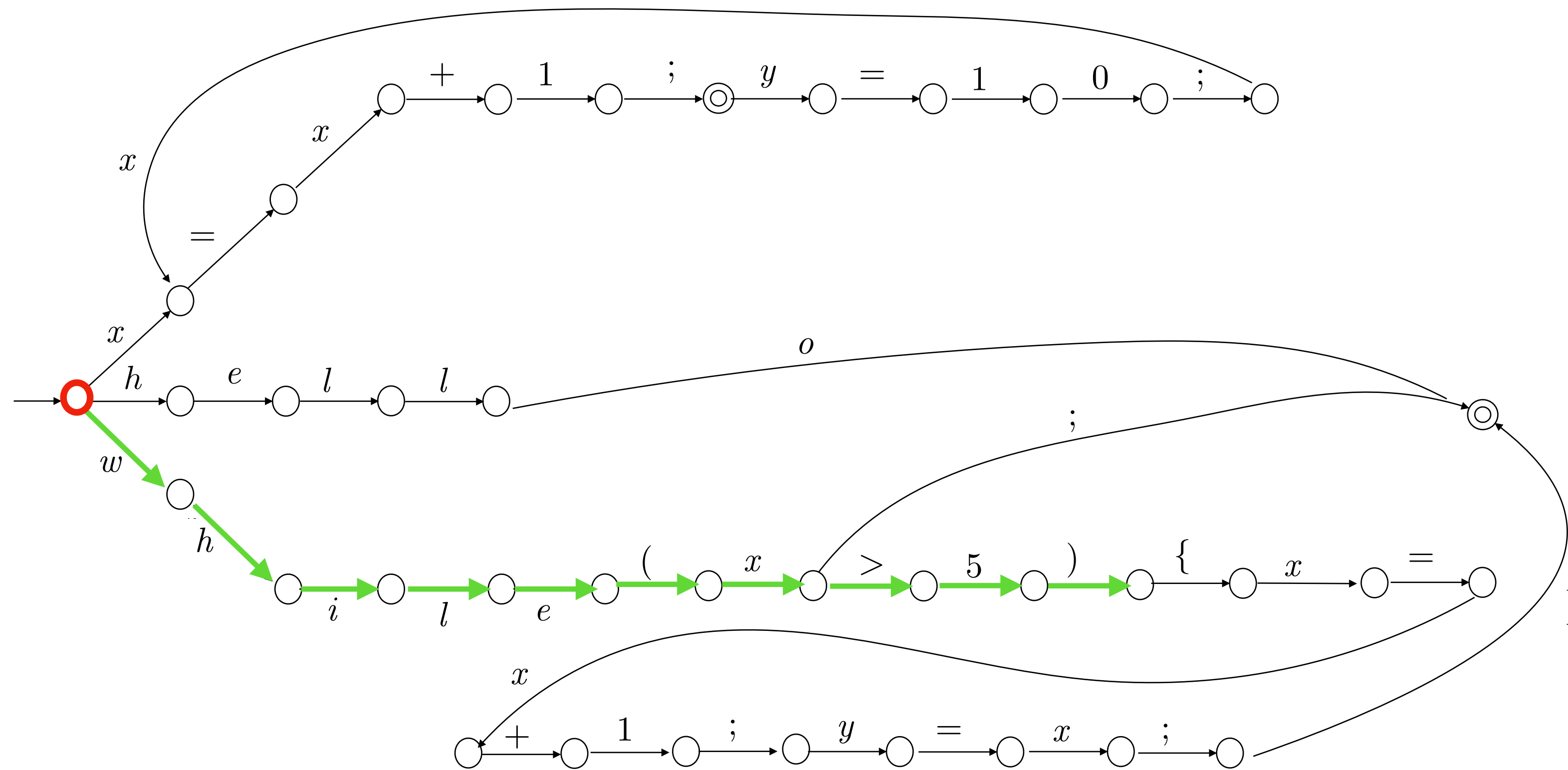
Bonus: Static analysis of eval



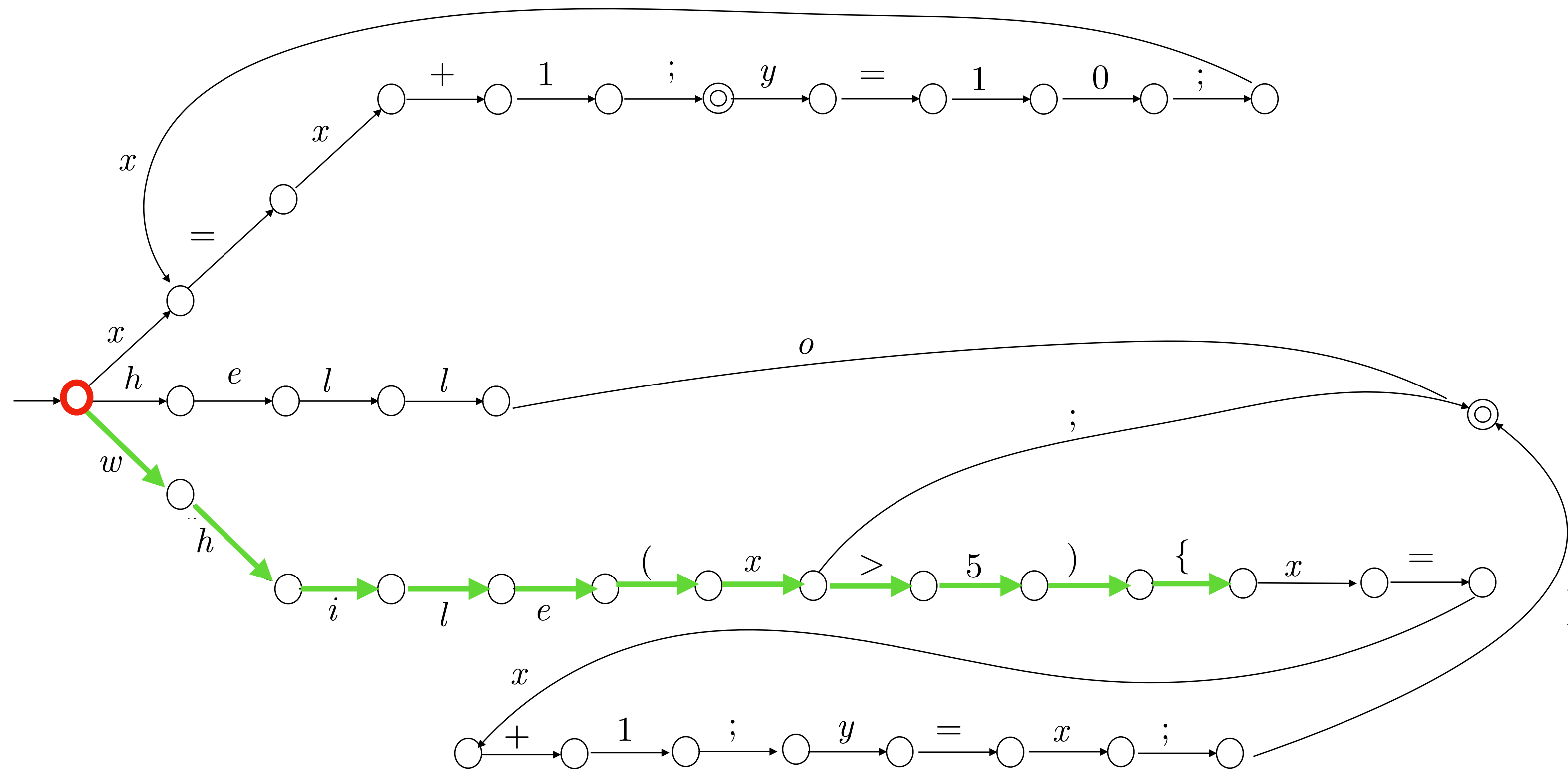
Bonus: Static analysis of eval



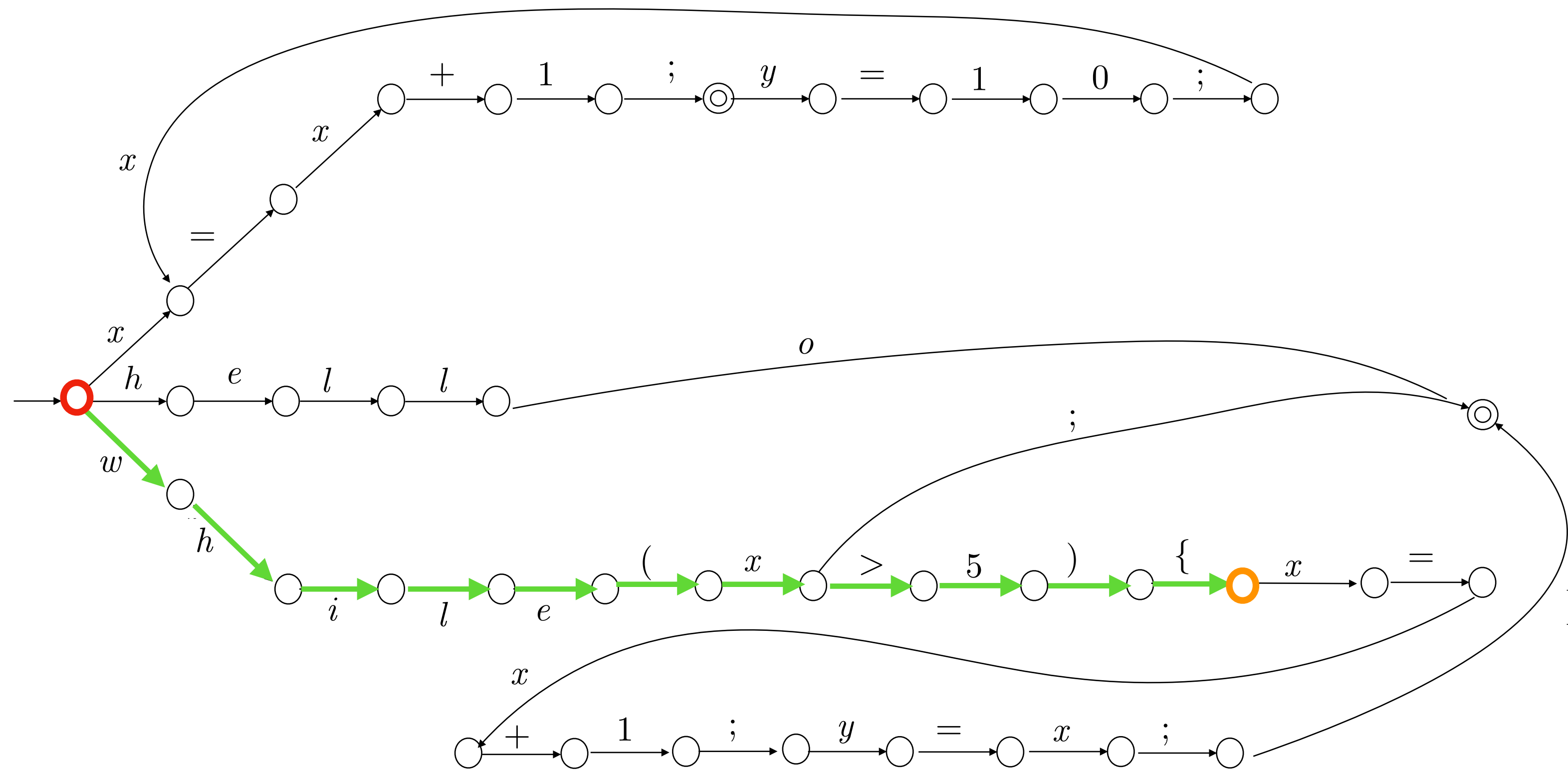
Bonus: Static analysis of eval



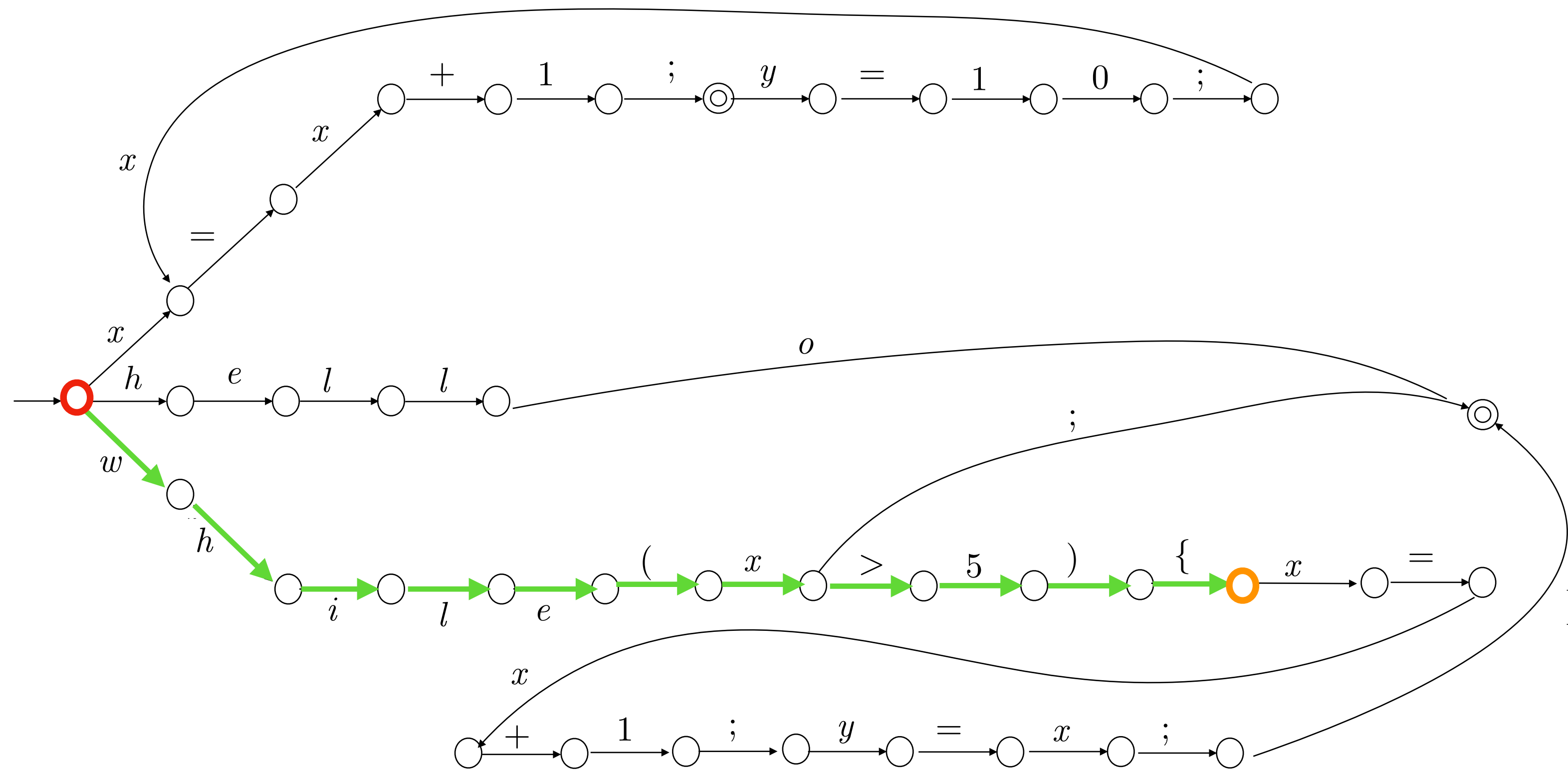
Bonus: Static analysis of eval



Bonus: Static analysis of eval

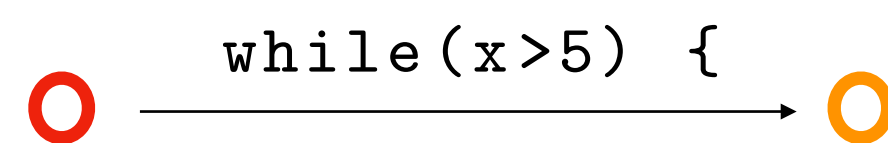
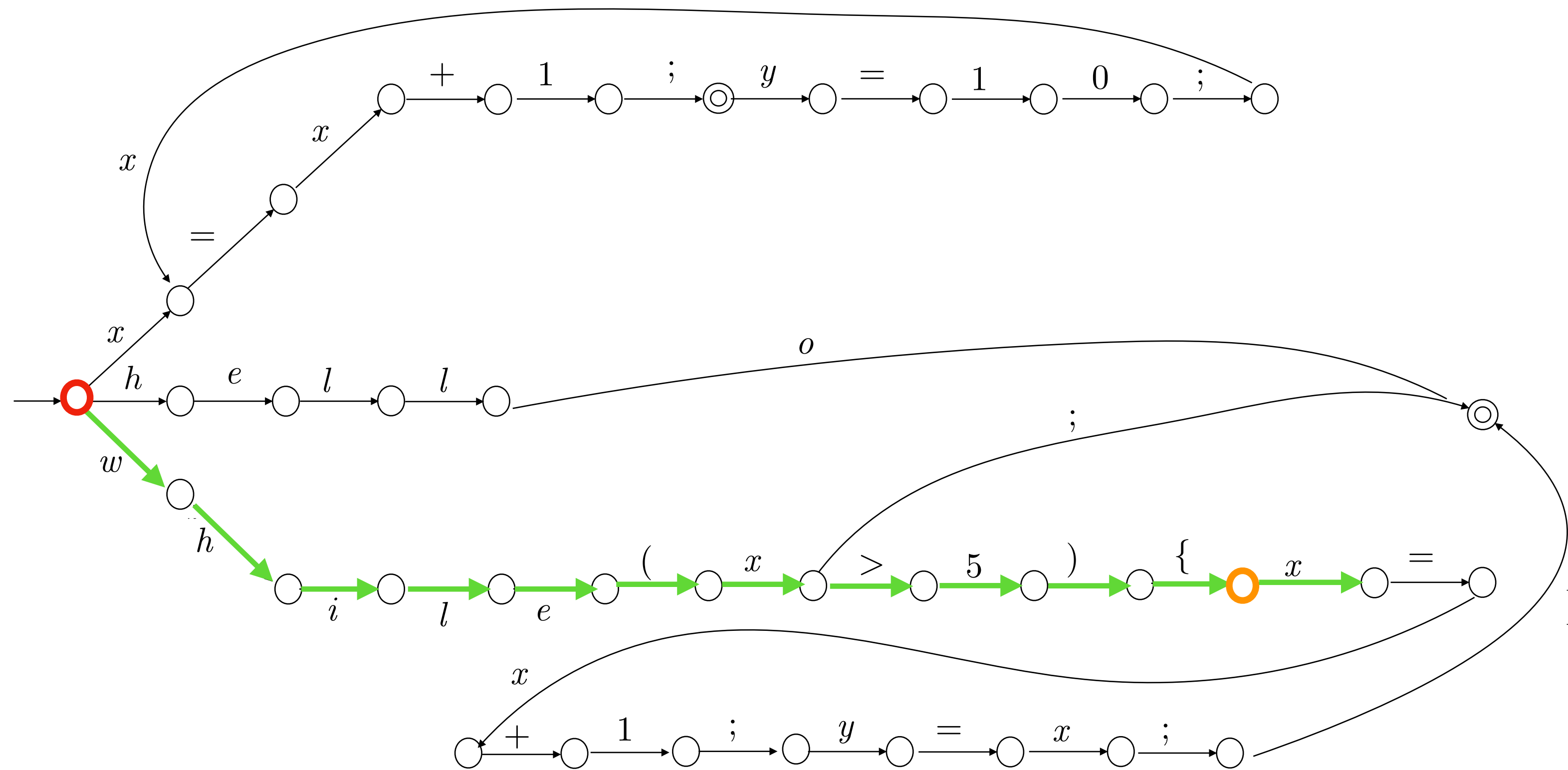


Bonus: Static analysis of eval

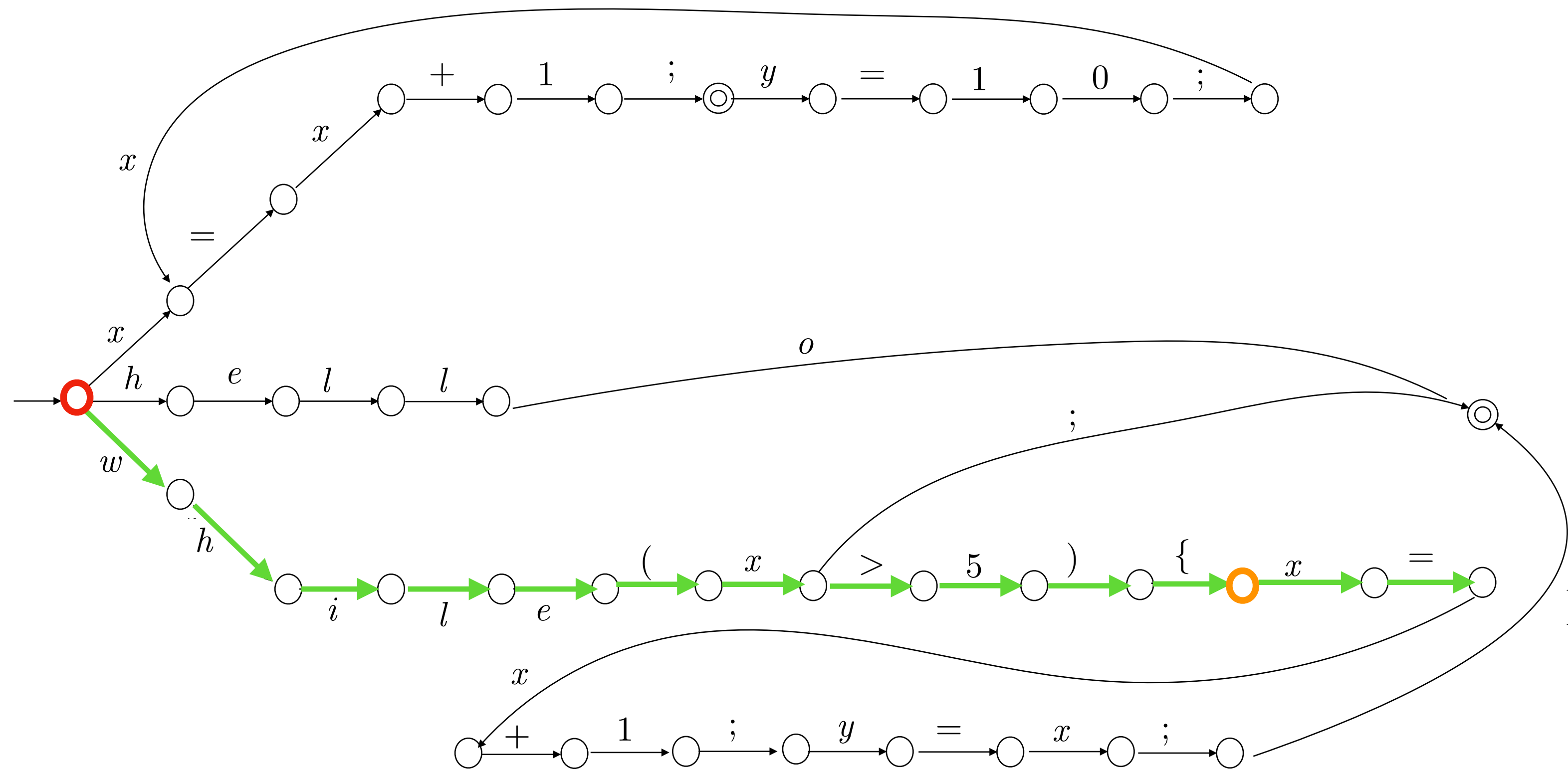


○ `while(x>5) {` ○

Bonus: Static analysis of eval

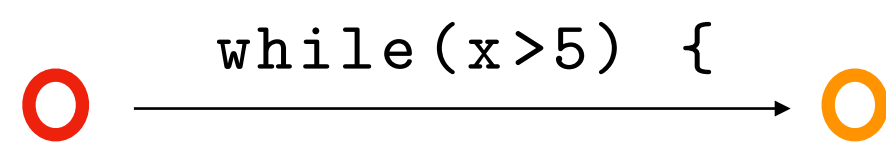
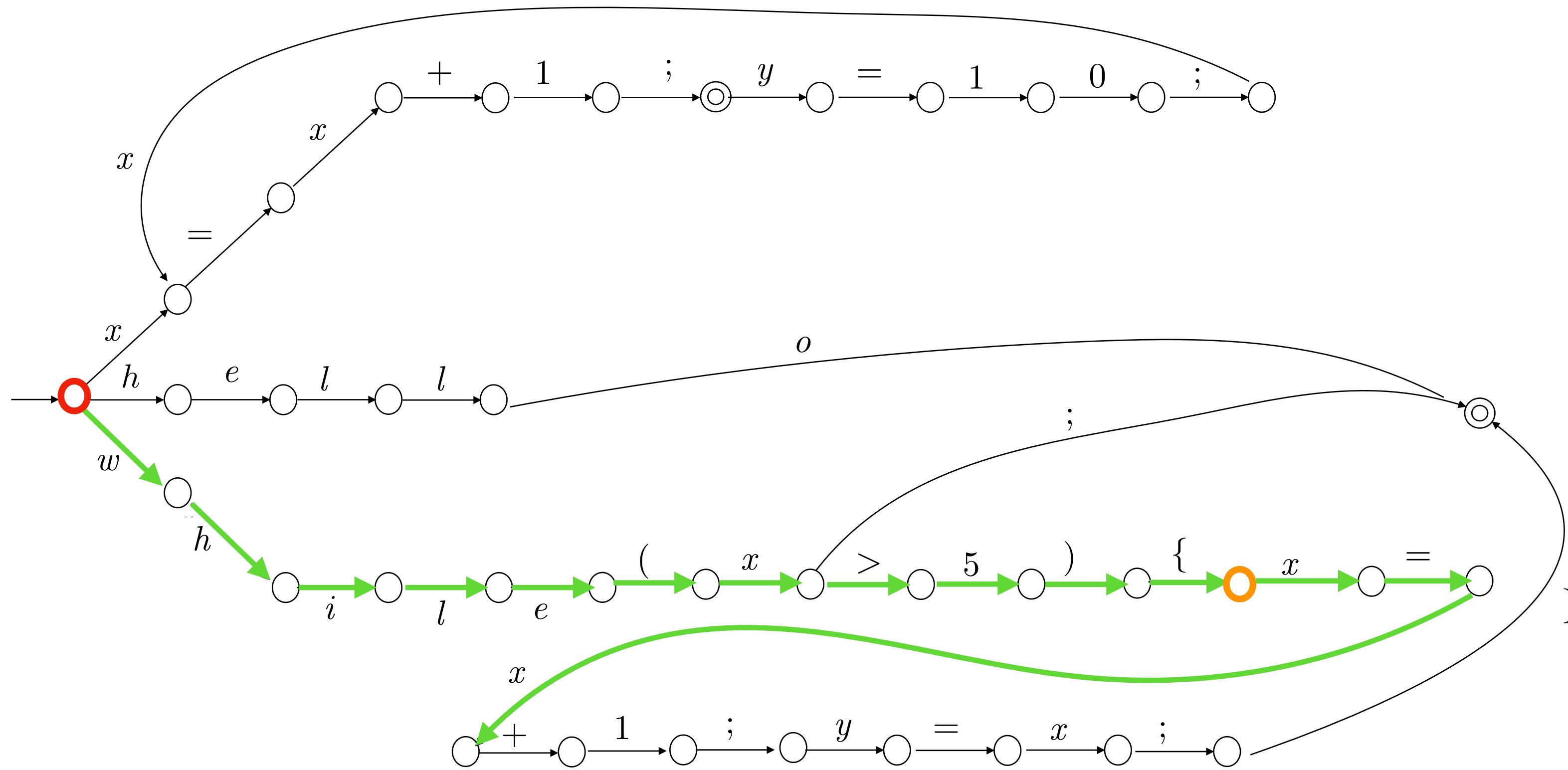


Bonus: Static analysis of eval

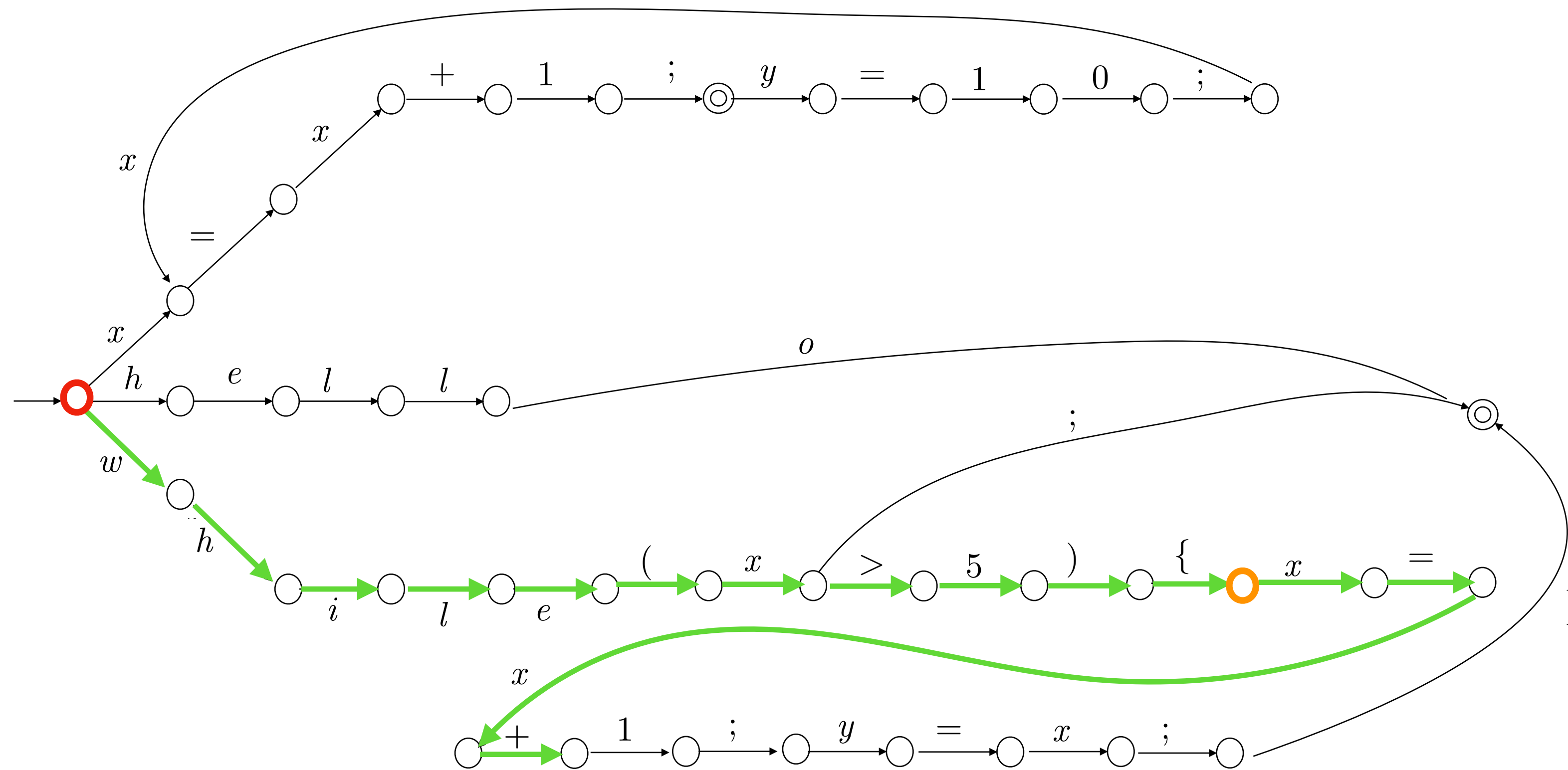


○ `while(x>5) {` ○

Bonus: Static analysis of eval

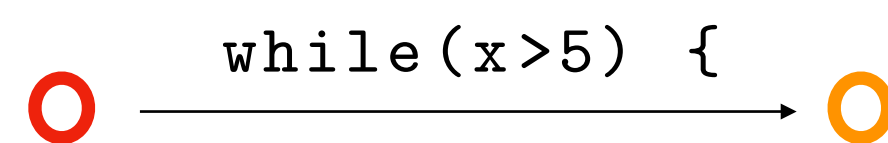
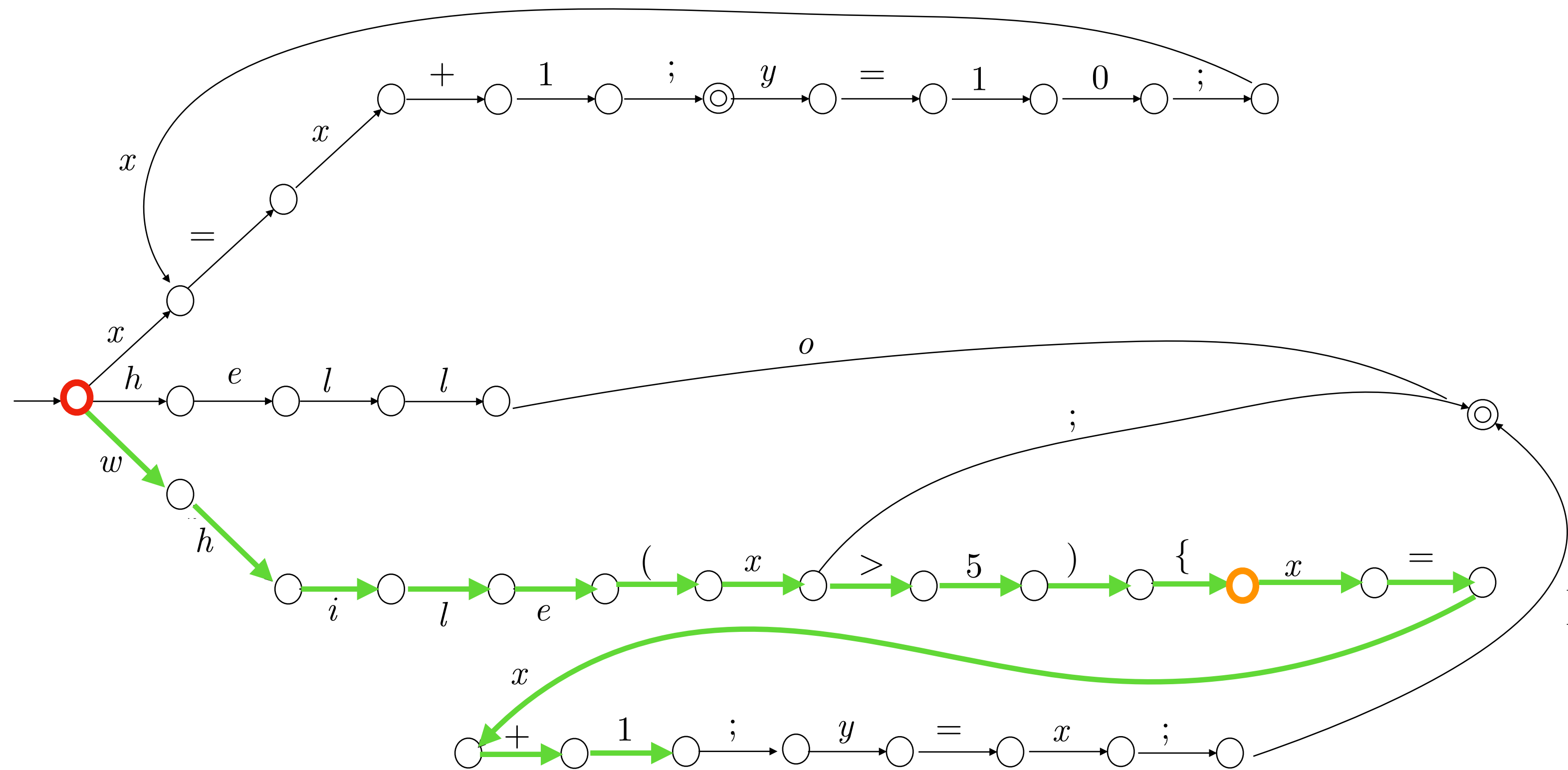


Bonus: Static analysis of eval

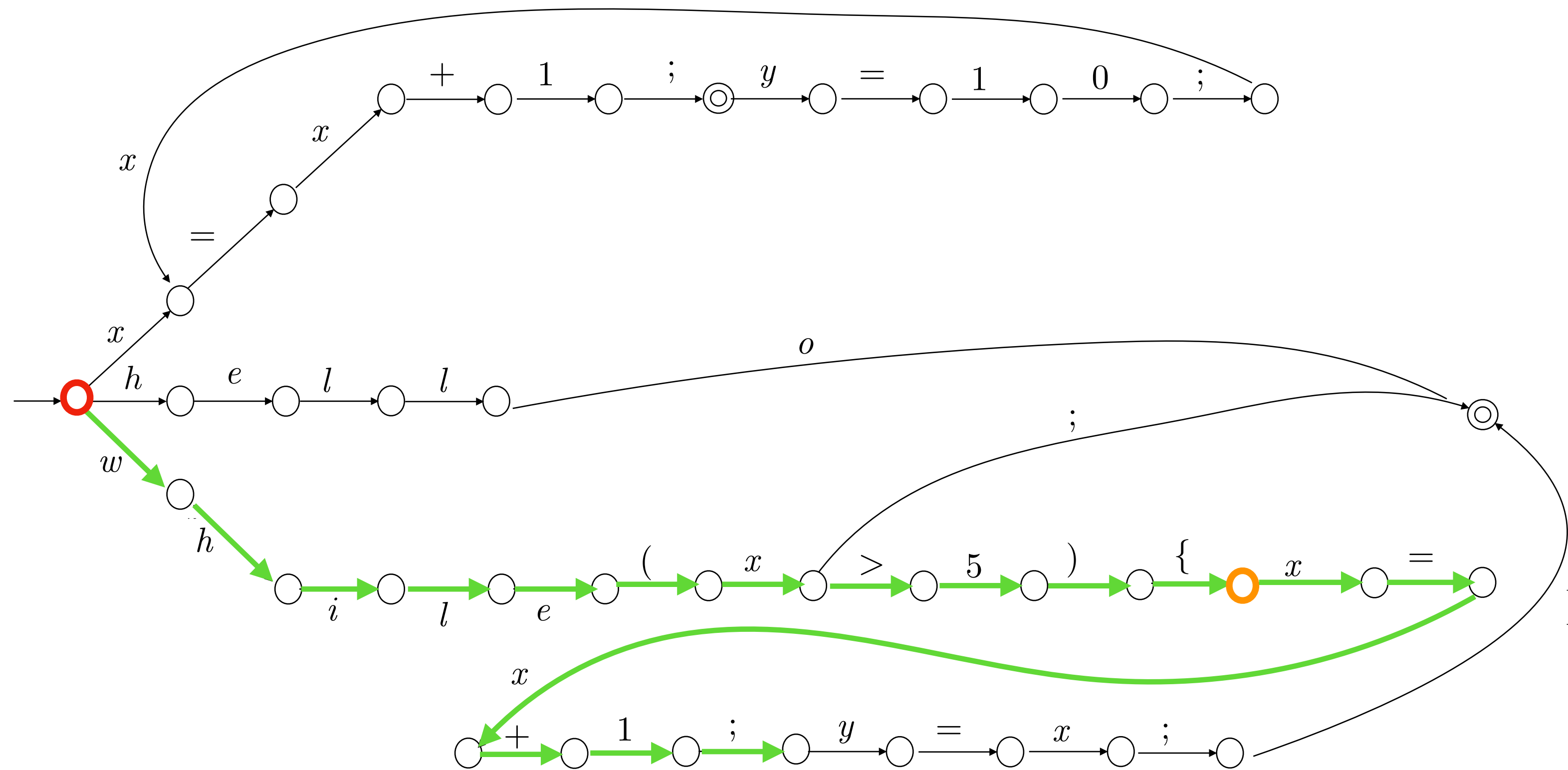


○ `while(x>5) {` ○

Bonus: Static analysis of eval

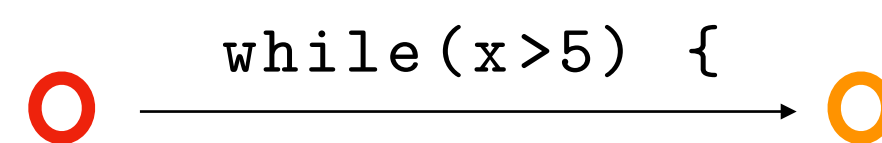
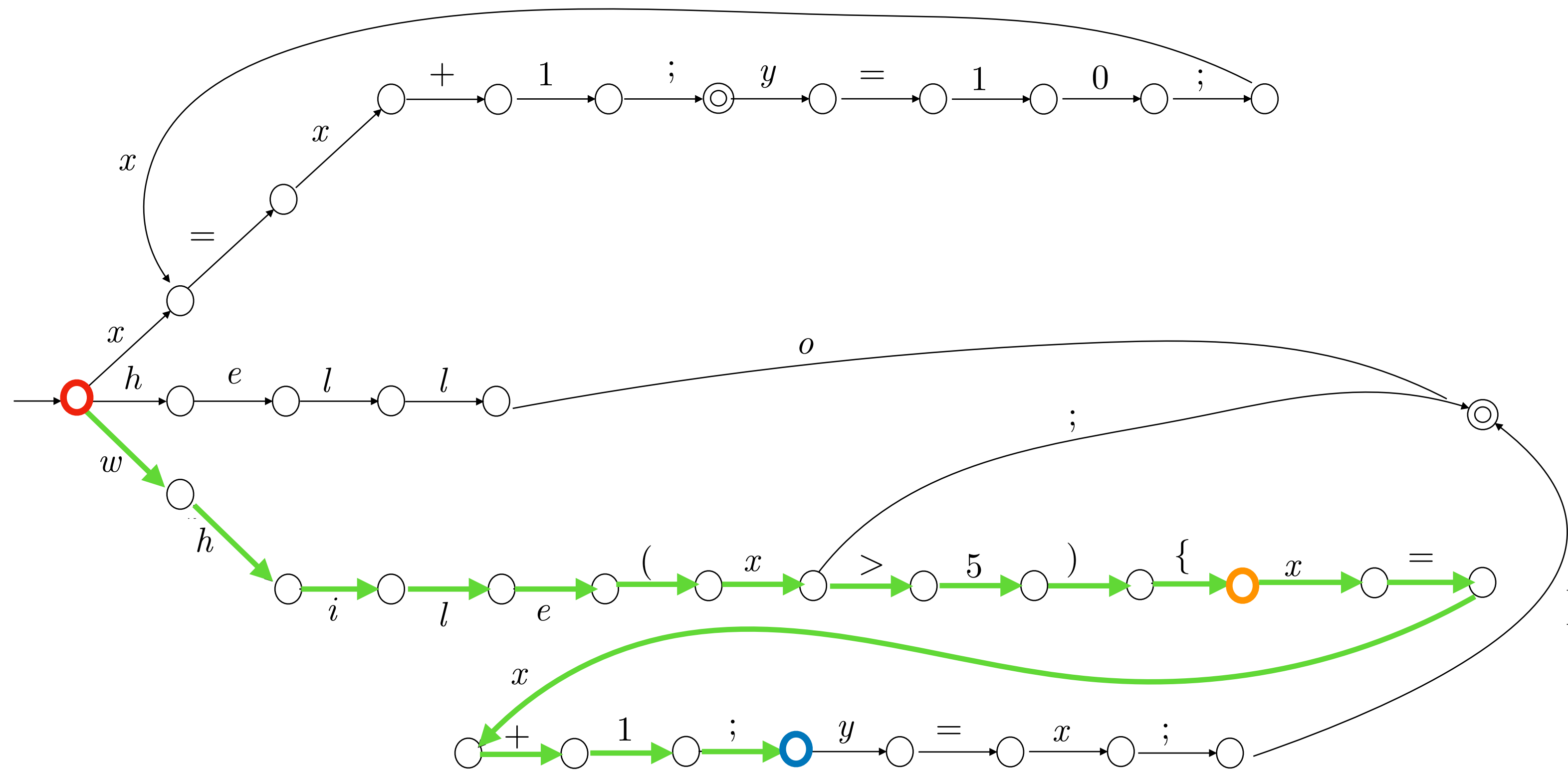


Bonus: Static analysis of eval

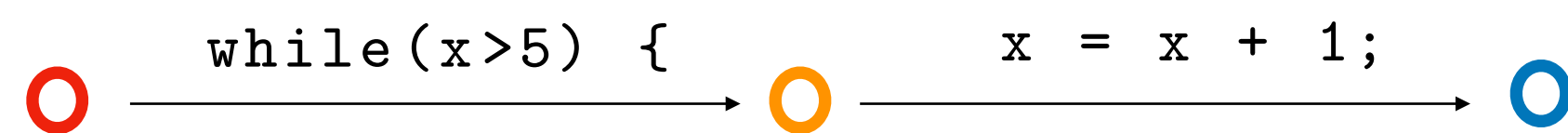
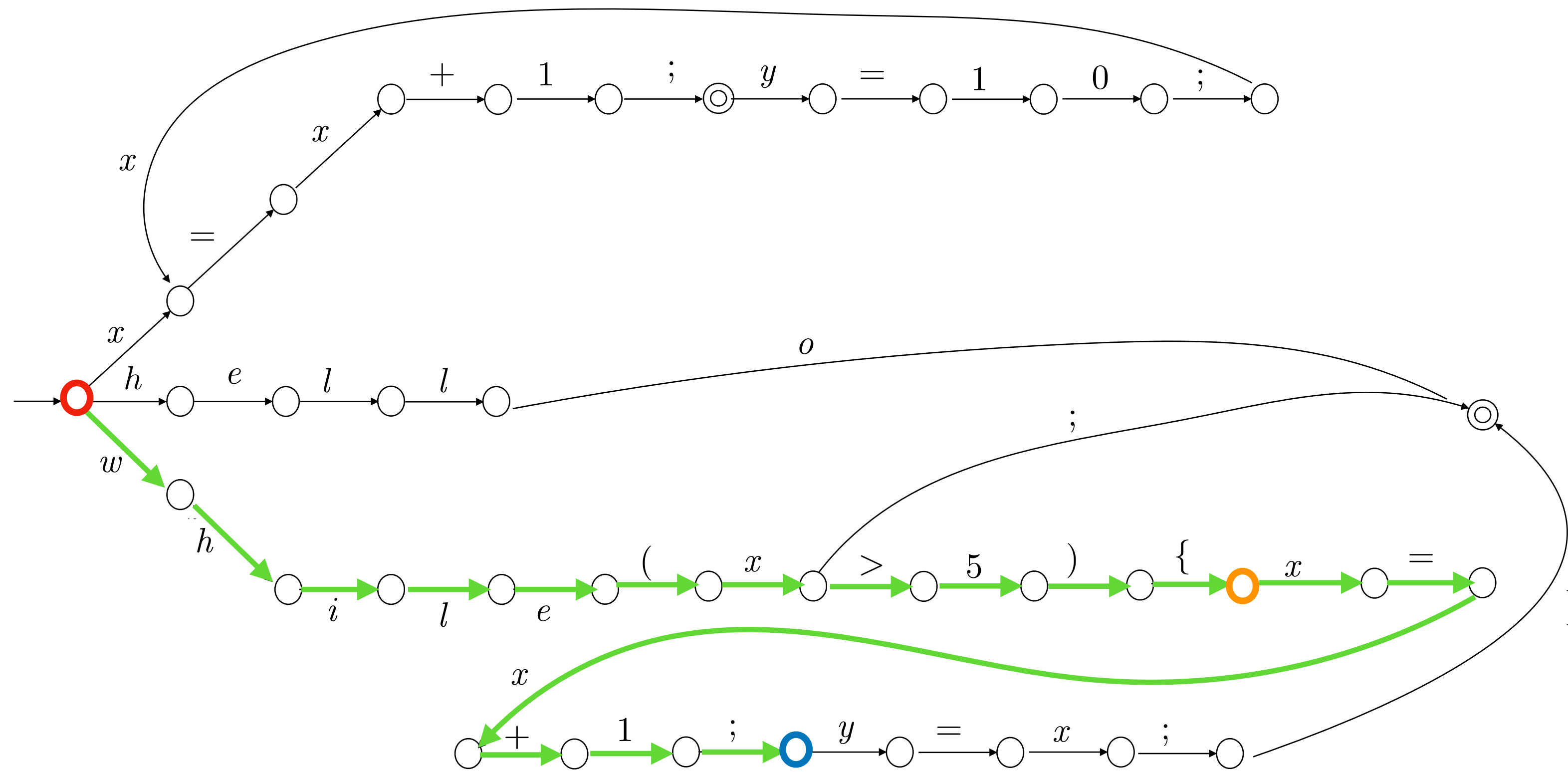


○ `while(x>5) {` ○

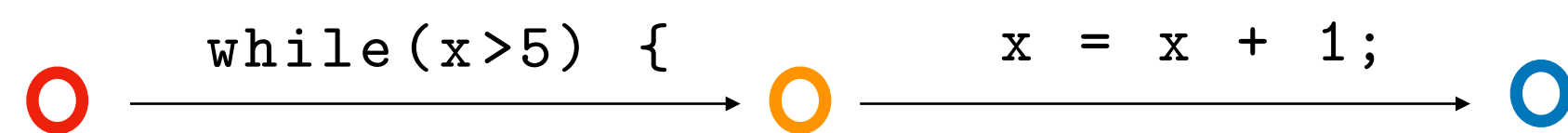
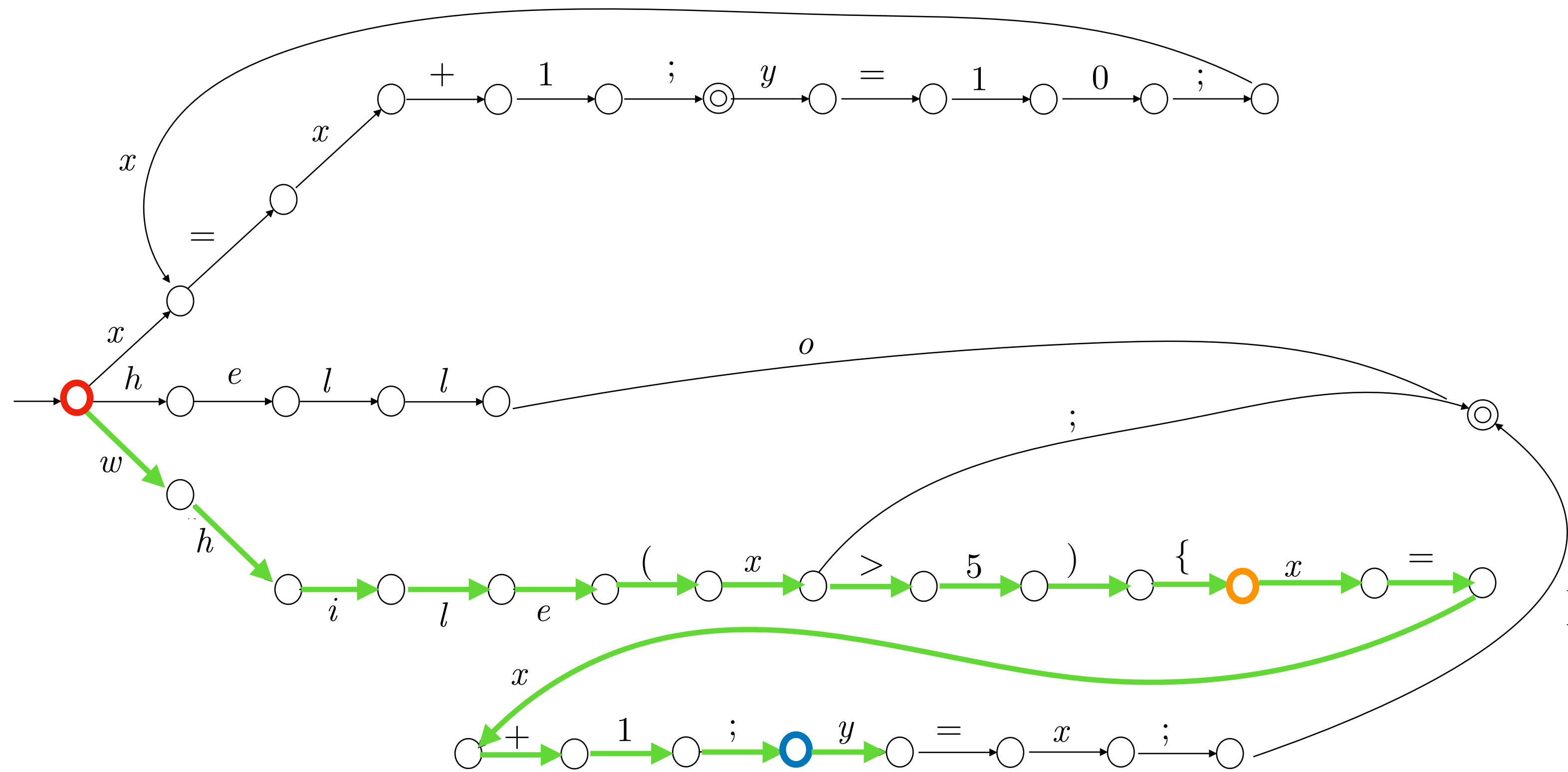
Bonus: Static analysis of eval



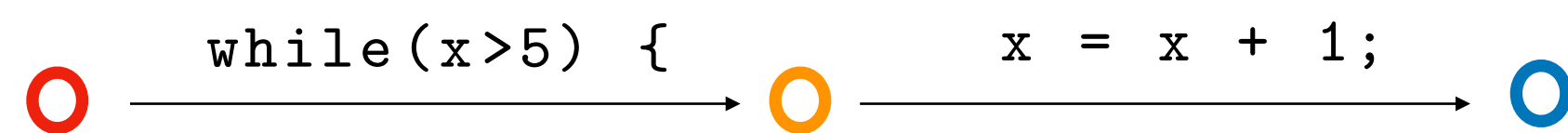
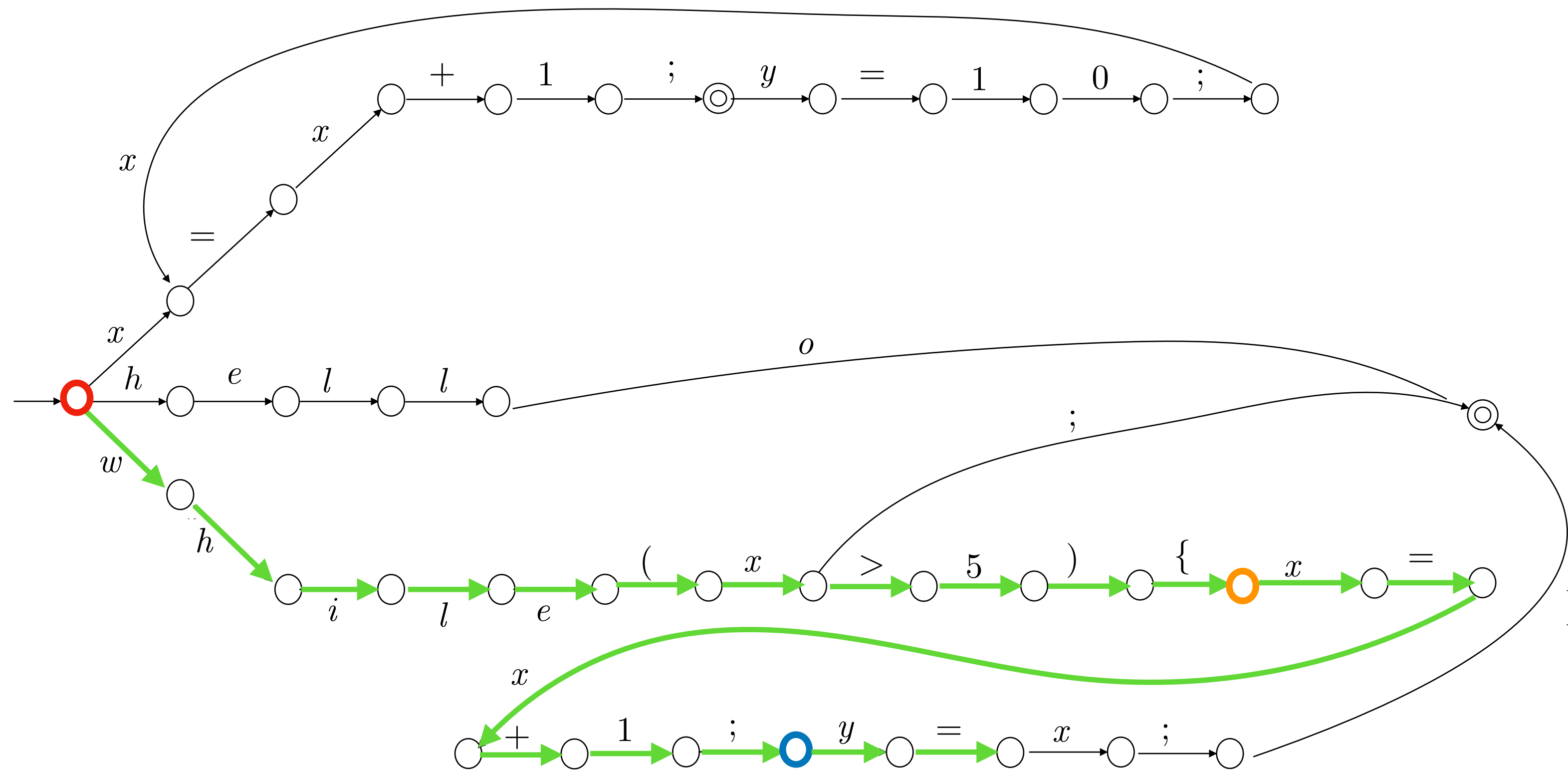
Bonus: Static analysis of eval



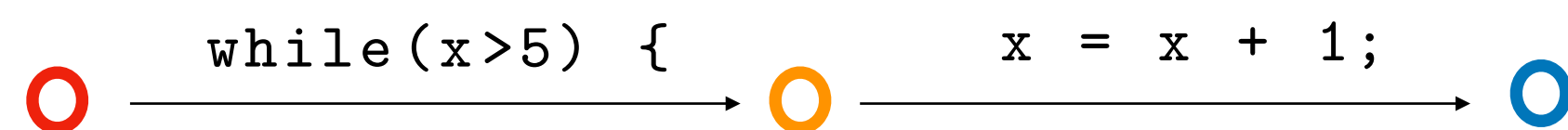
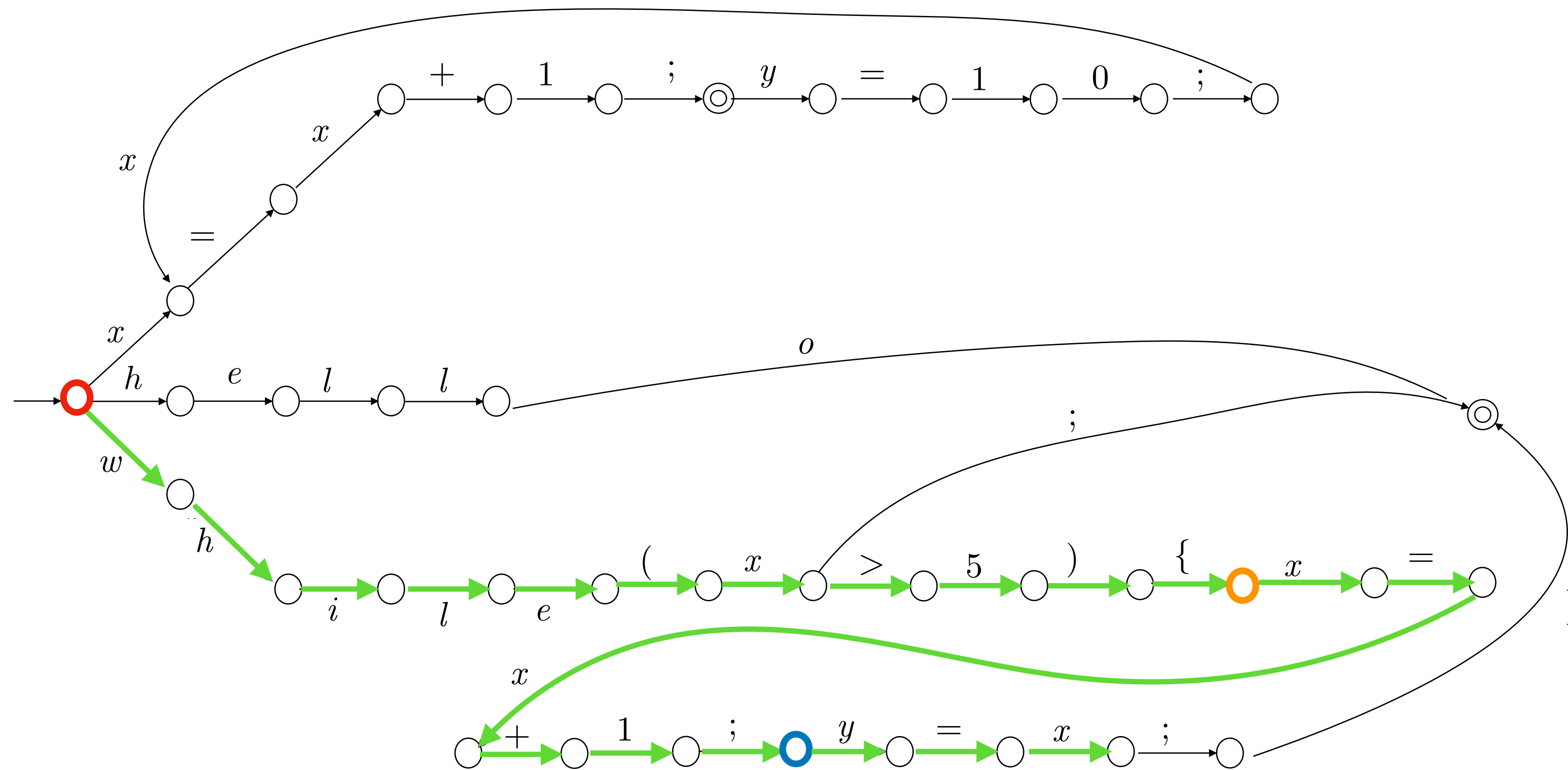
Bonus: Static analysis of eval



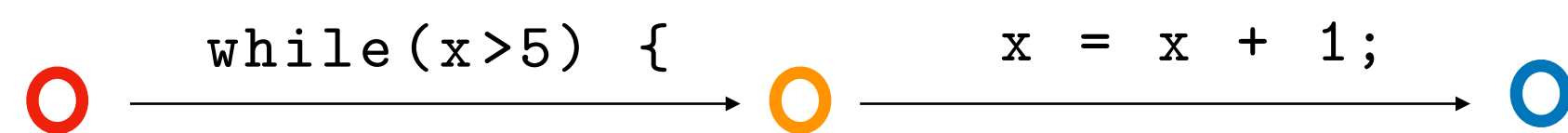
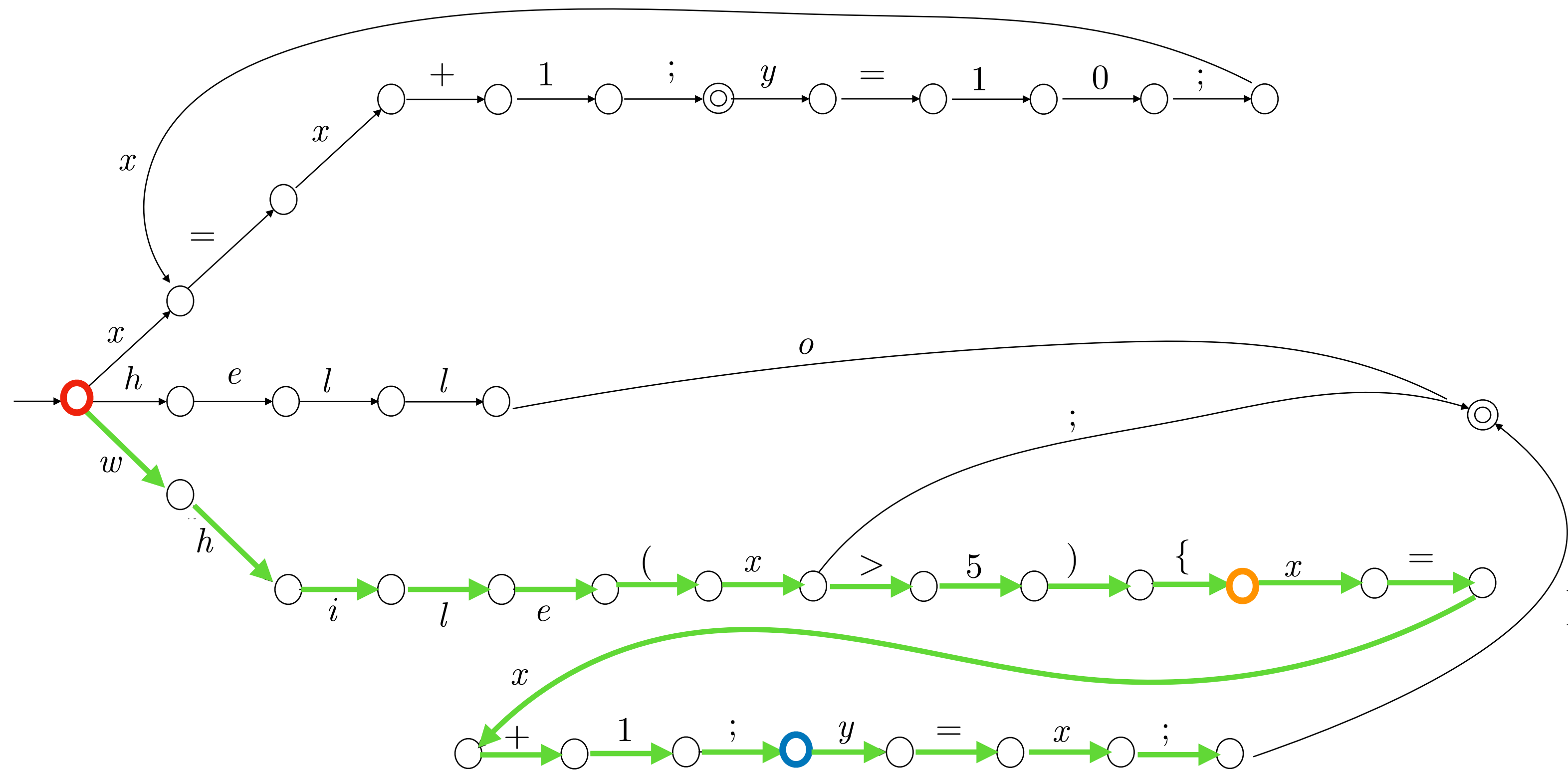
Bonus: Static analysis of eval



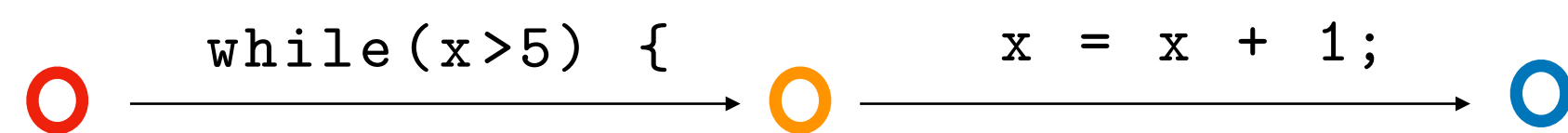
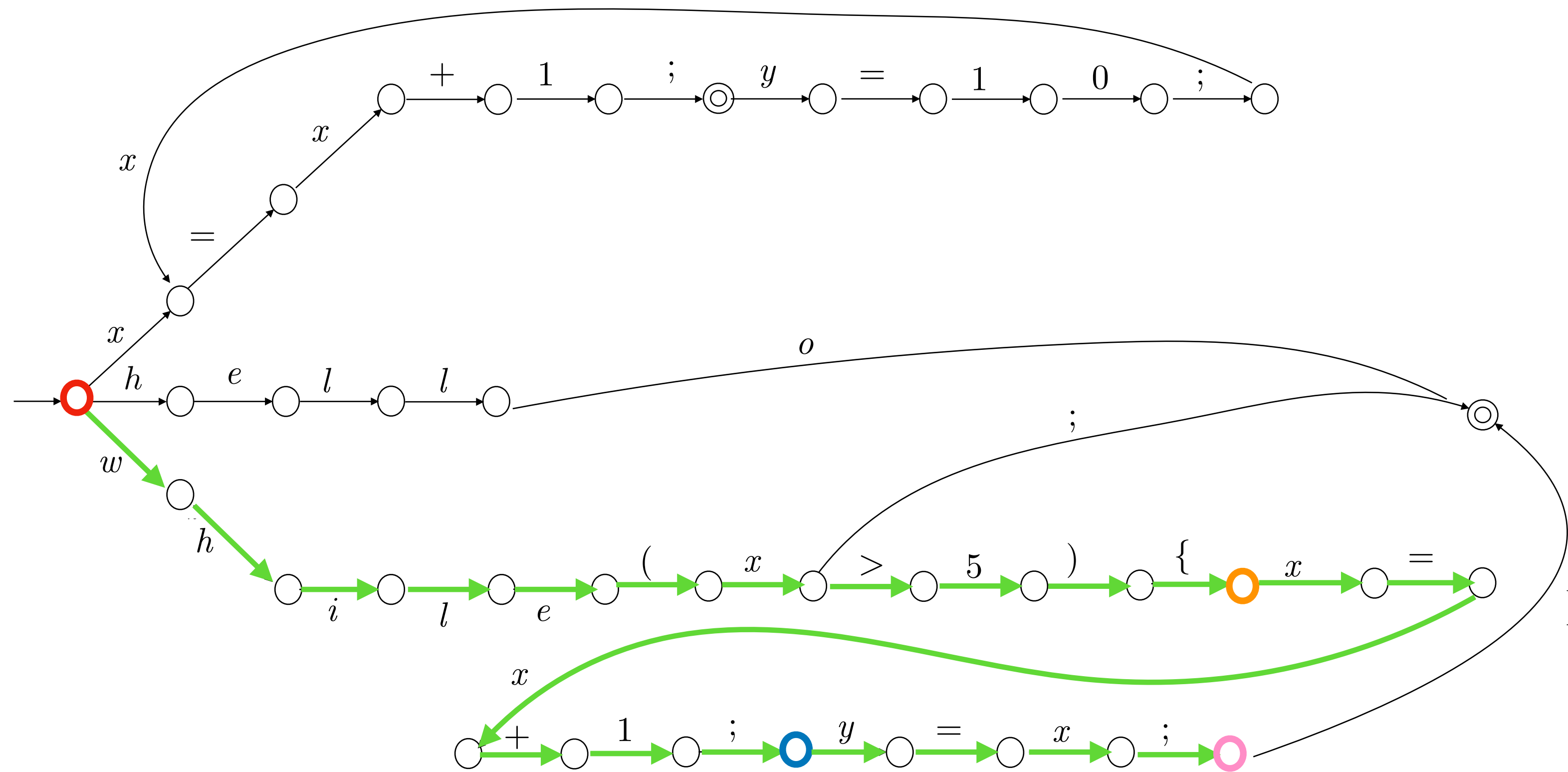
Bonus: Static analysis of eval



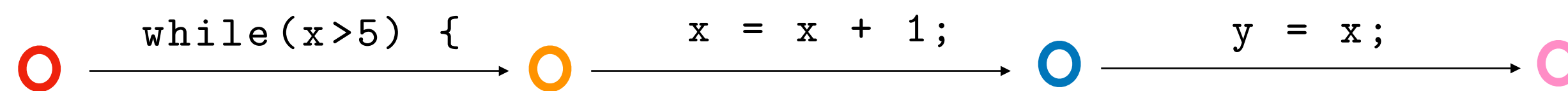
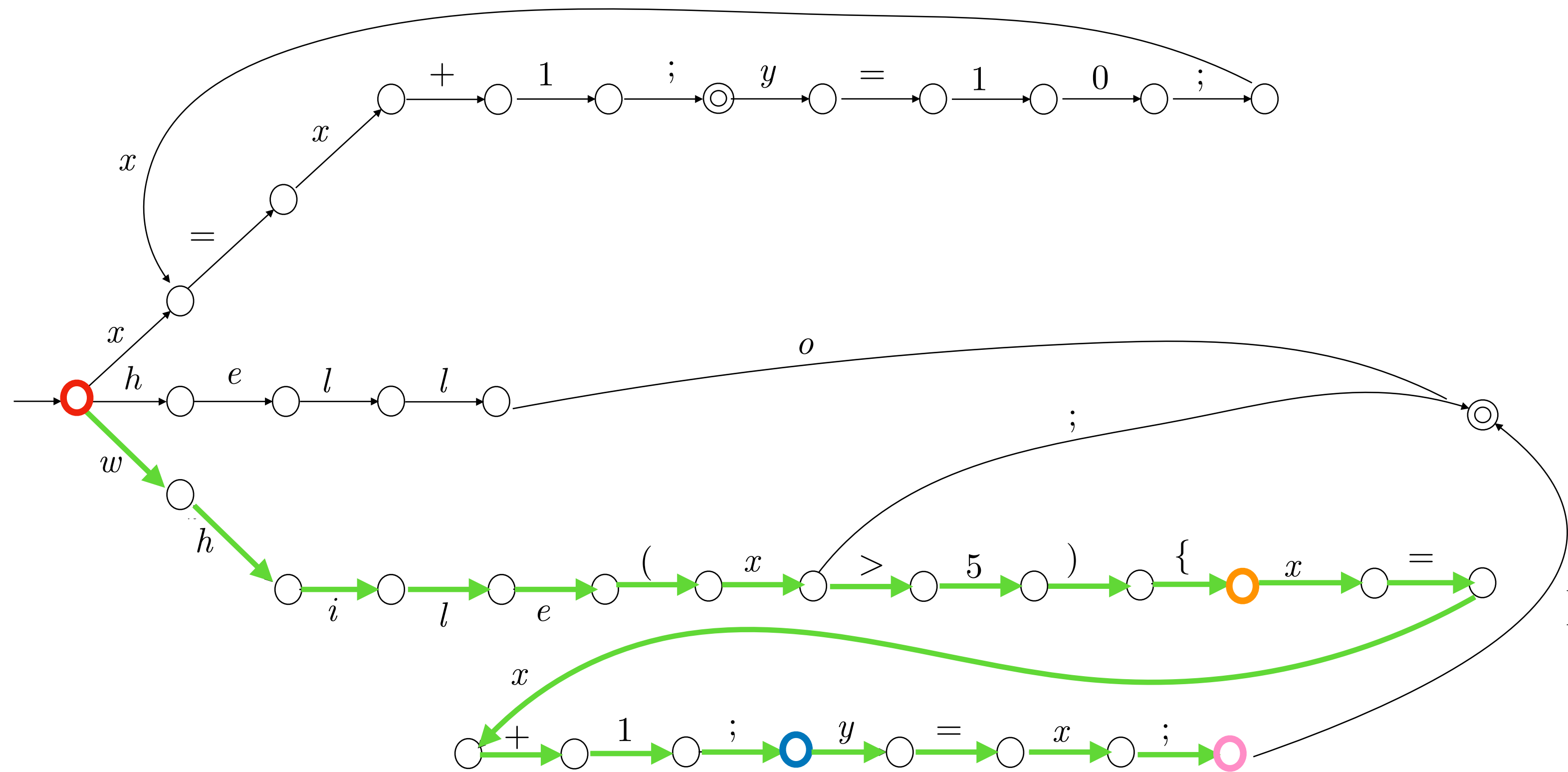
Bonus: Static analysis of eval



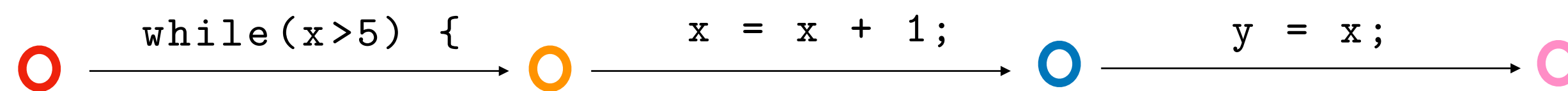
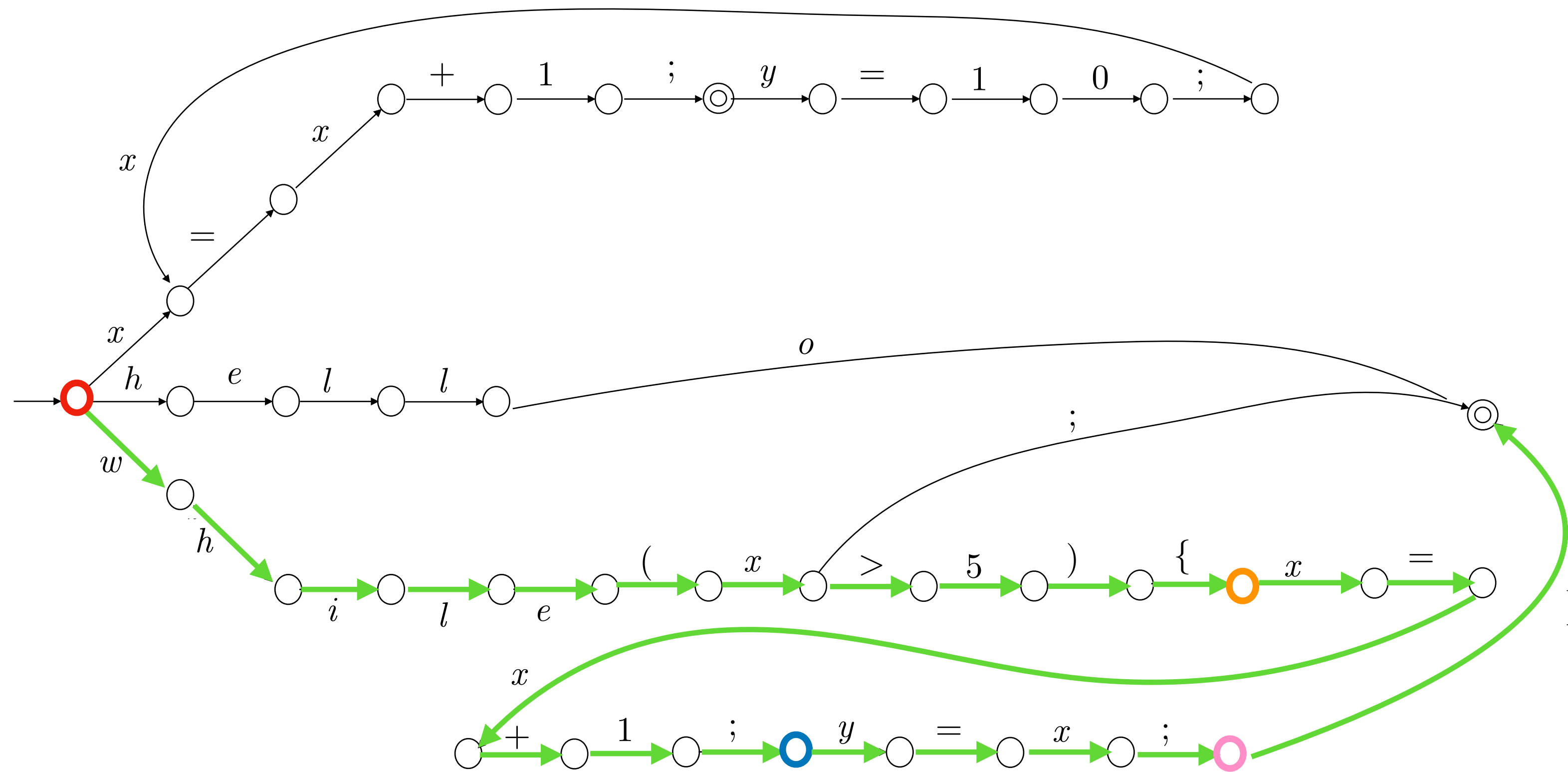
Bonus: Static analysis of eval



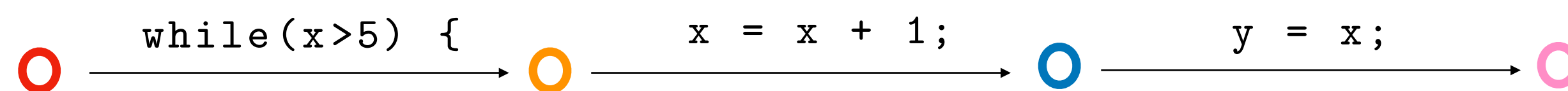
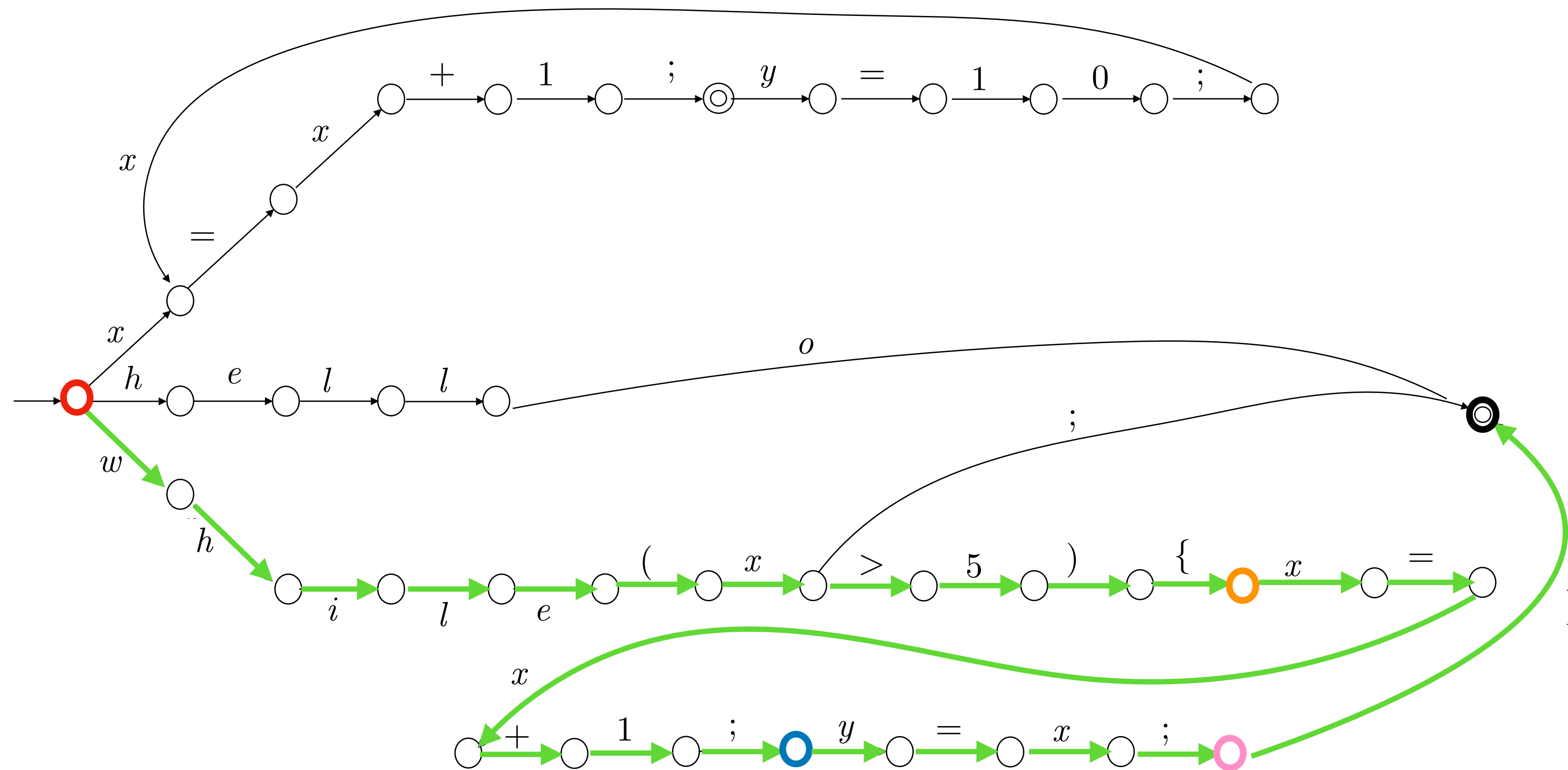
Bonus: Static analysis of eval



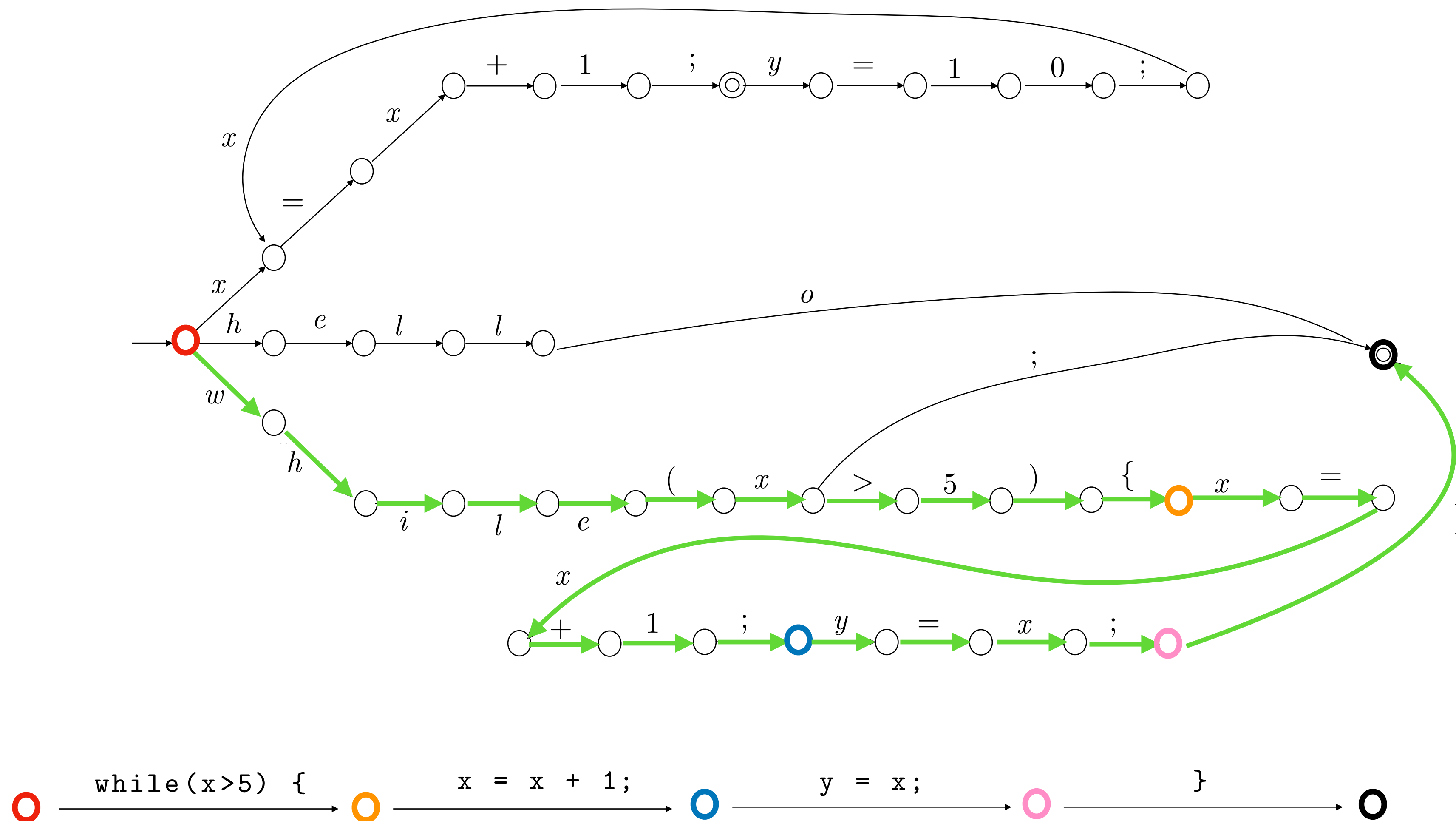
Bonus: Static analysis of eval



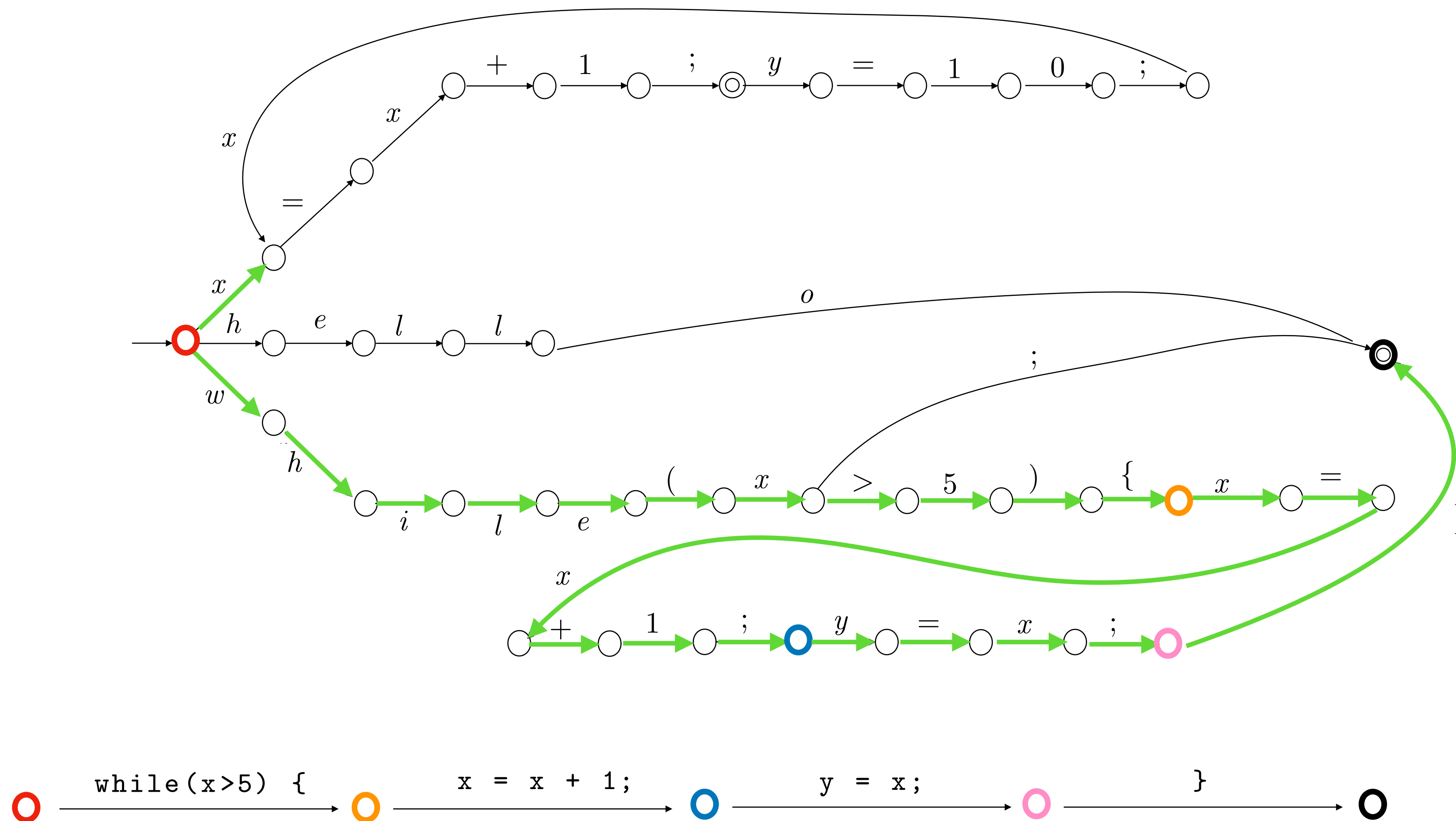
Bonus: Static analysis of eval



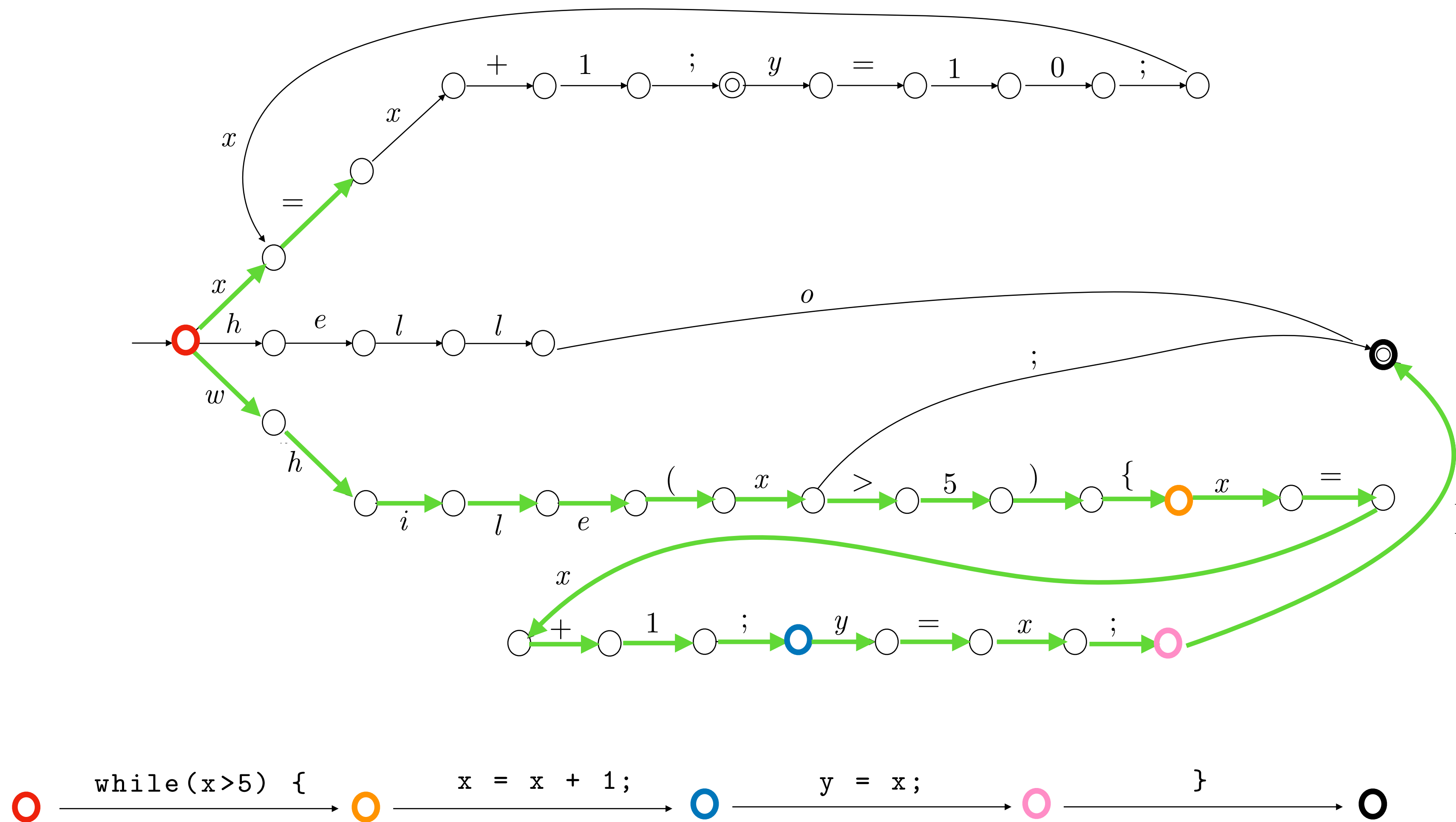
Bonus: Static analysis of eval



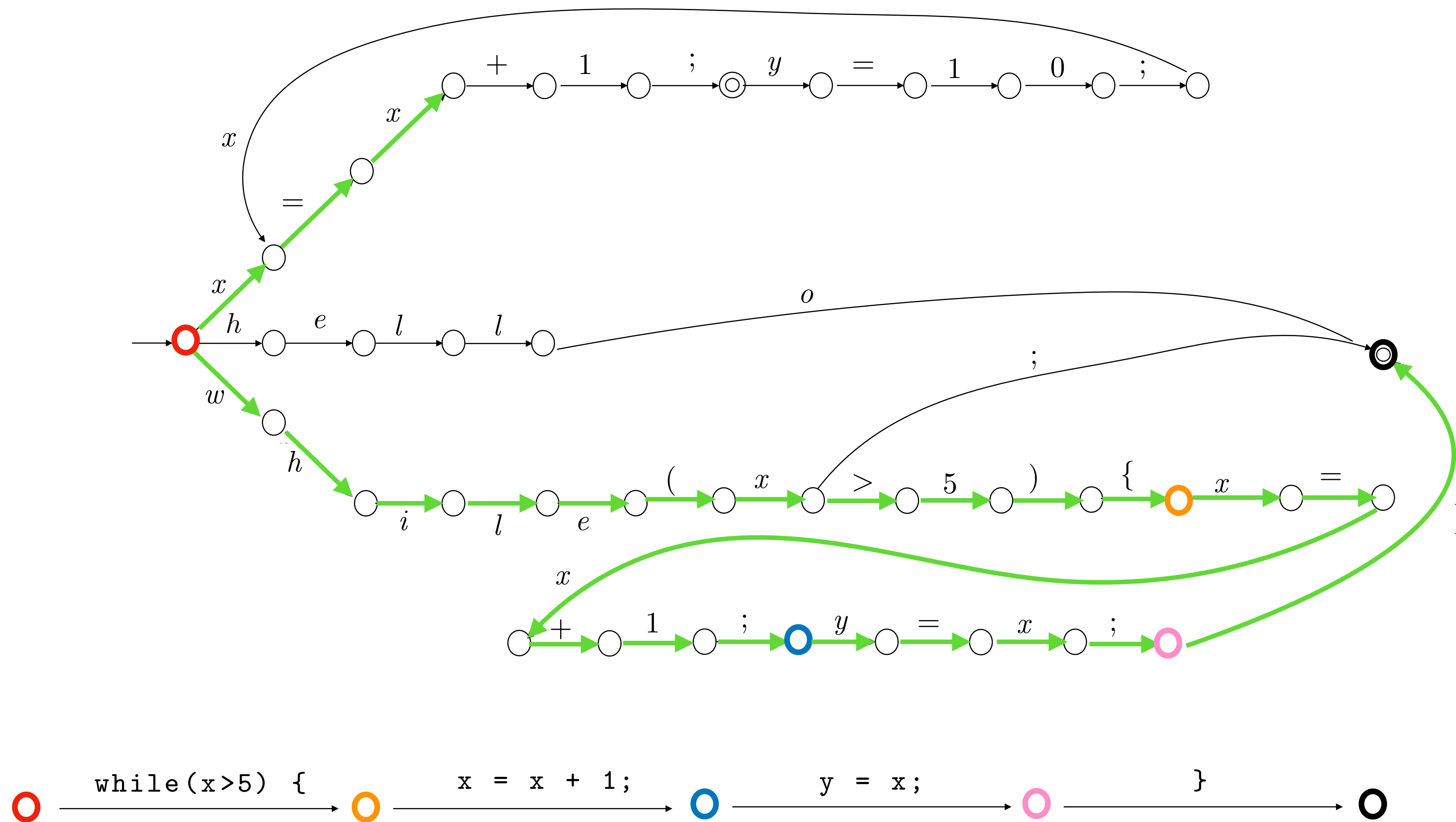
Bonus: Static analysis of eval



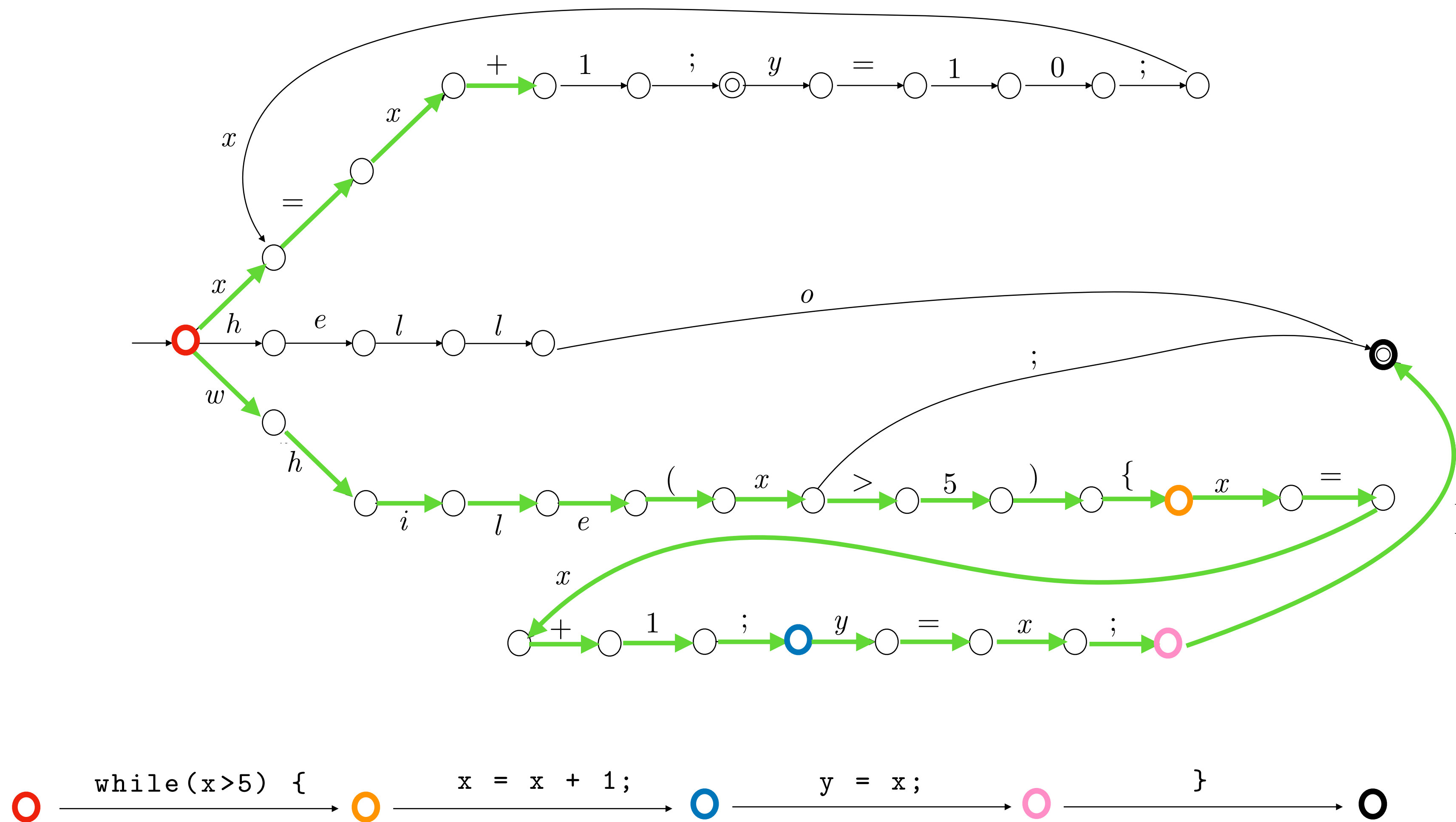
Bonus: Static analysis of eval



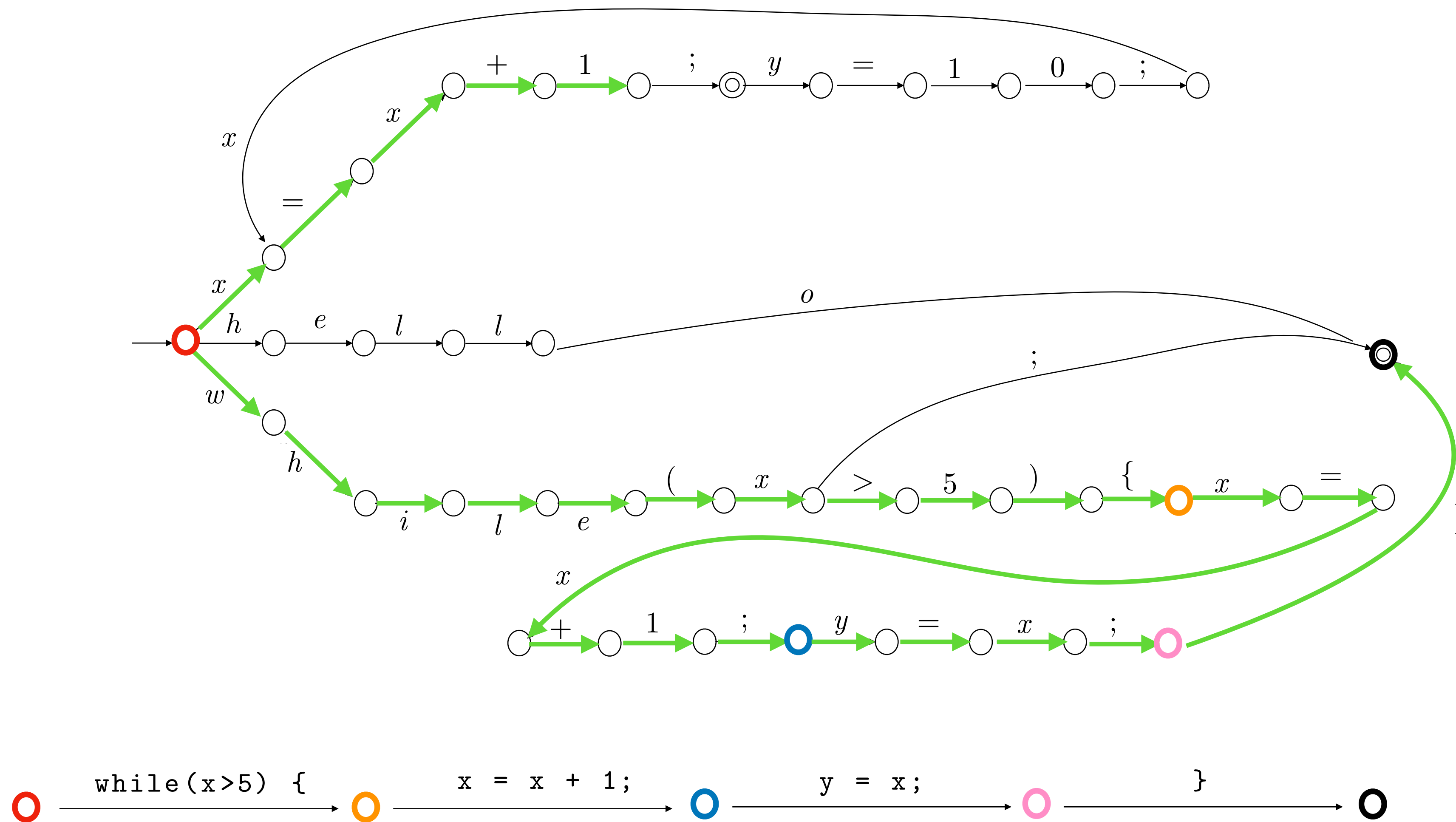
Bonus: Static analysis of eval



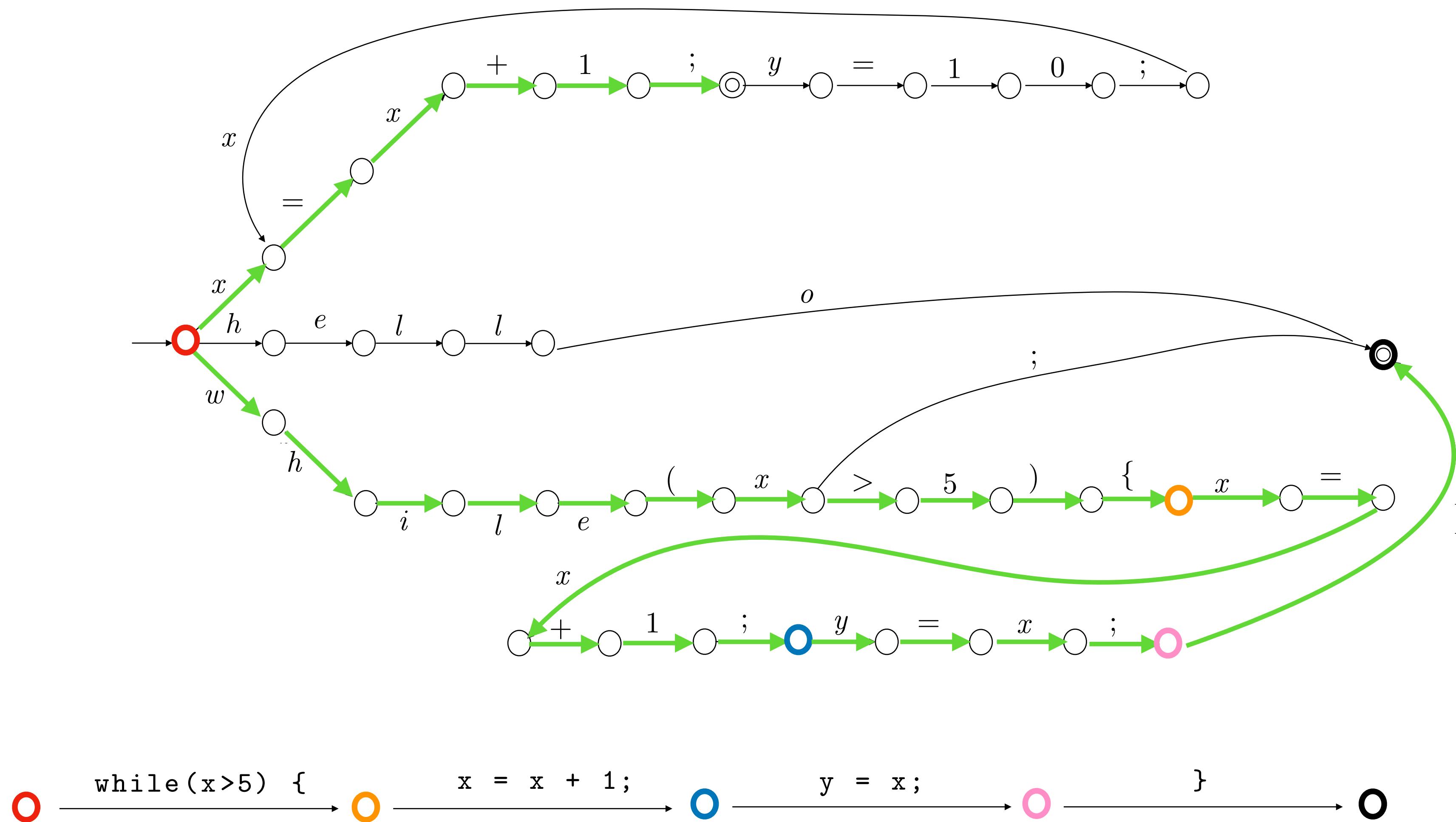
Bonus: Static analysis of eval



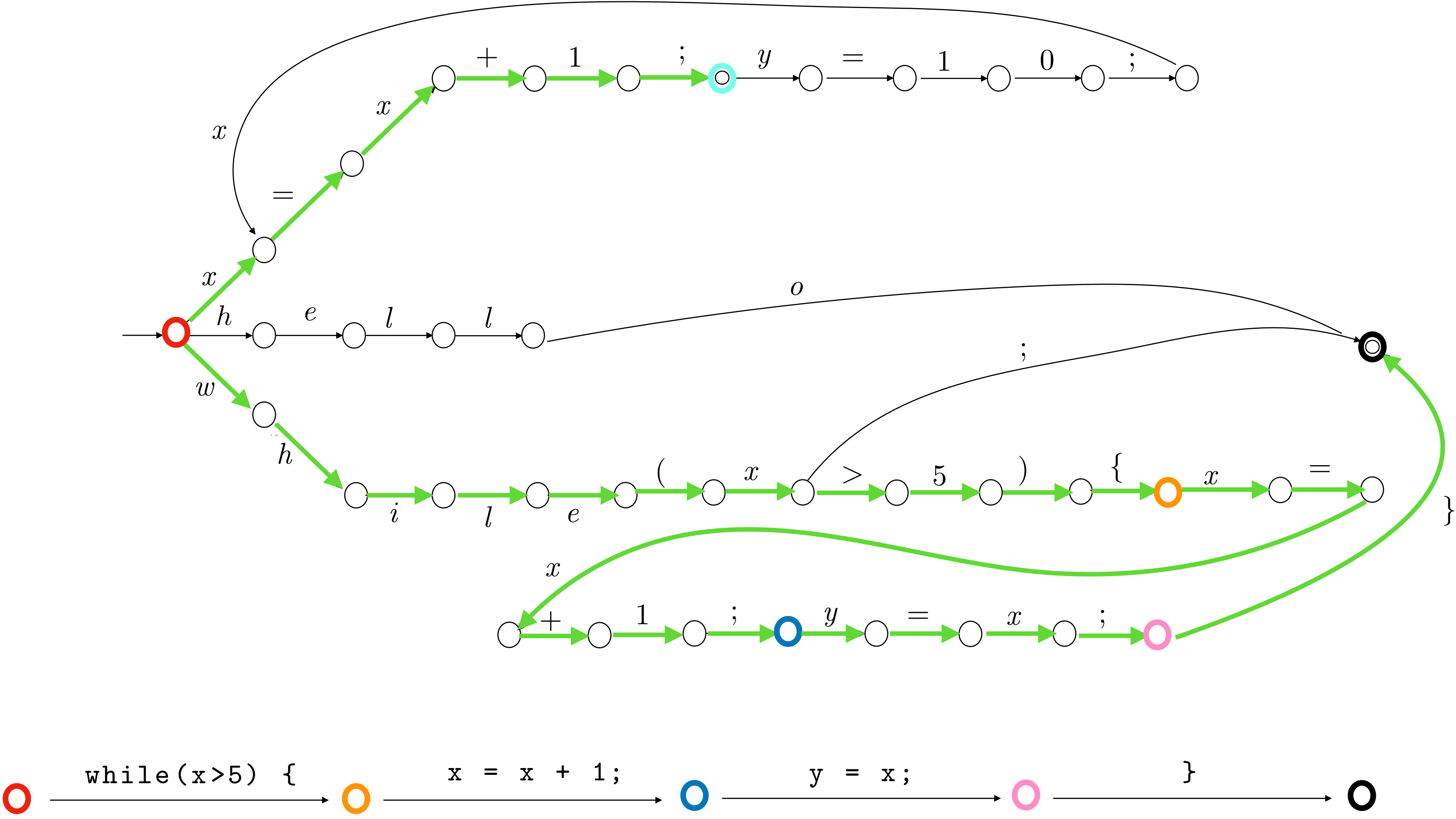
Bonus: Static analysis of eval



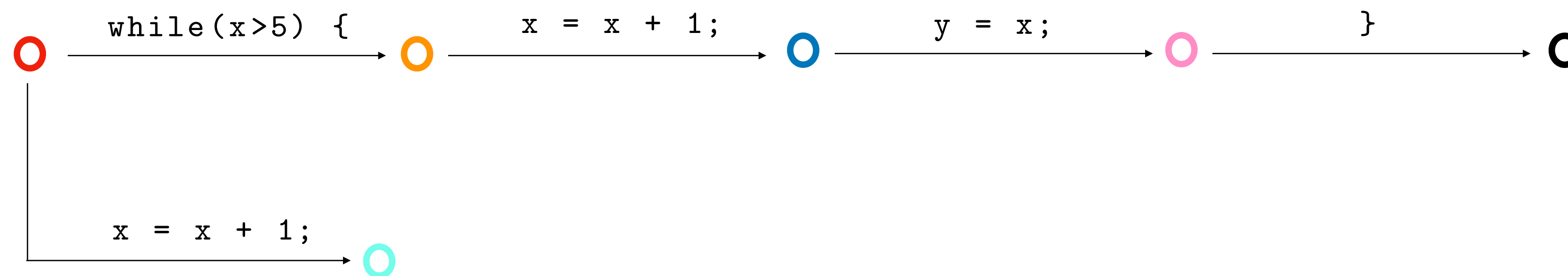
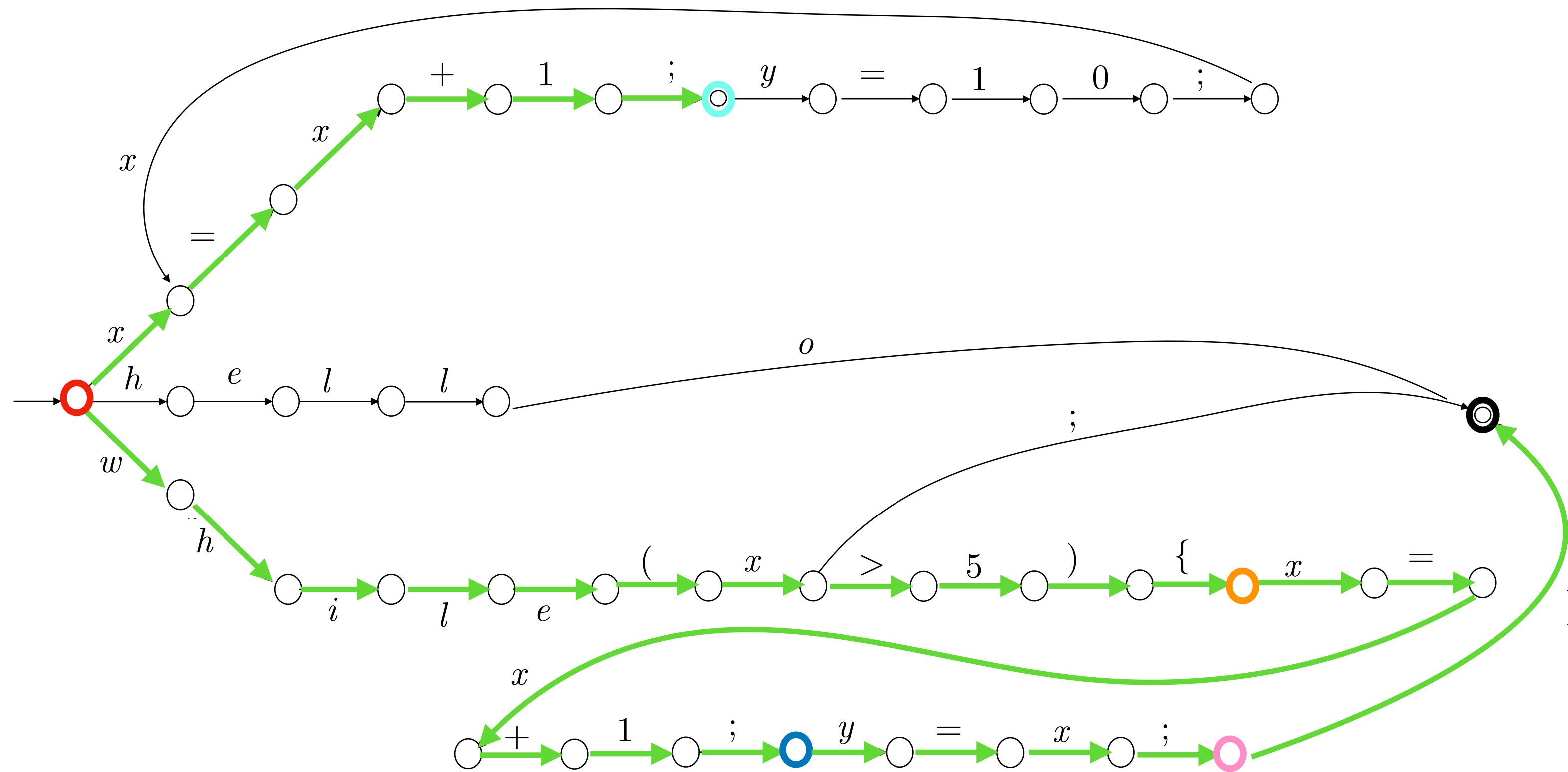
Bonus: Static analysis of eval



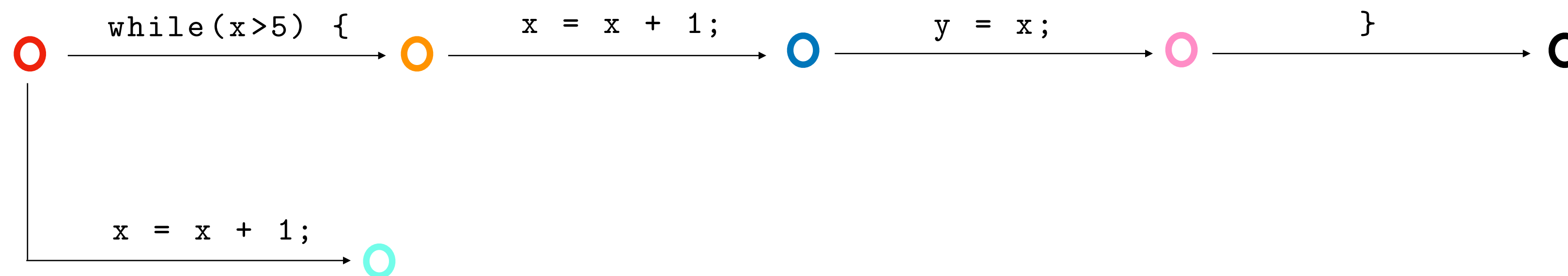
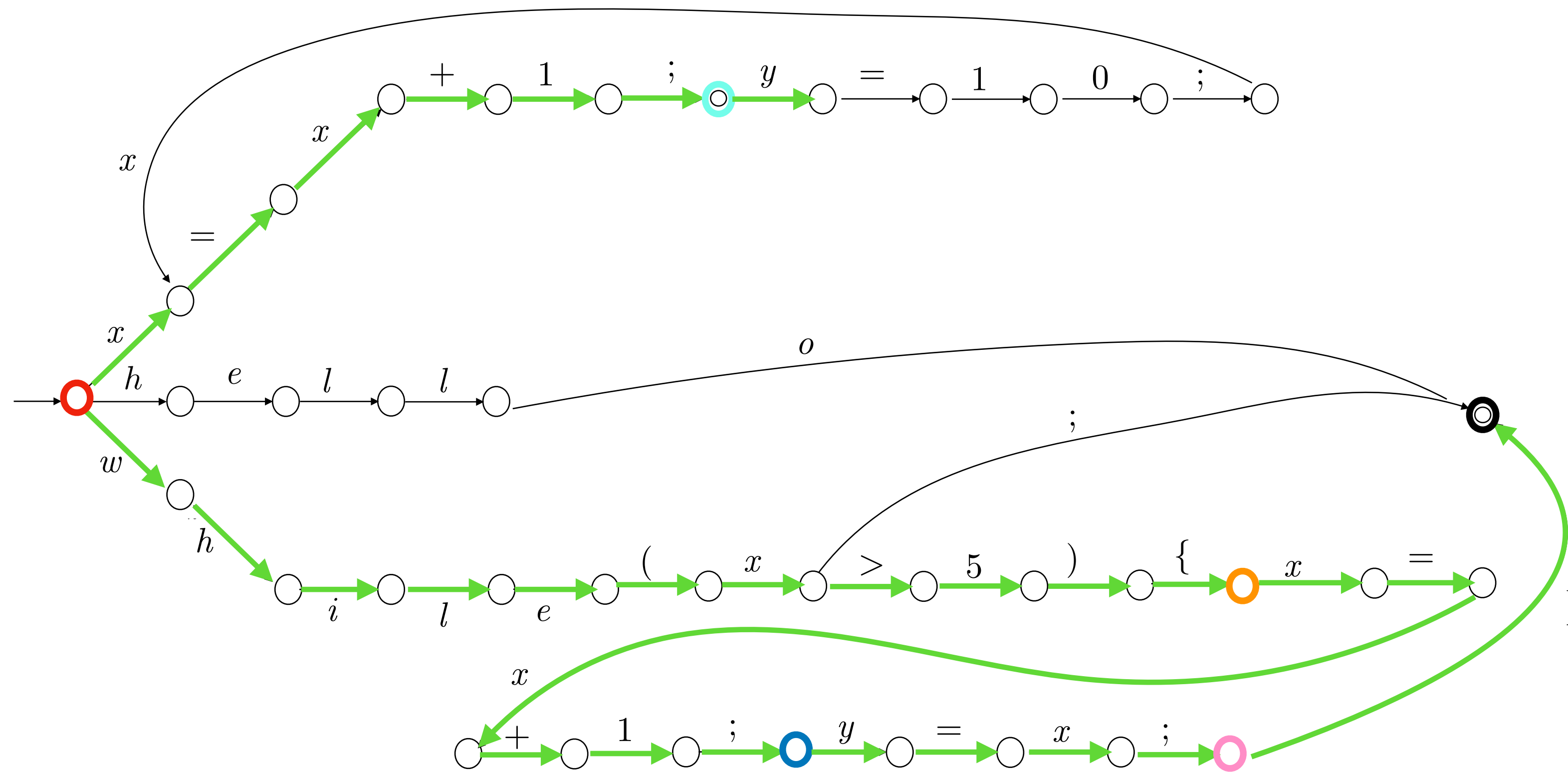
Bonus: Static analysis of eval



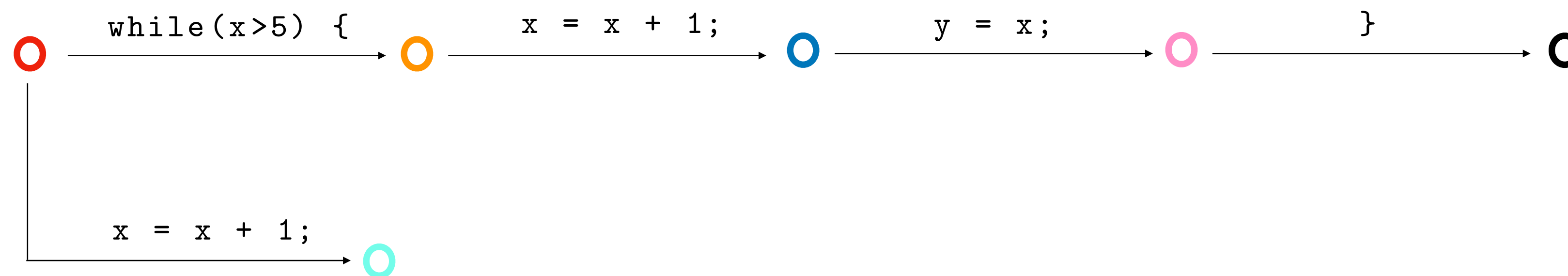
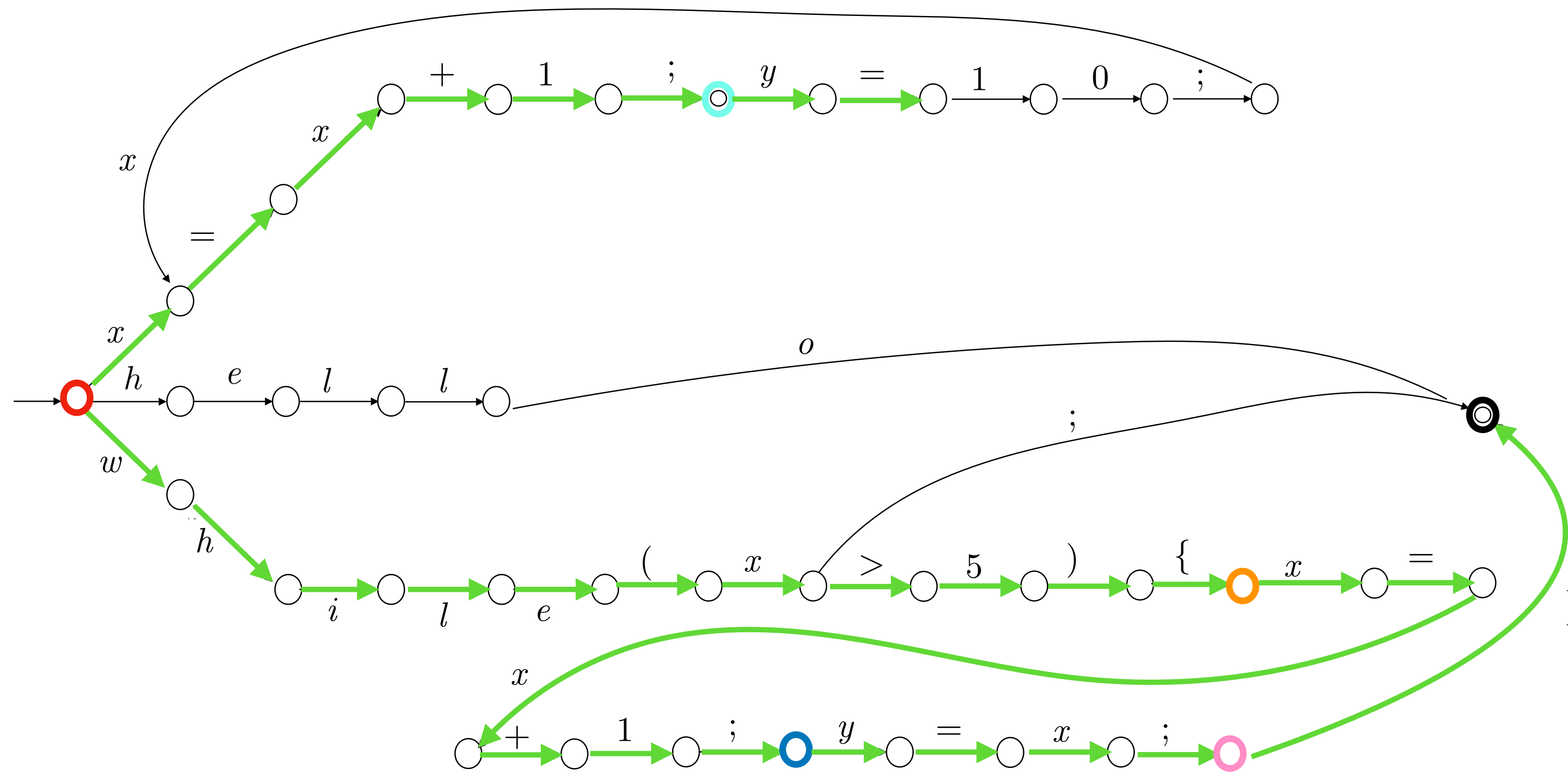
Bonus: Static analysis of eval



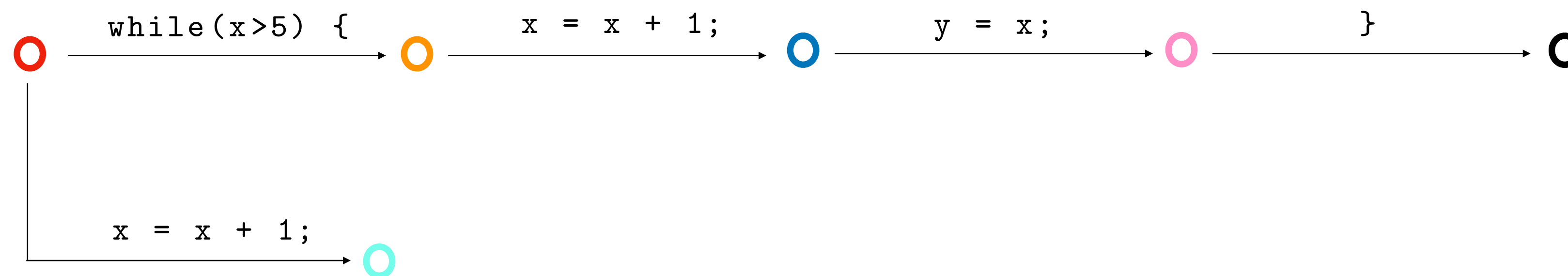
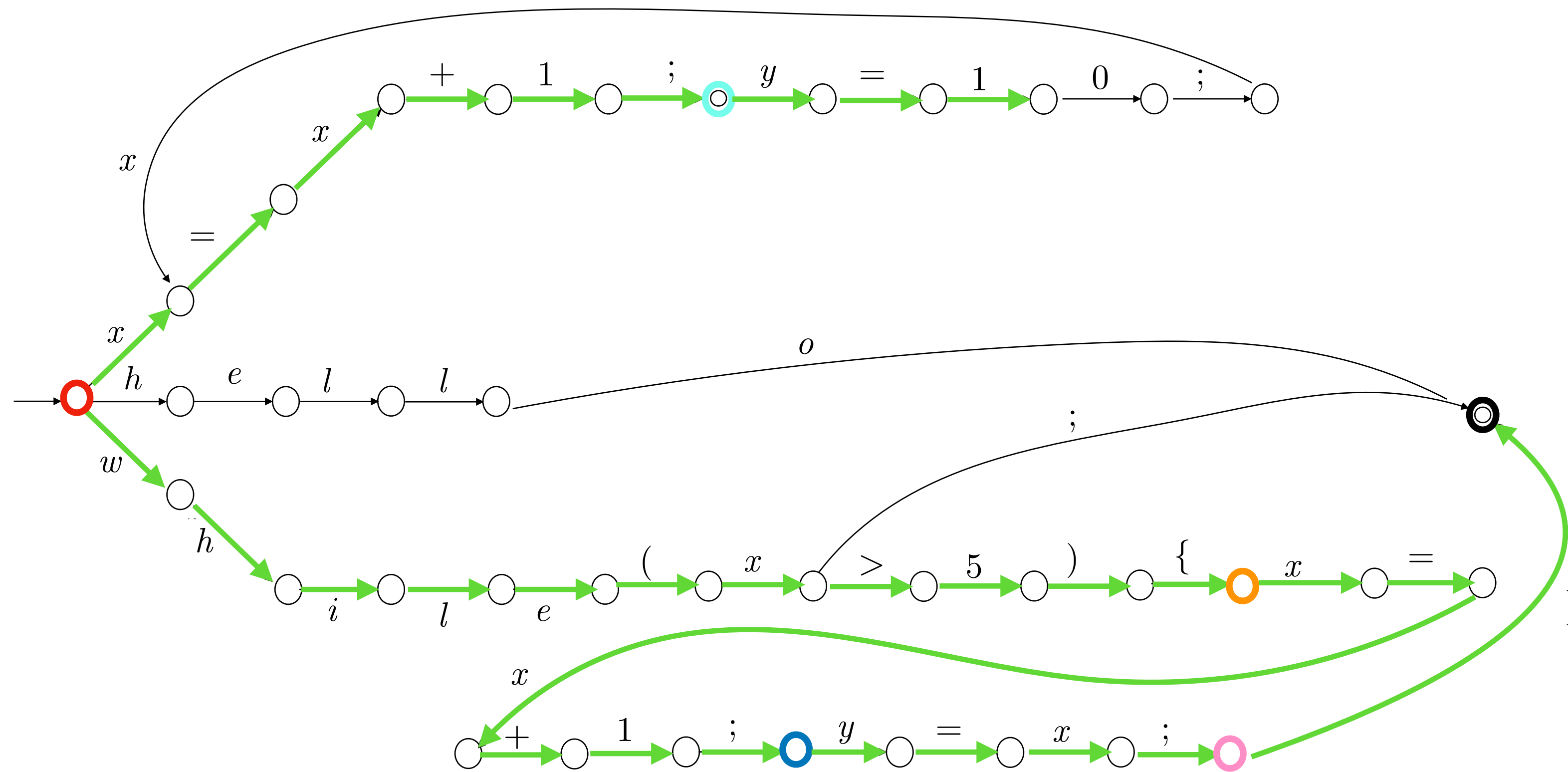
Bonus: Static analysis of eval



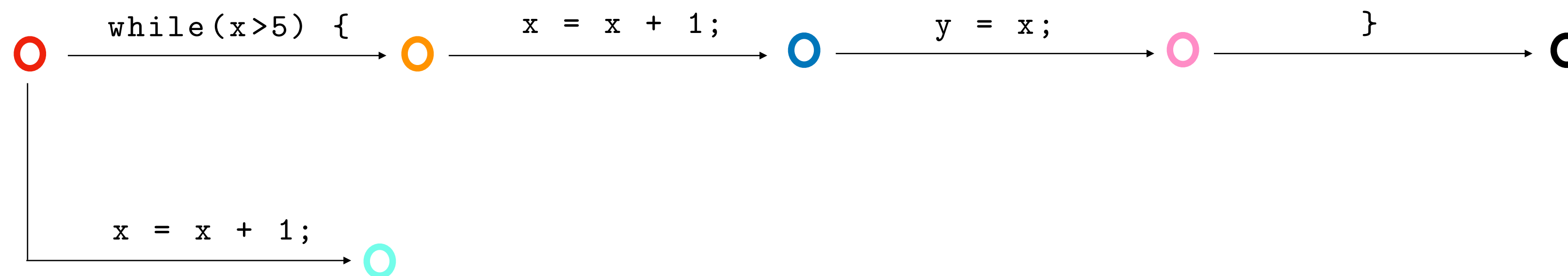
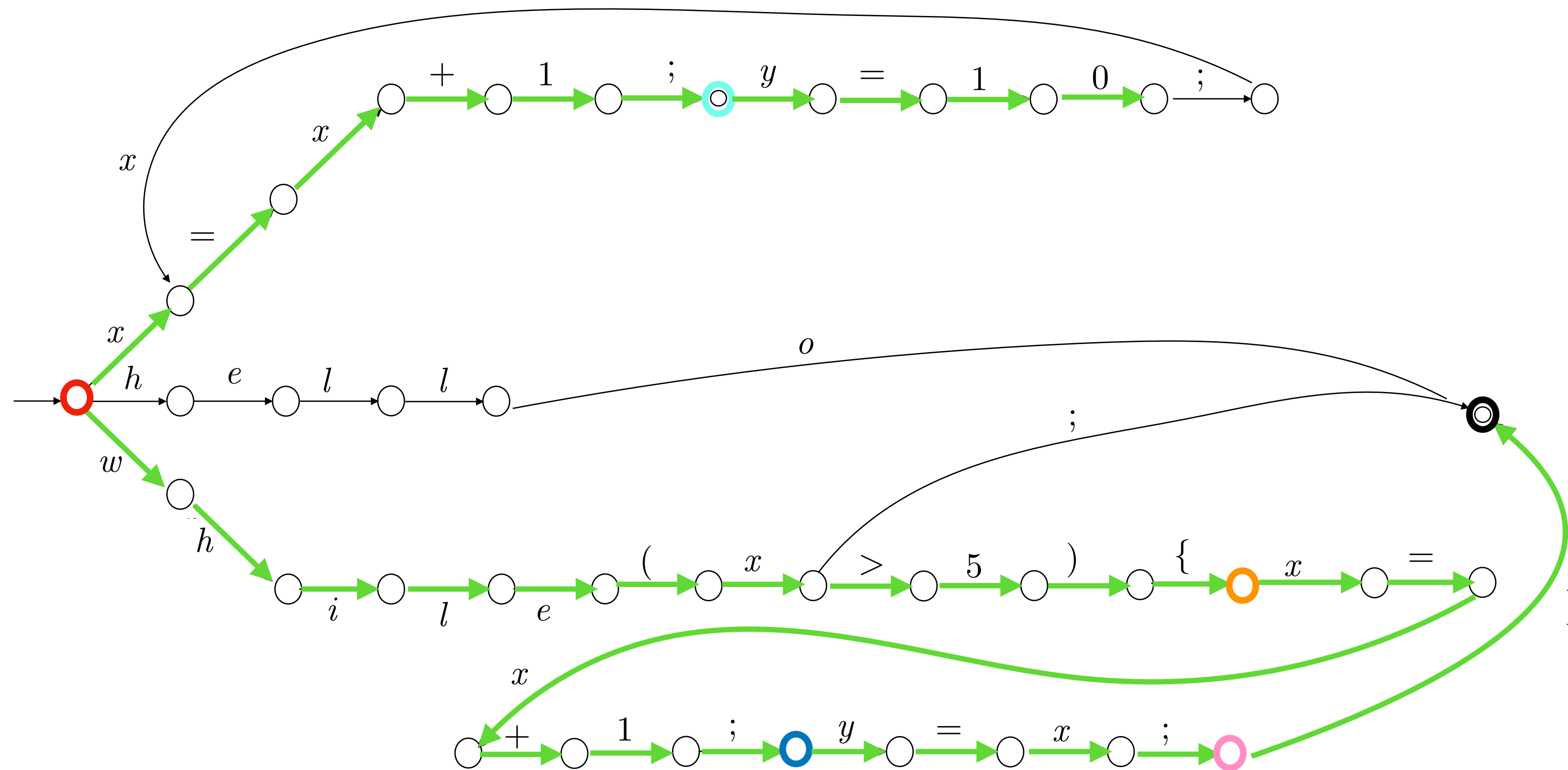
Bonus: Static analysis of eval



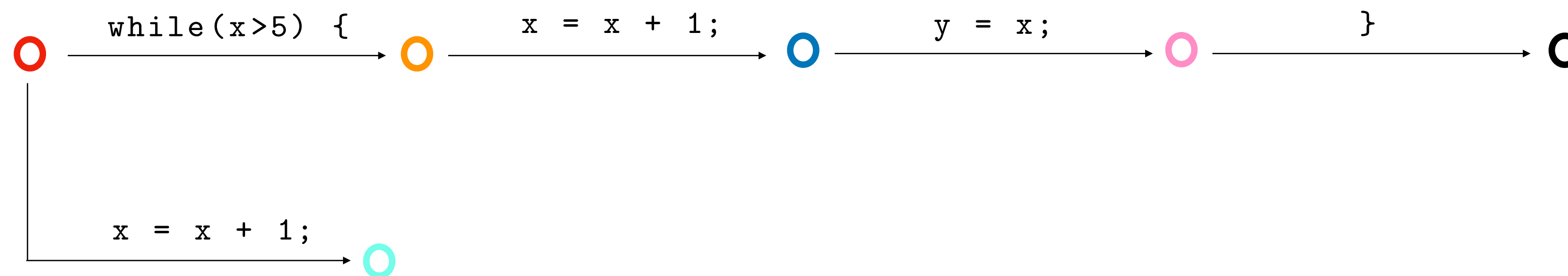
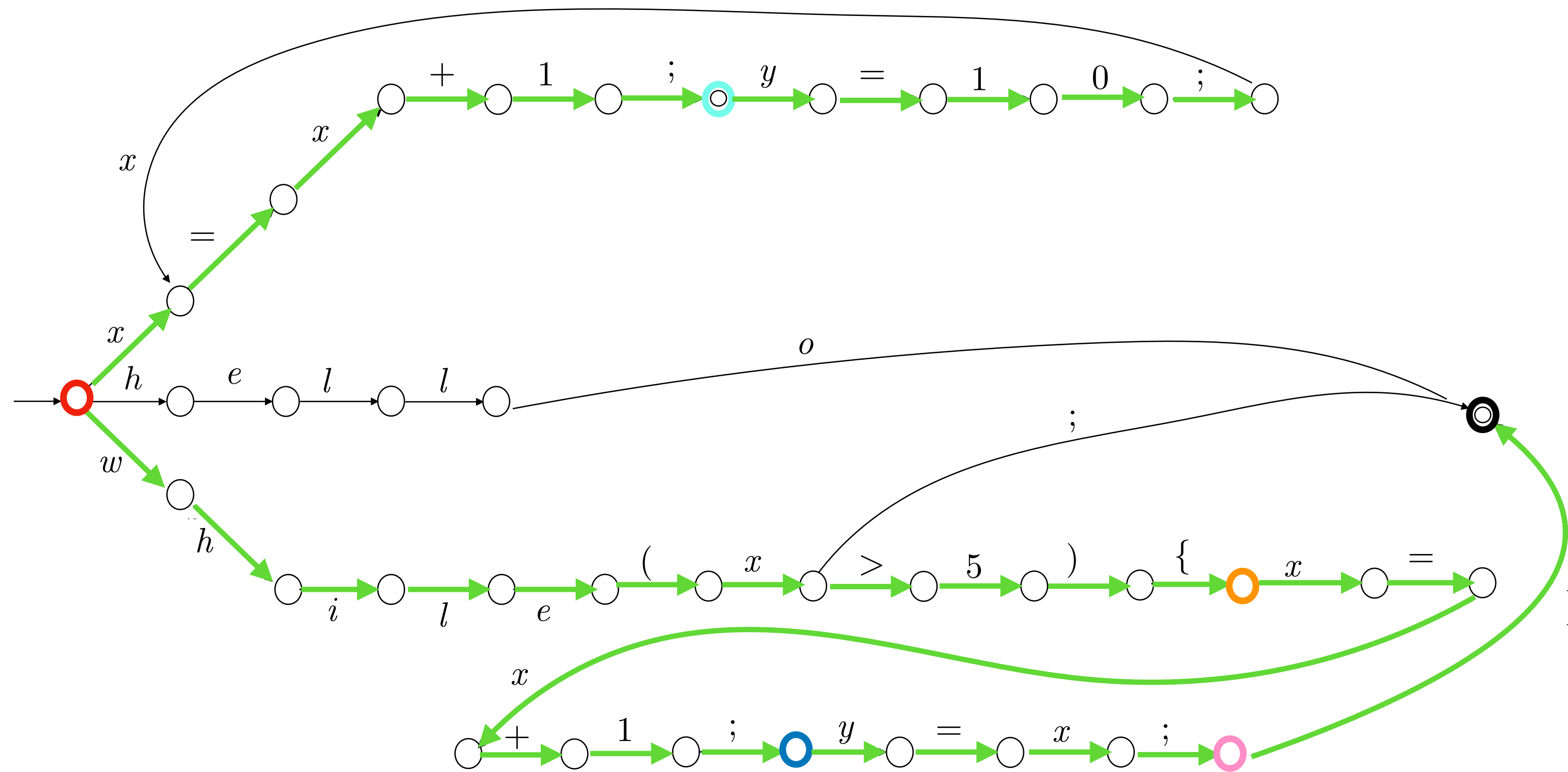
Bonus: Static analysis of eval



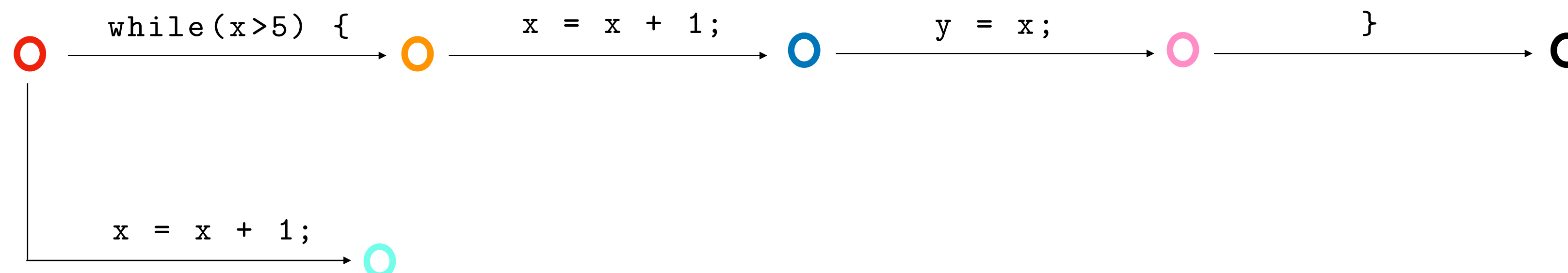
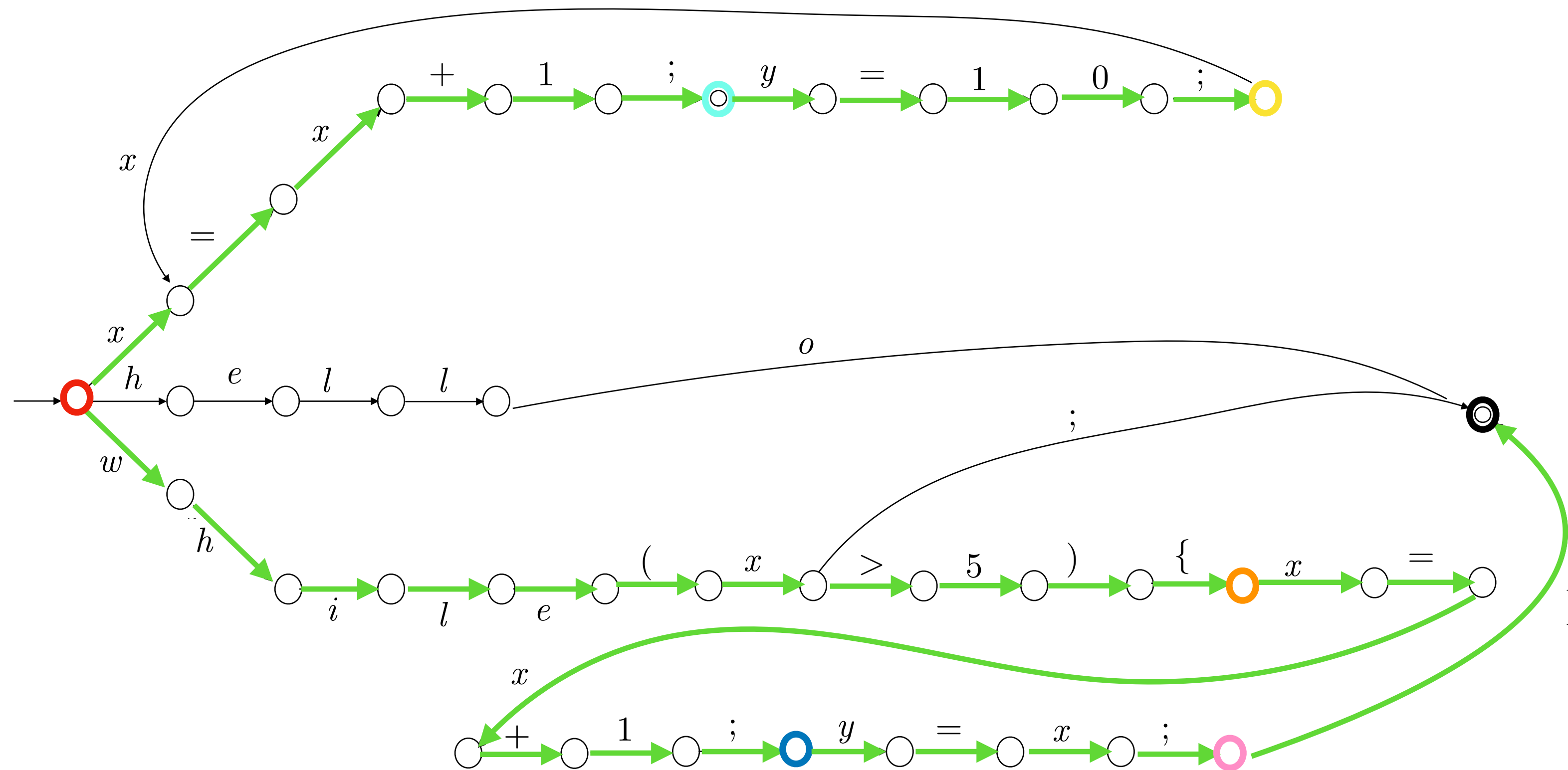
Bonus: Static analysis of eval



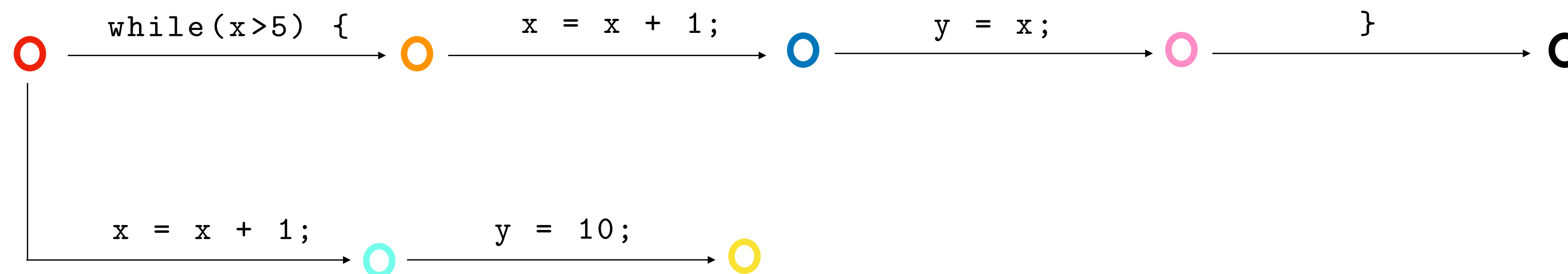
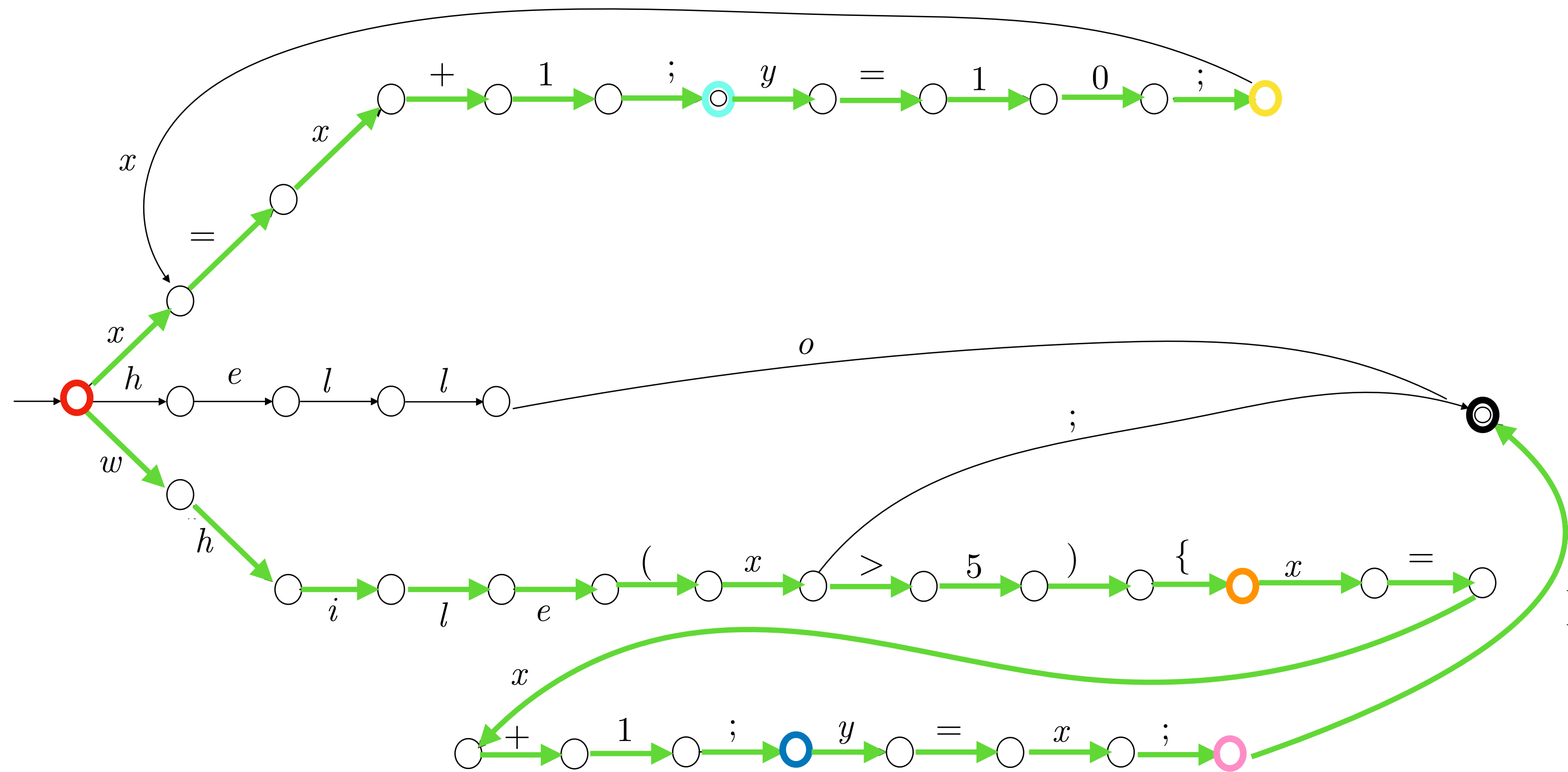
Bonus: Static analysis of eval



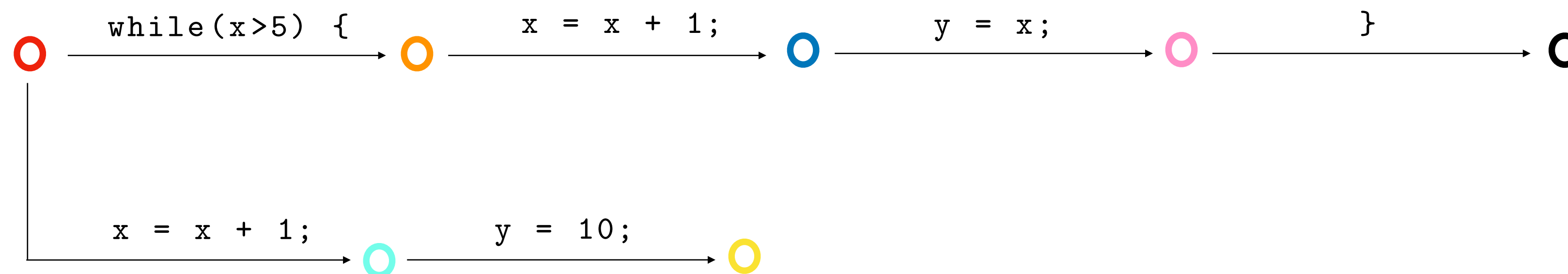
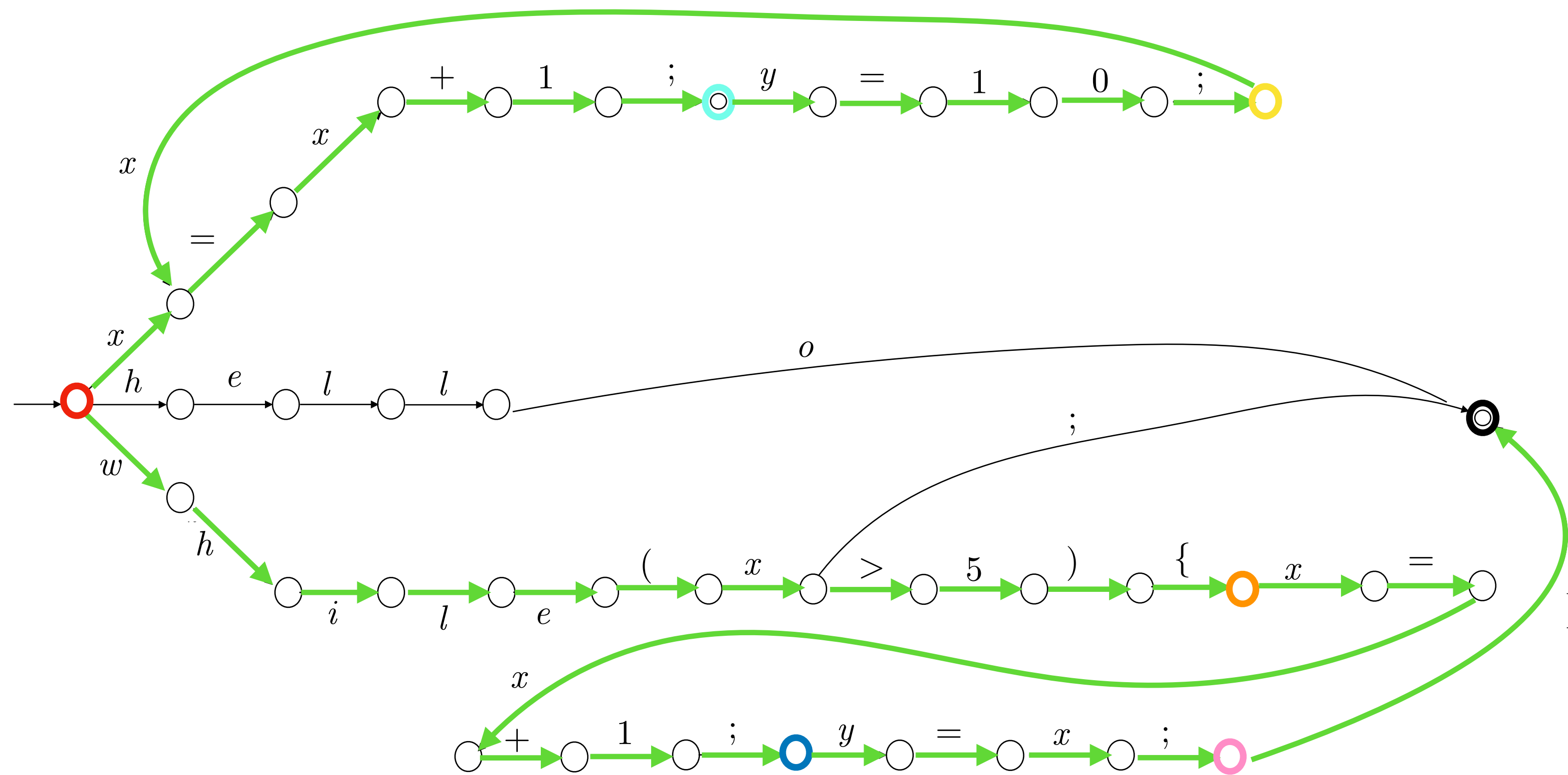
Bonus: Static analysis of eval



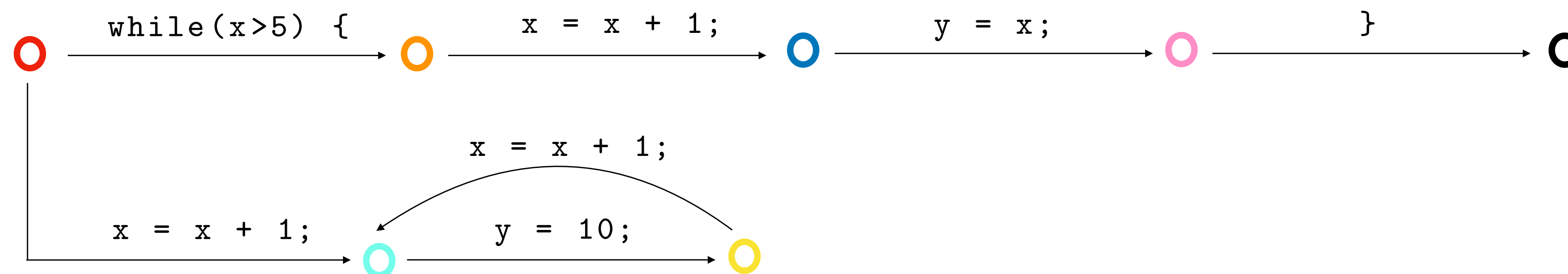
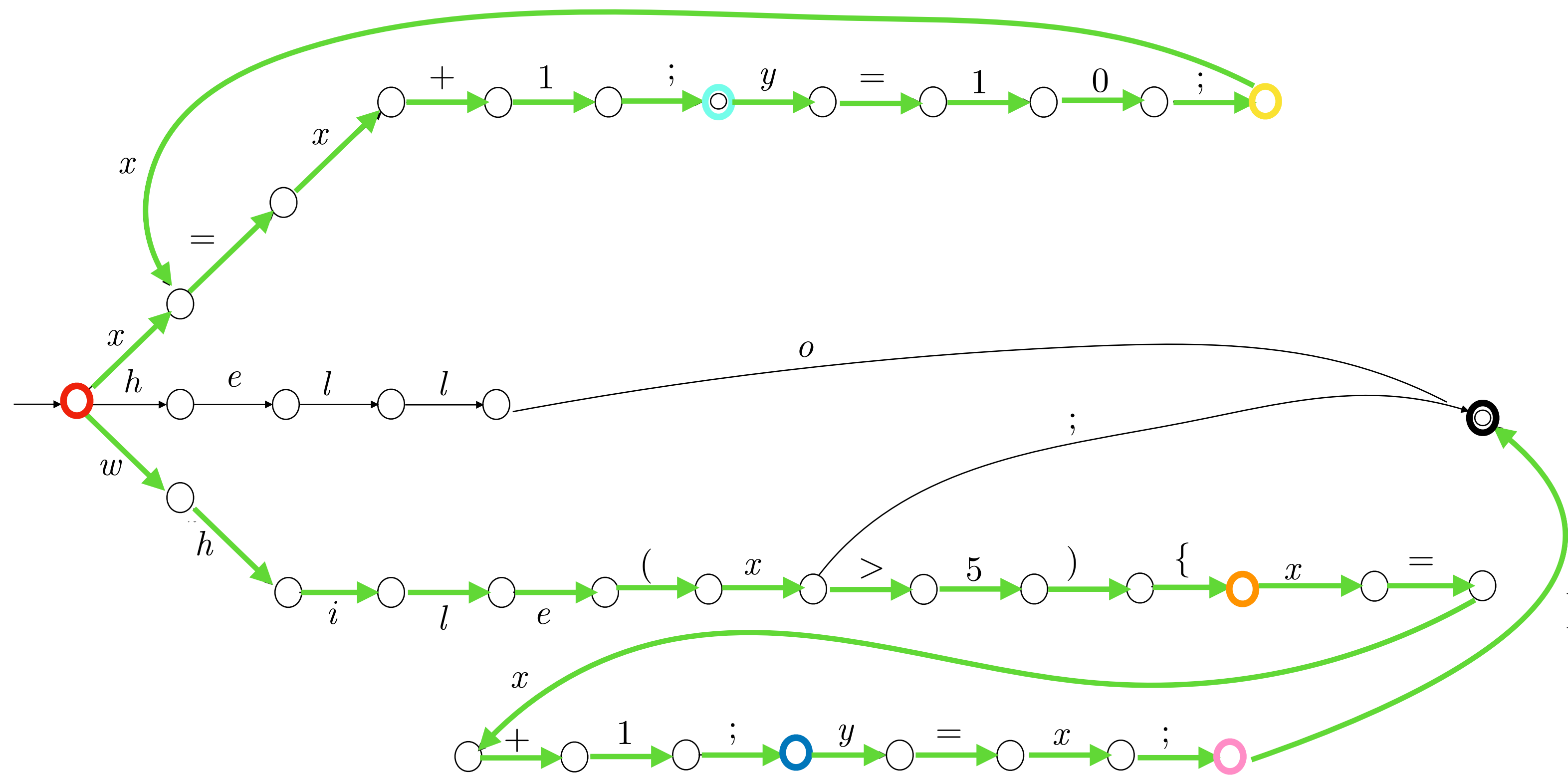
Bonus: Static analysis of eval



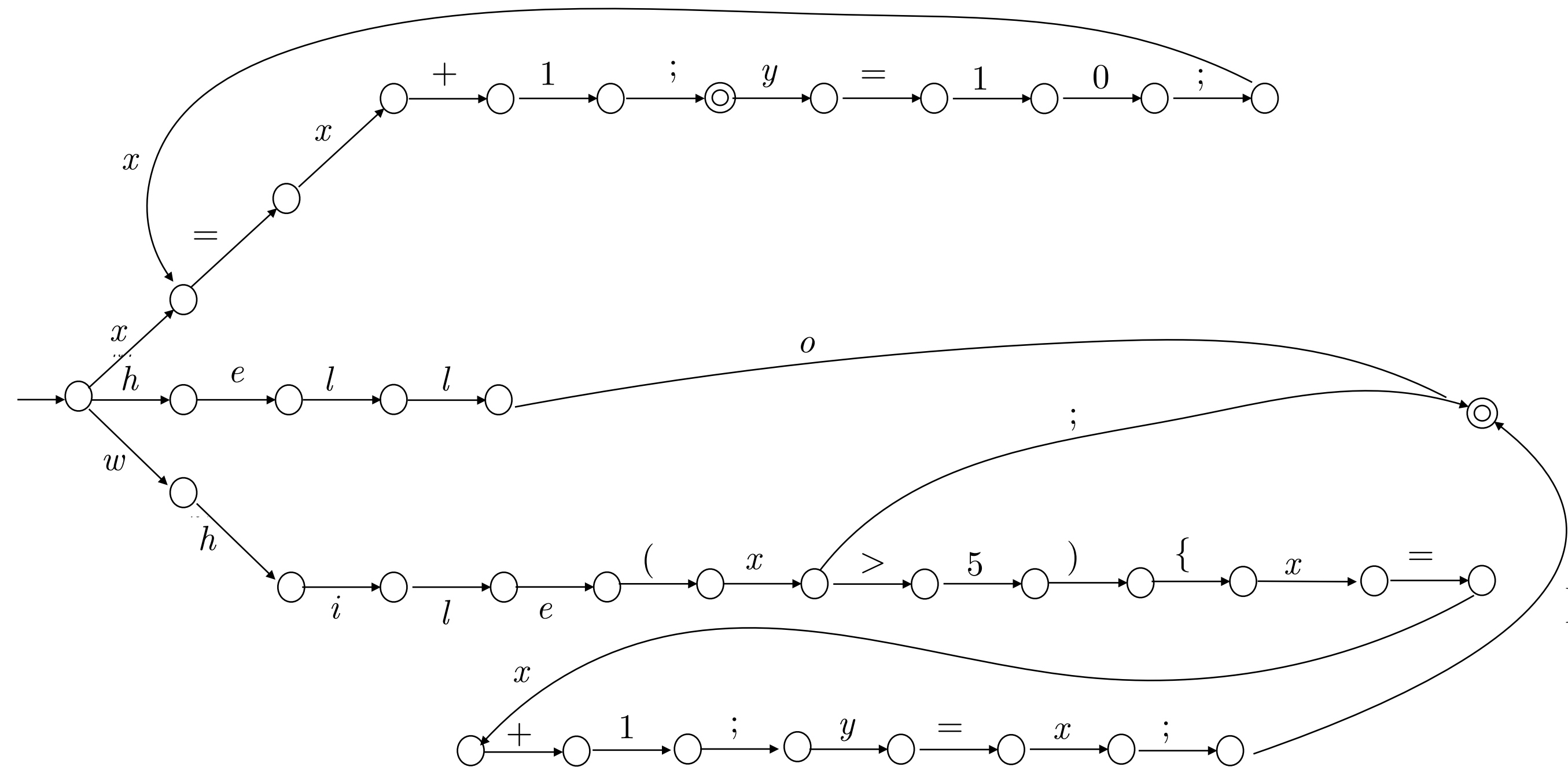
Bonus: Static analysis of eval



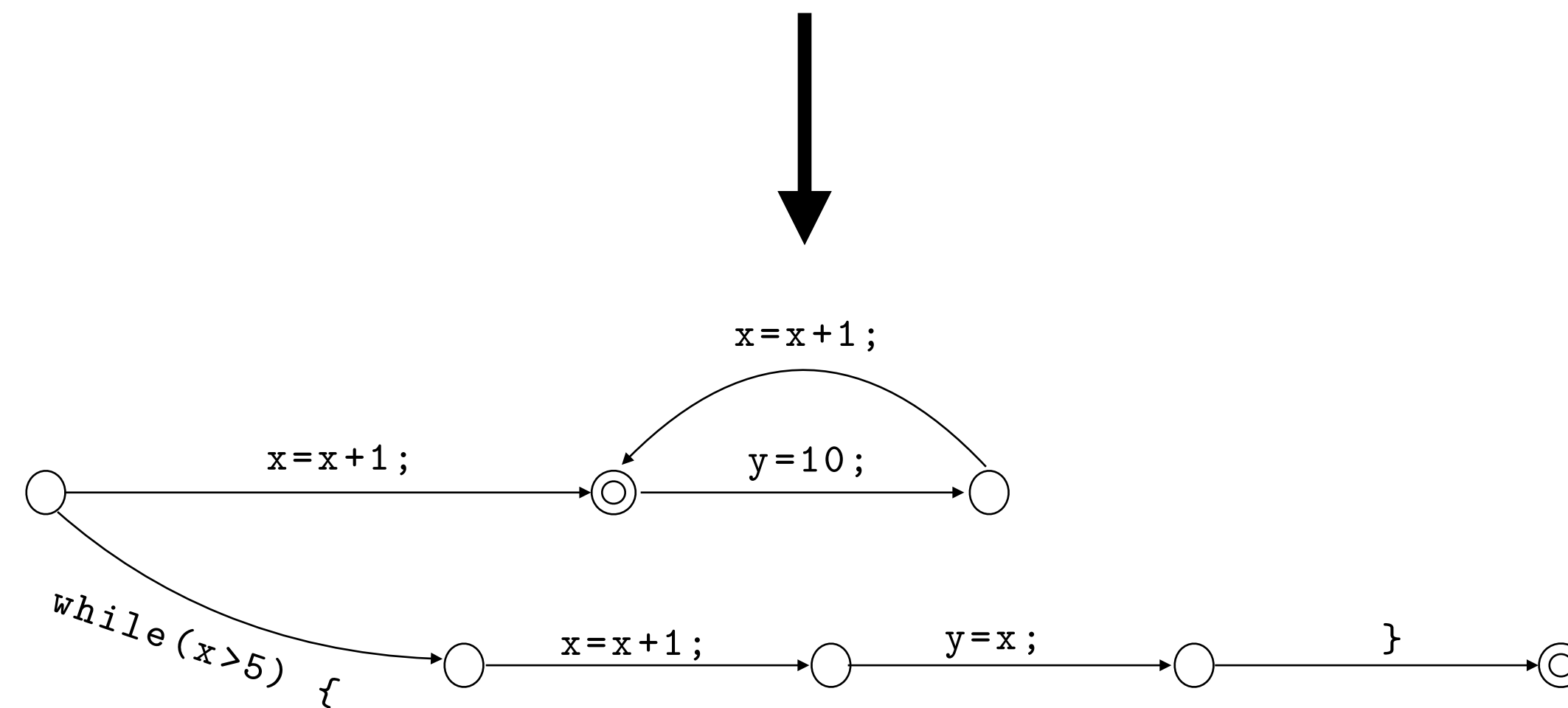
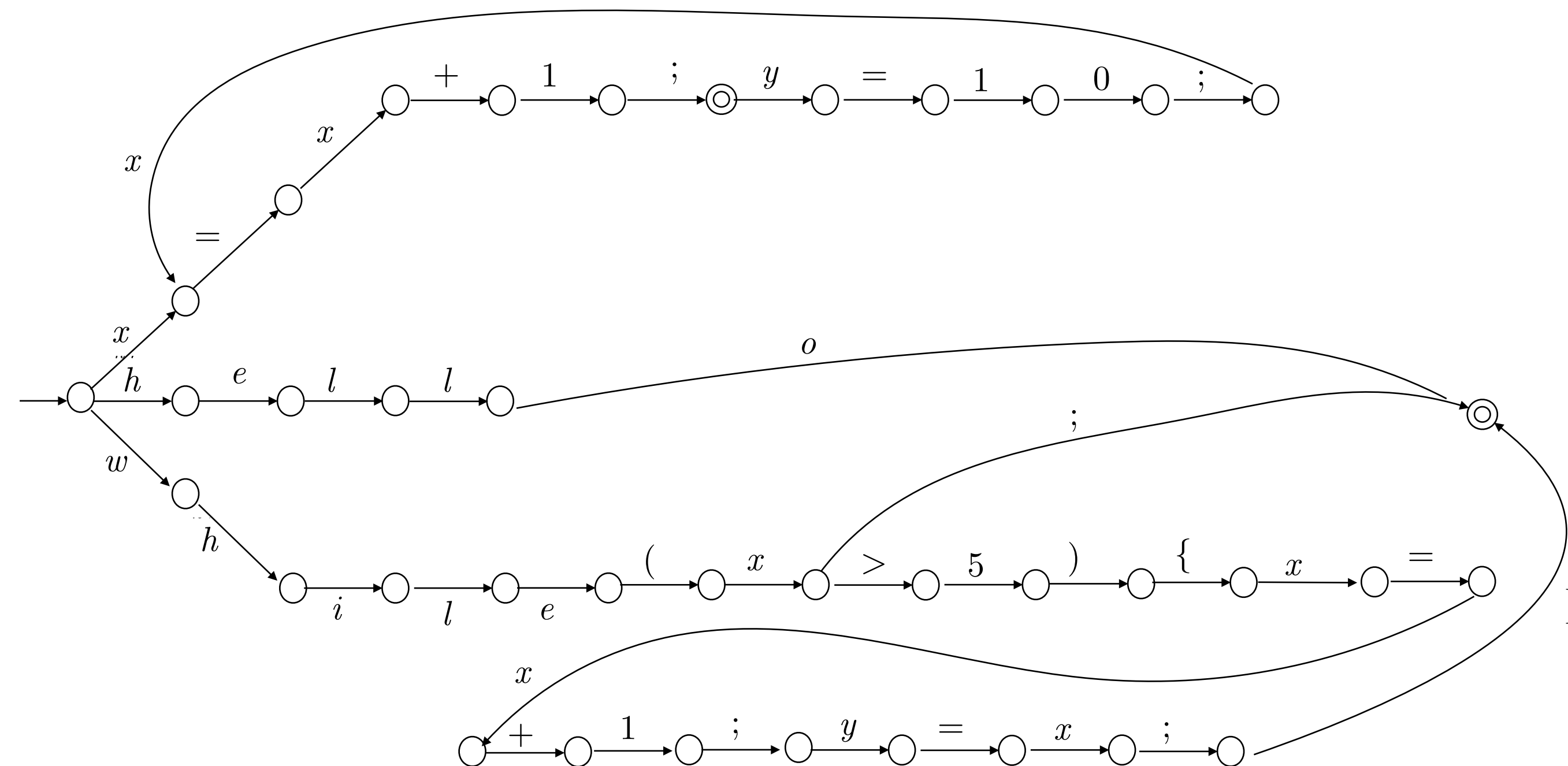
Bonus: Static analysis of eval



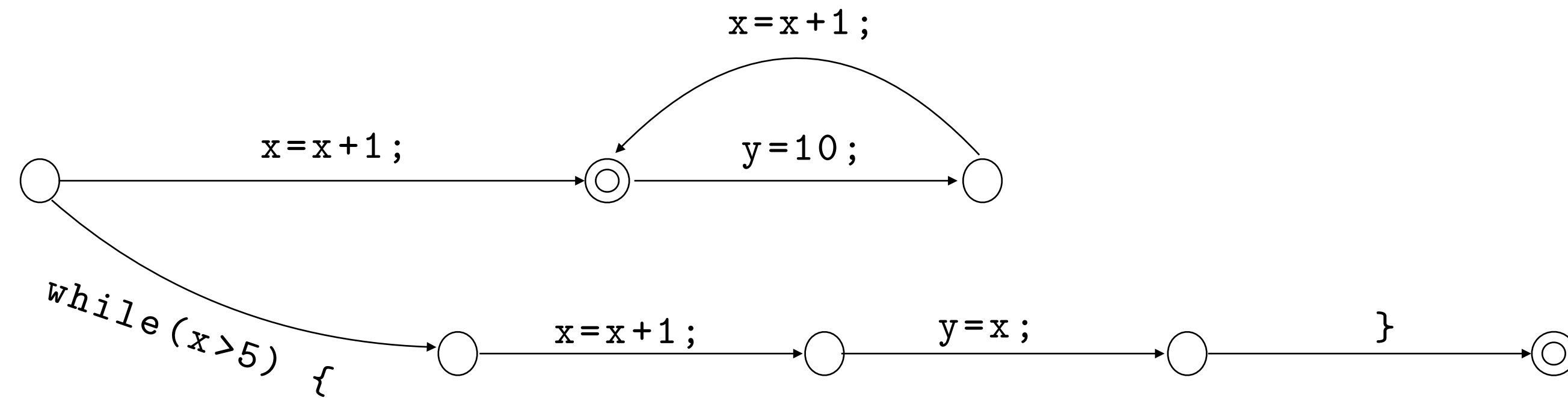
Static analysis of eval



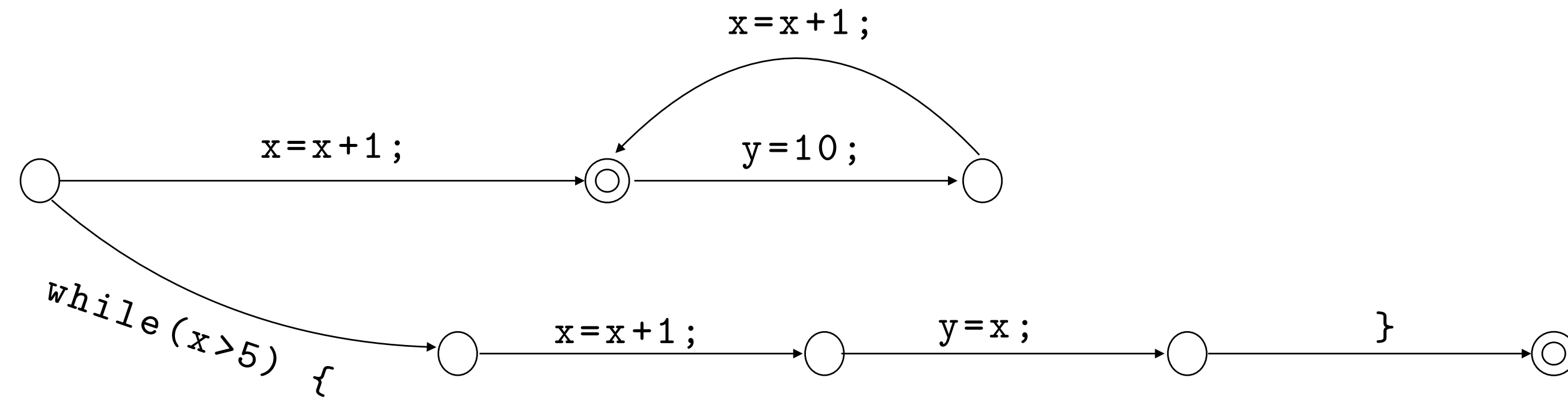
Static analysis of eval



Static analysis of eval



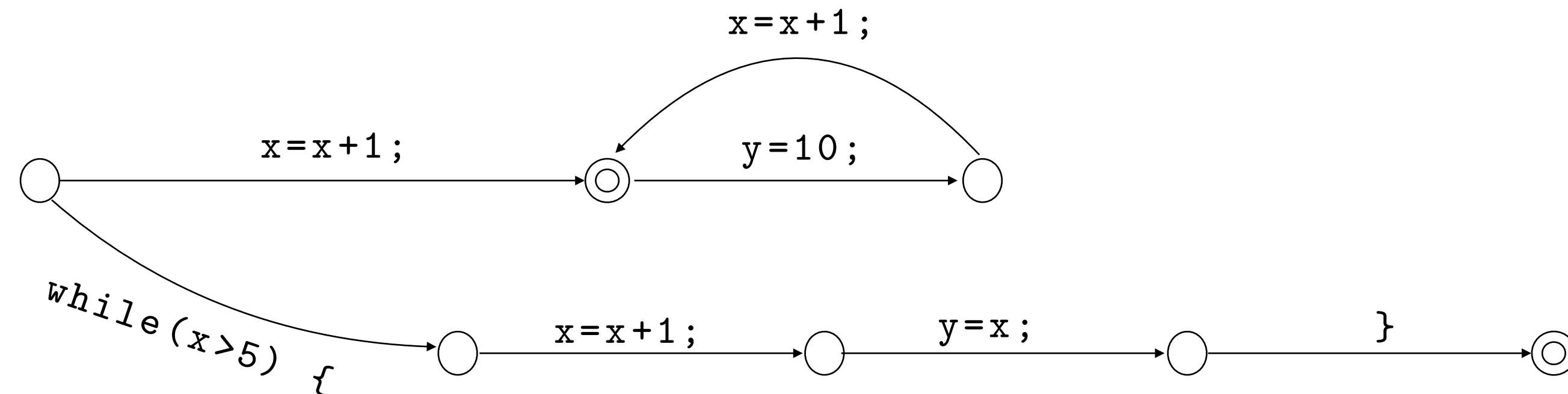
Static analysis of eval



To regular expression

`while(x>5) { x = x + 1; y = 10; } || x = x + 1; || x = x + 1; ( y = 10; x = x + 1; )*`

Static analysis of eval



To regular expression

`while(x>5) { x = x + 1; y = 10; } || x = x + 1; || x = x + 1; ( y = 10; x = x + 1; )*`

How to convert regular expressions to something the abstract interpreter can understand?

Static analysis of eval

We report the conversion of or, concat and Kleene star expressions to control-flow graph

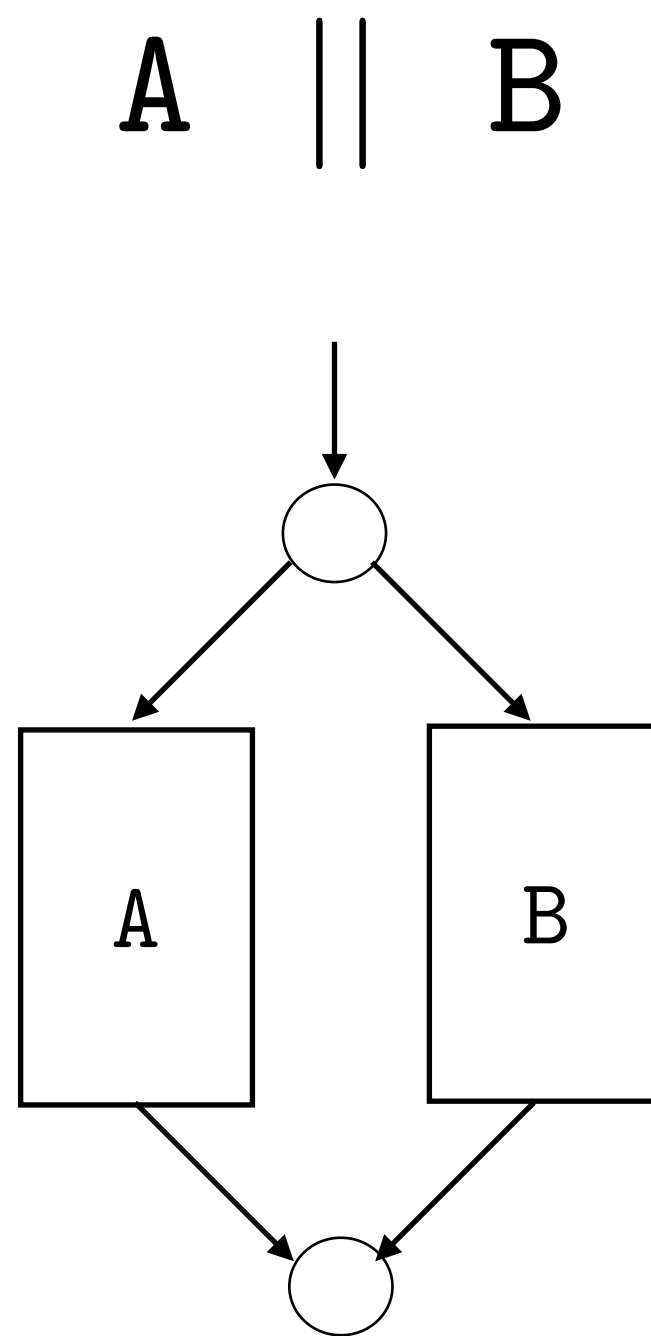
$A \parallel B$

AB

A^*

Static analysis of eval

We report the conversion of or, concat and Kleene star expressions to control-flow graph

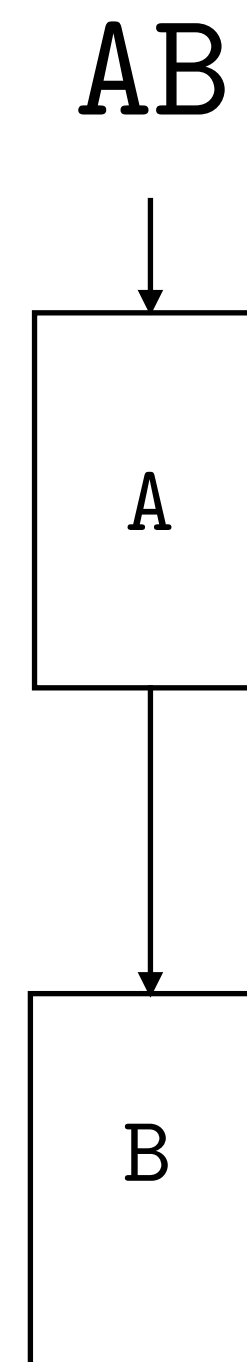
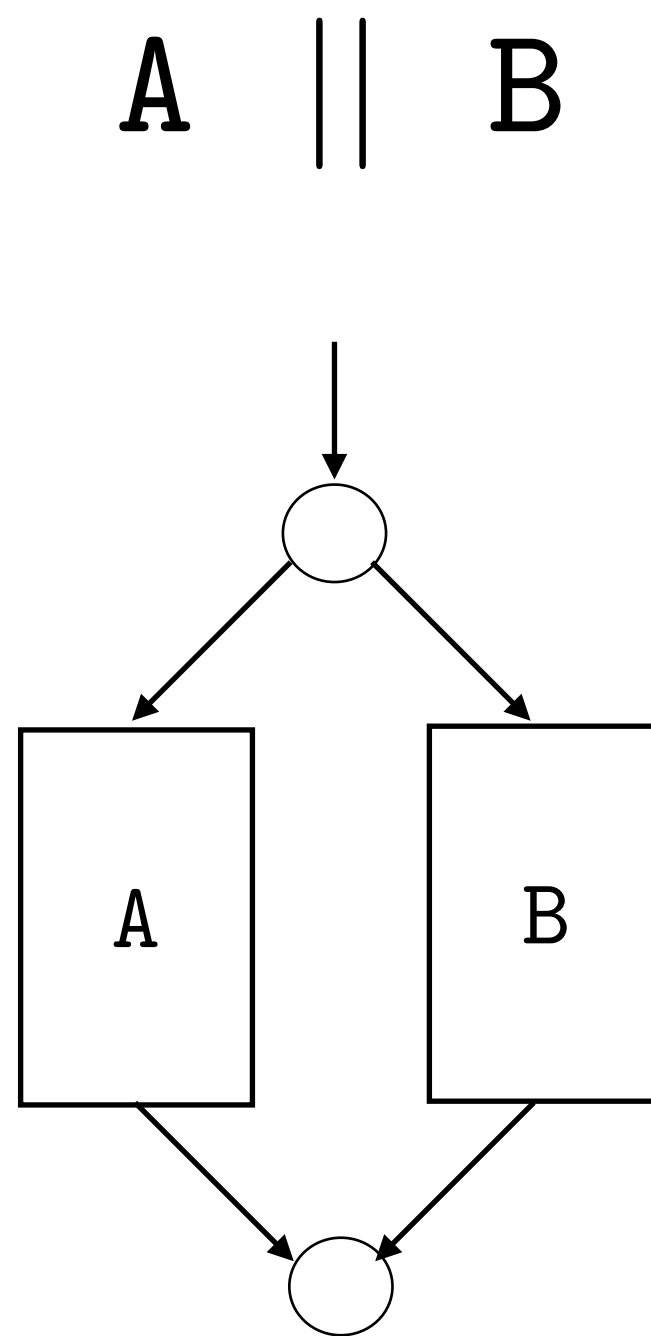


AB

A^*

Static analysis of eval

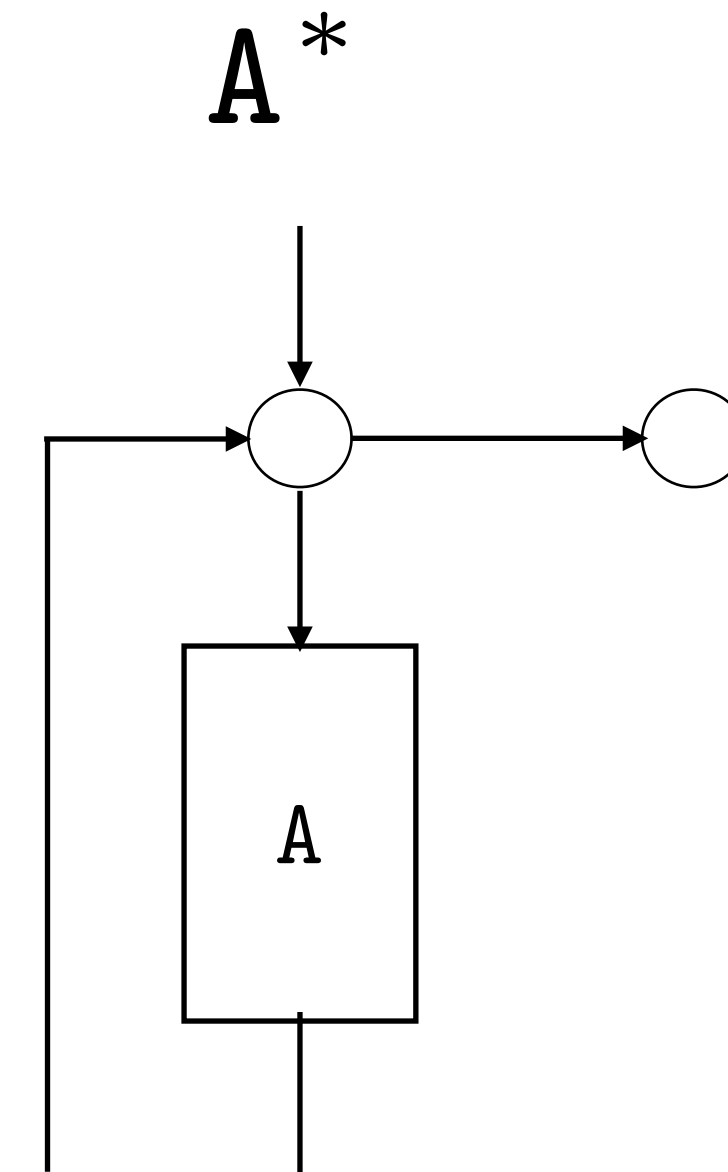
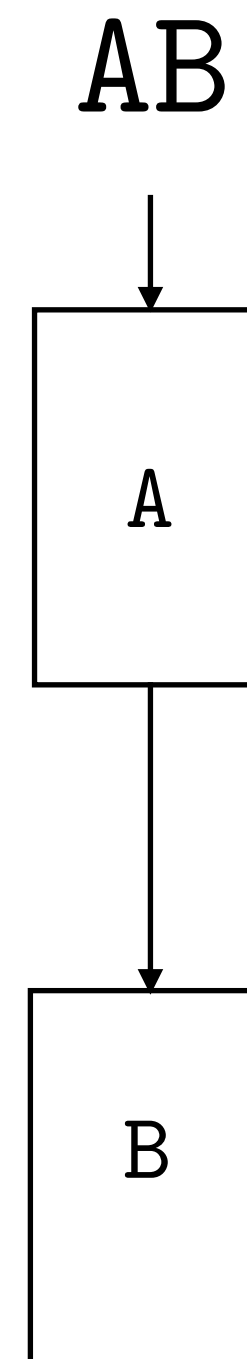
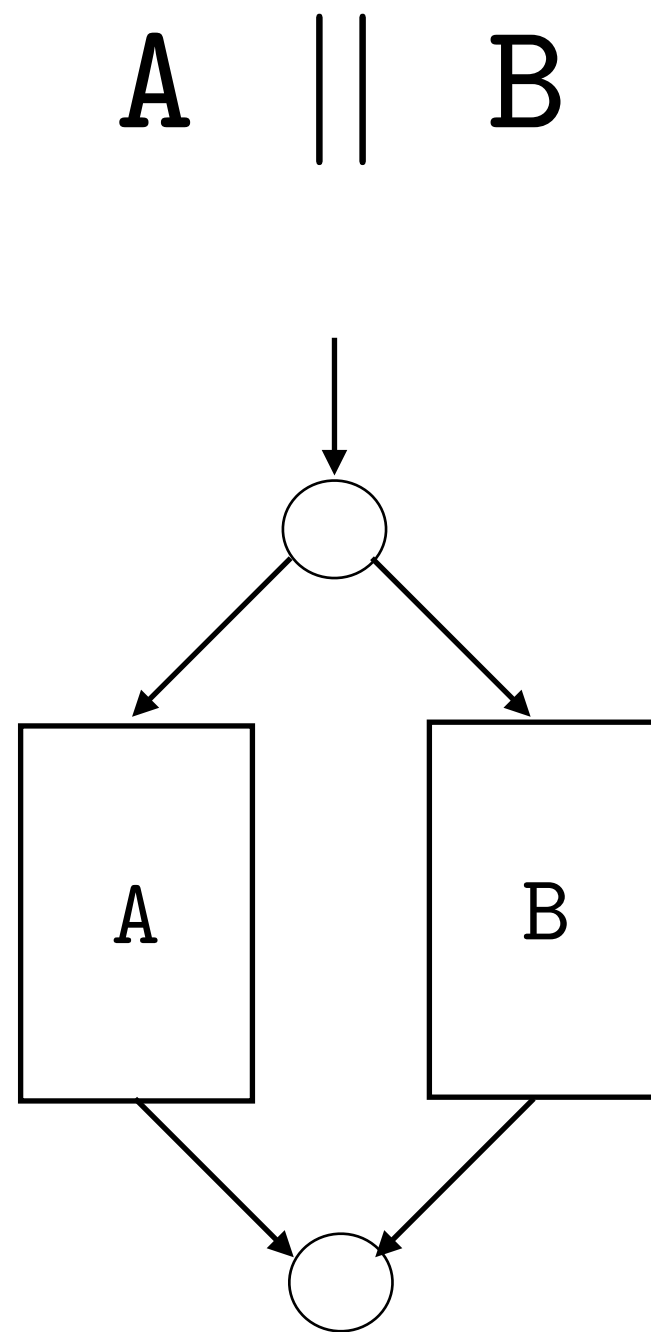
We report the conversion of or, concat and Kleene star expressions to control-flow graph



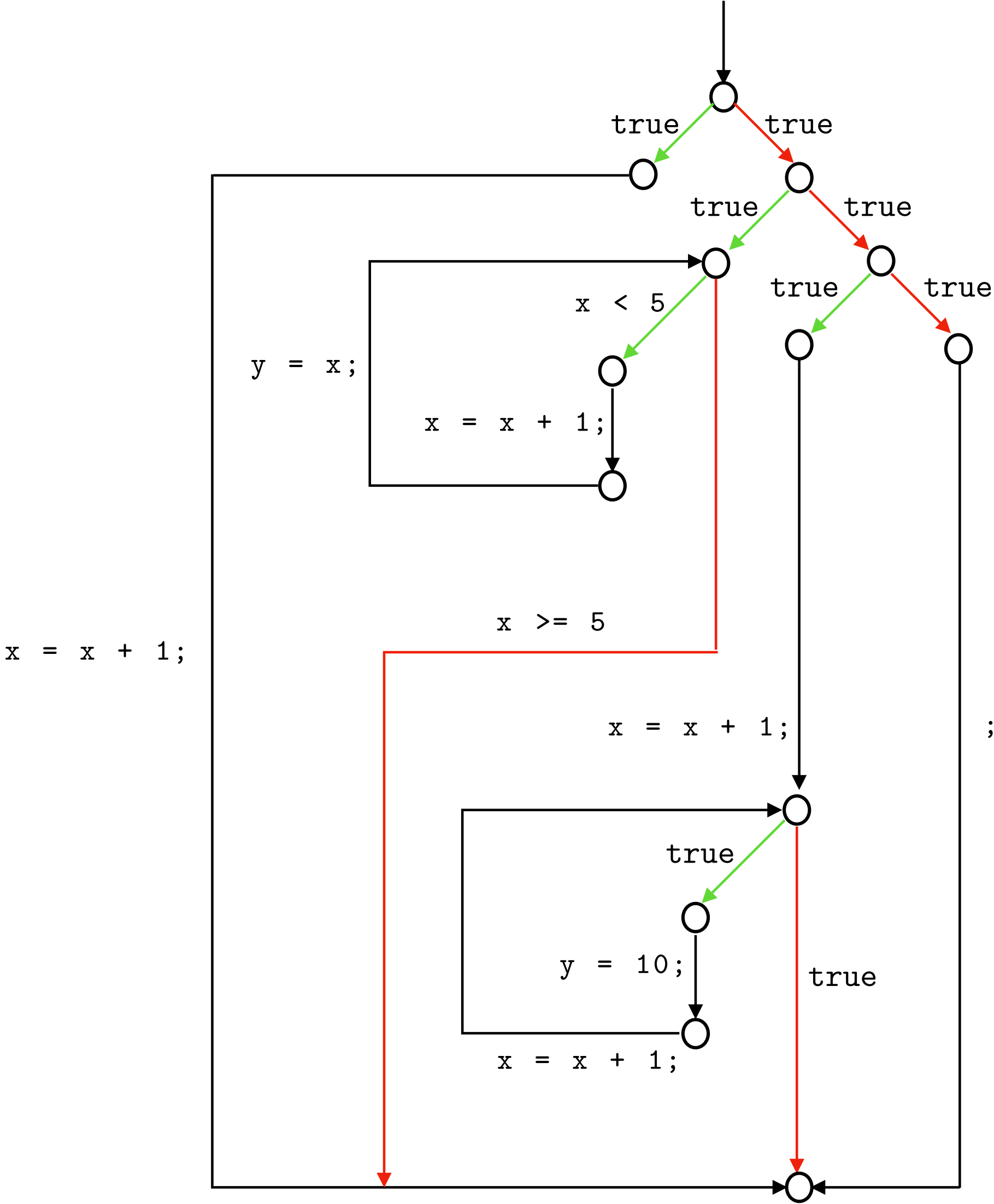
A^*

Static analysis of eval

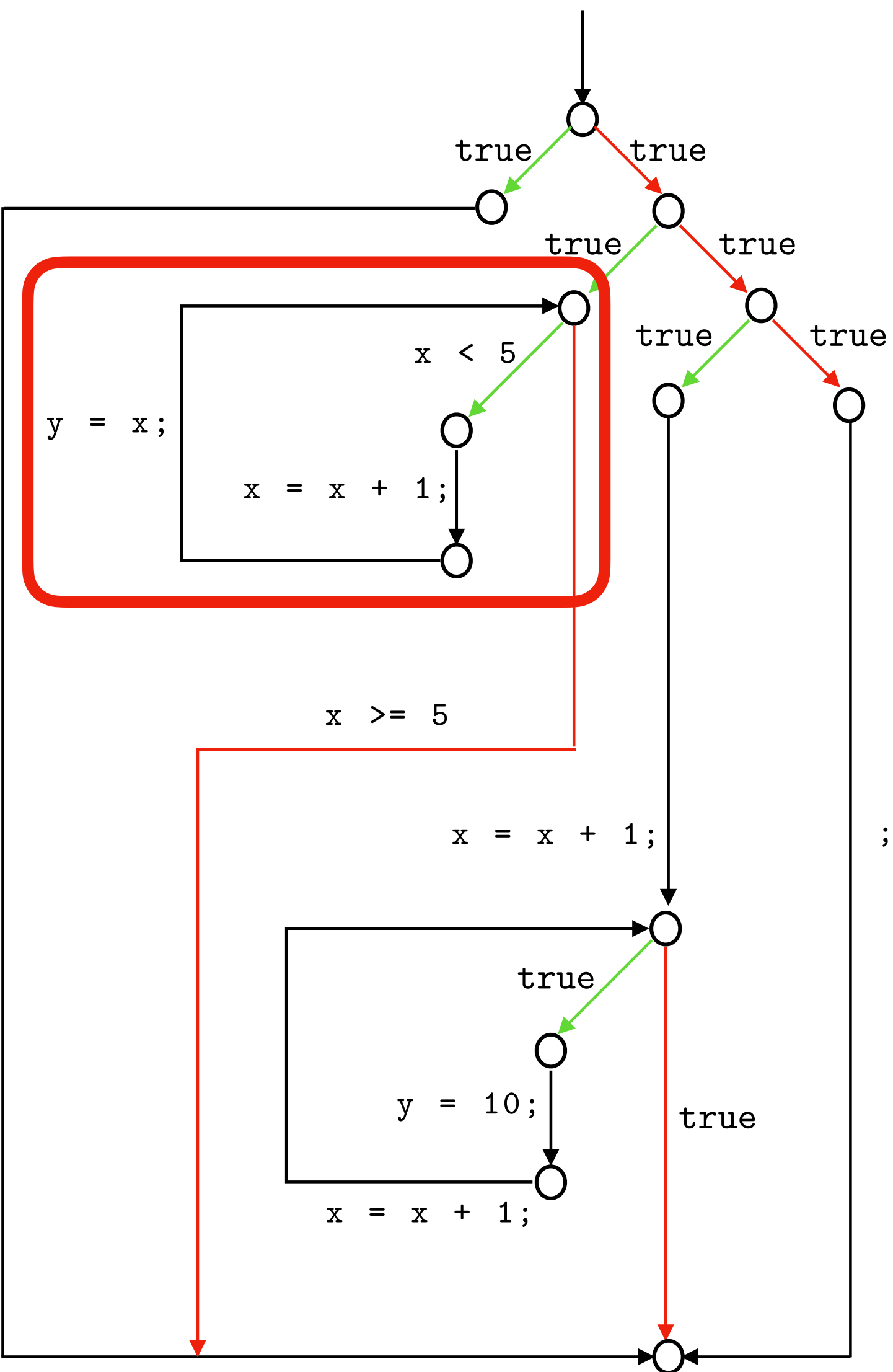
We report the conversion of or, concat and Kleene star expressions to control-flow graph



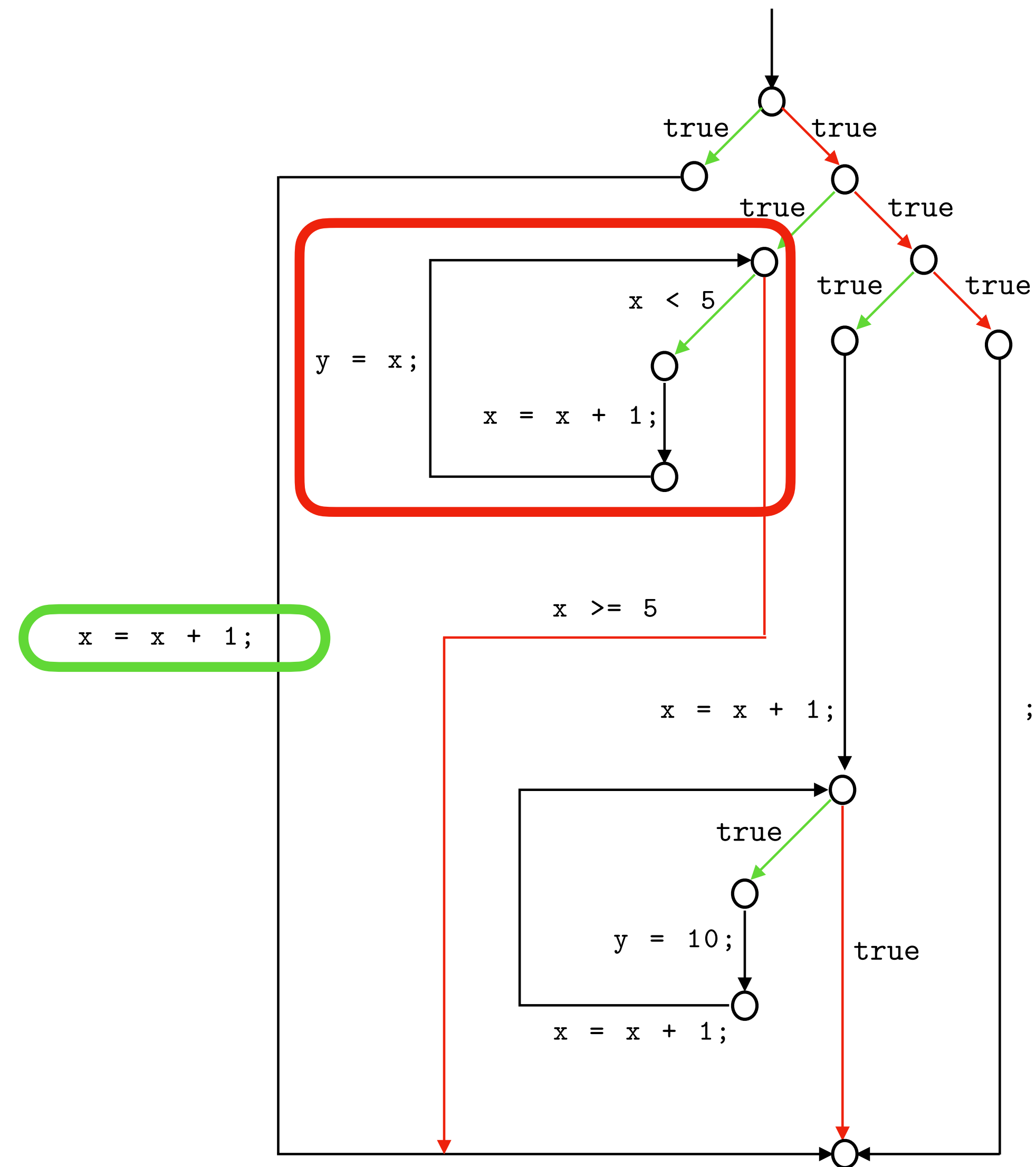
```
while(x>5) { x = x + 1; y = 10; } || x = x + 1; || x = x + 1; ( y = 10; x = x + 1; )*
```



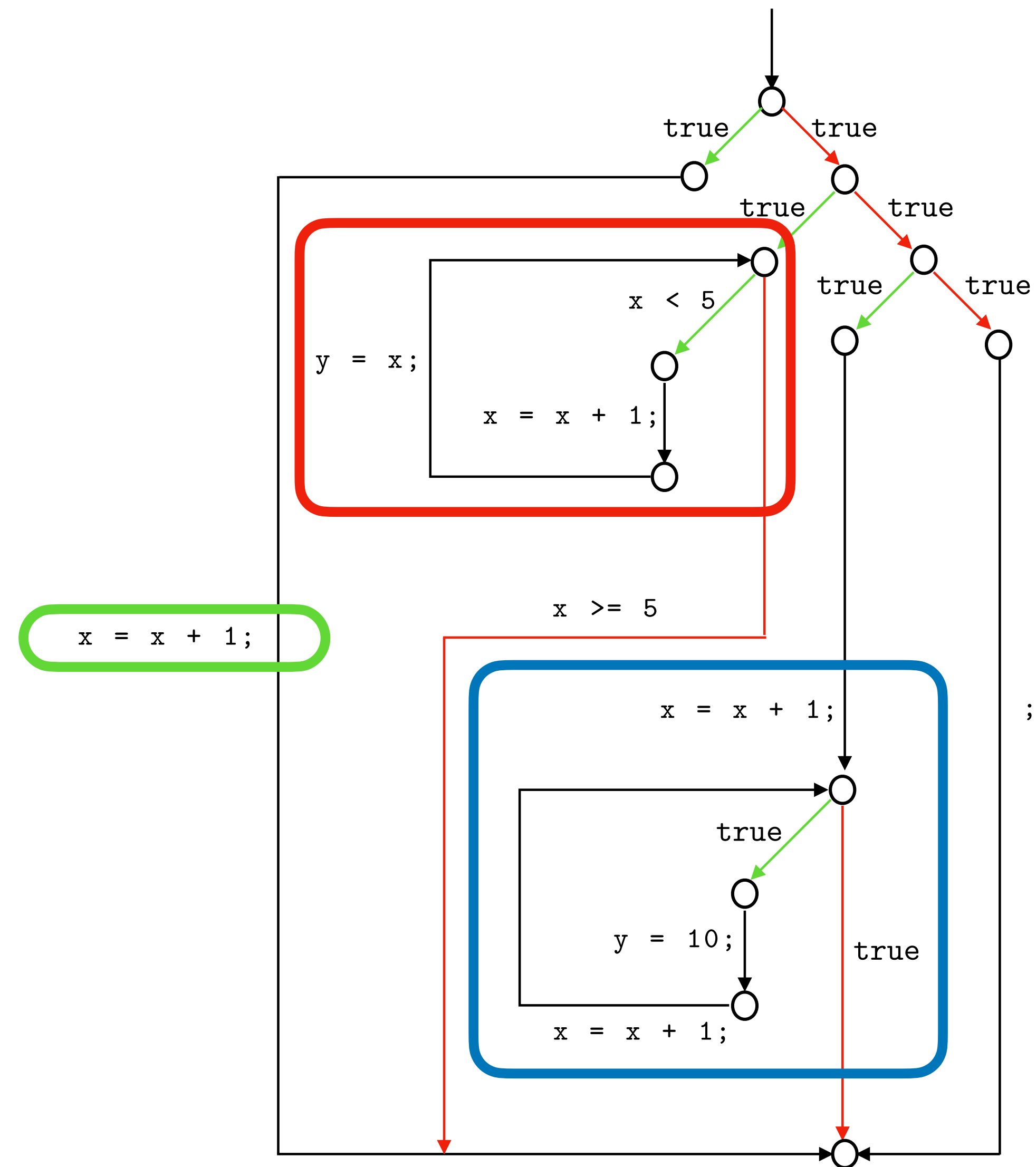
```
while(x>5) { x = x + 1; y = 10; }
```



`while(x>5) { x = x + 1; y = 10; } || x = x + 1; || x = x + 1; ( y = 10; x = x + 1; )*`



`while(x>5) { x = x + 1; y = 10; }` || `x = x + 1;` || `x = x + 1; ( y = 10; x = x + 1; )*`



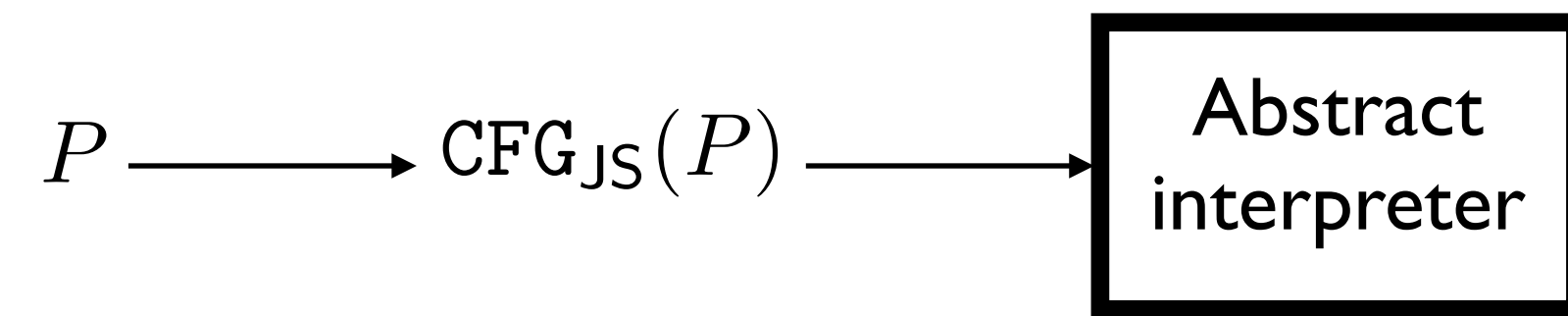
Architecture

P

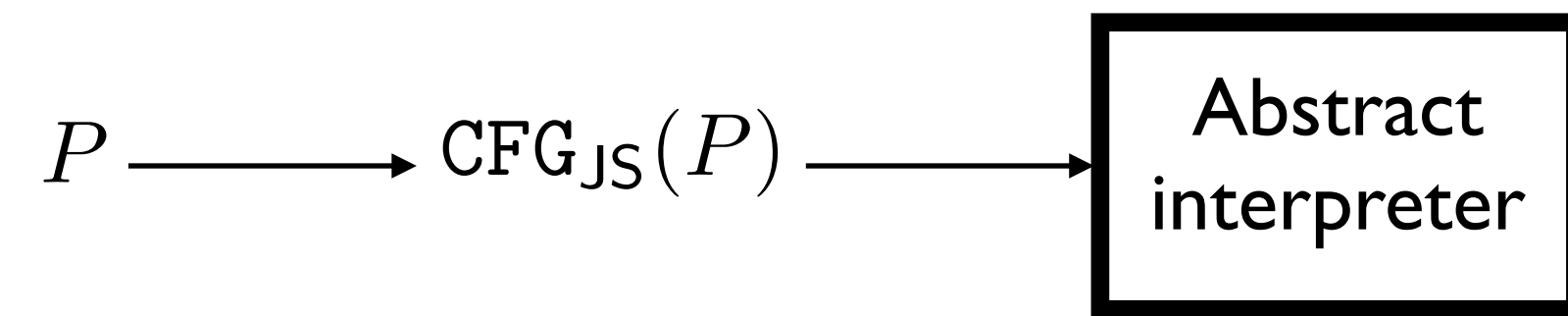
Architecture

$$P \longrightarrow \text{CFG}_{\text{JS}}(P)$$

Architecture

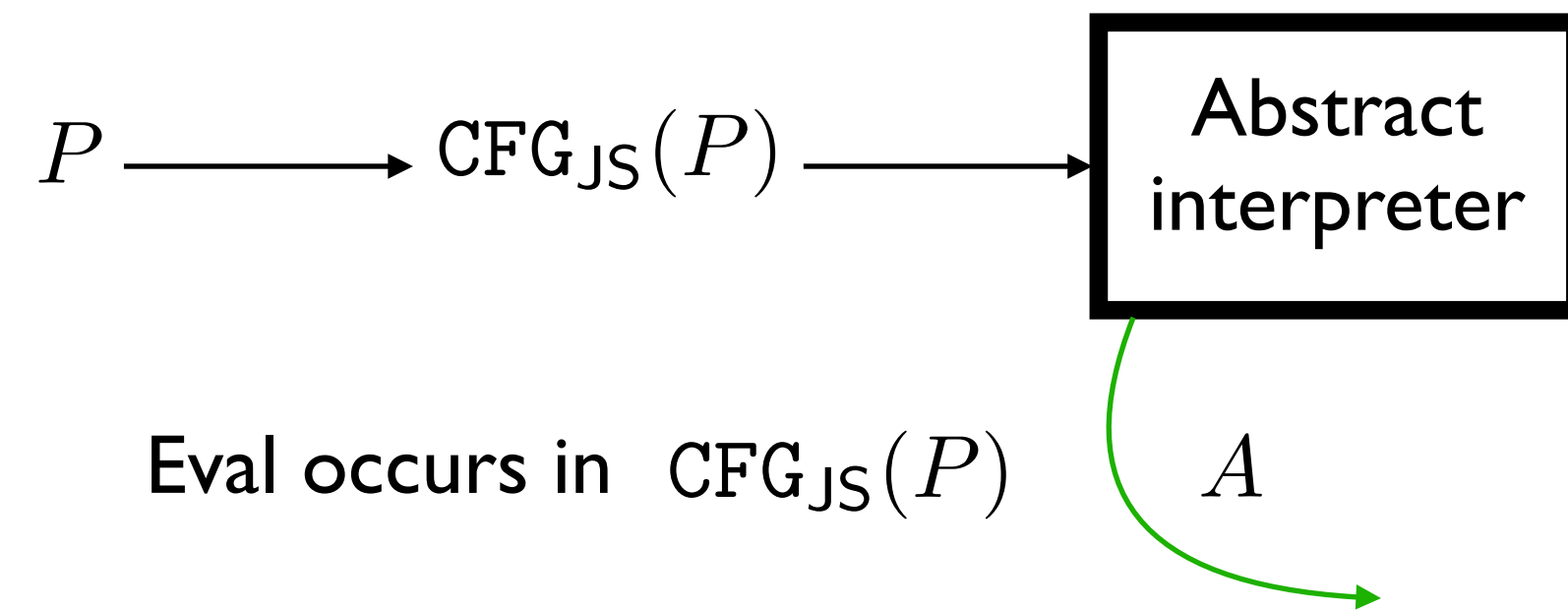


Architecture

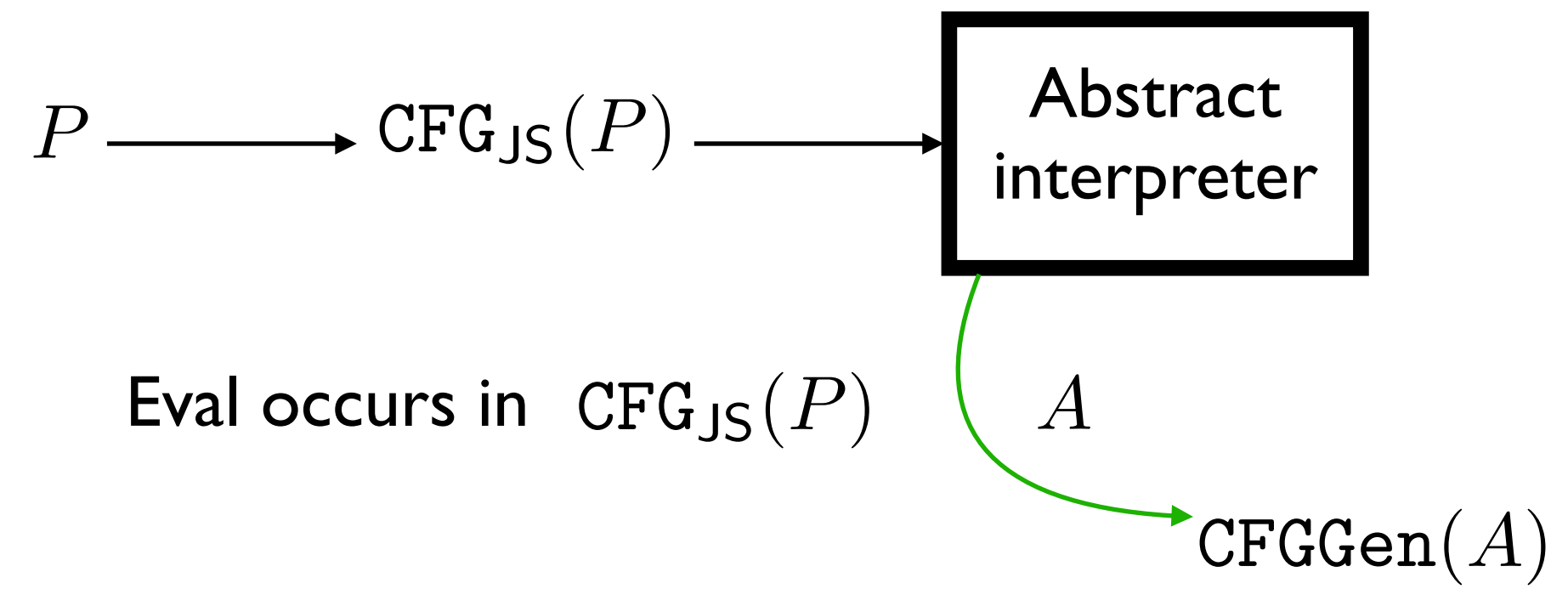


Eval occurs in $\text{CFG}_{\text{JS}}(P)$

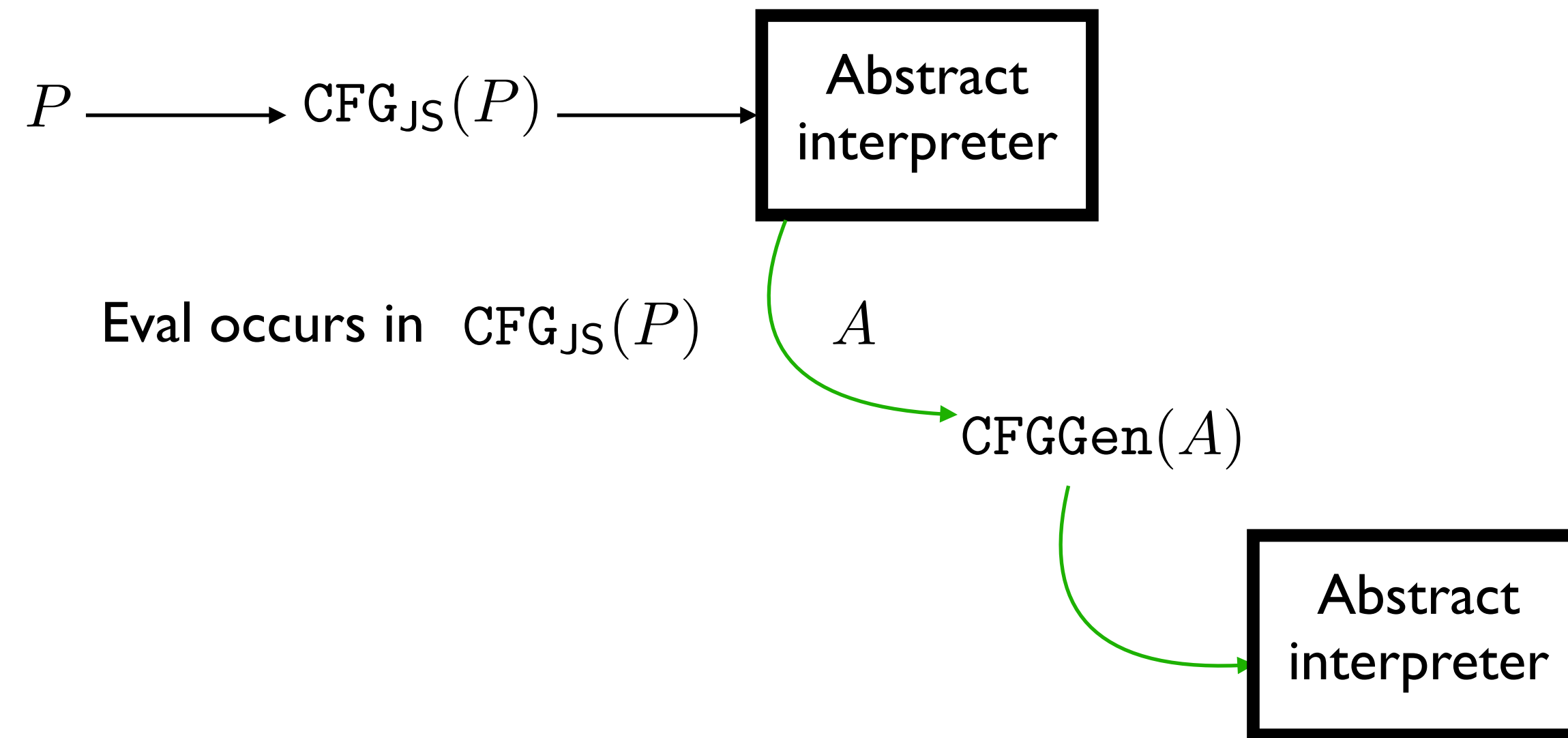
Architecture



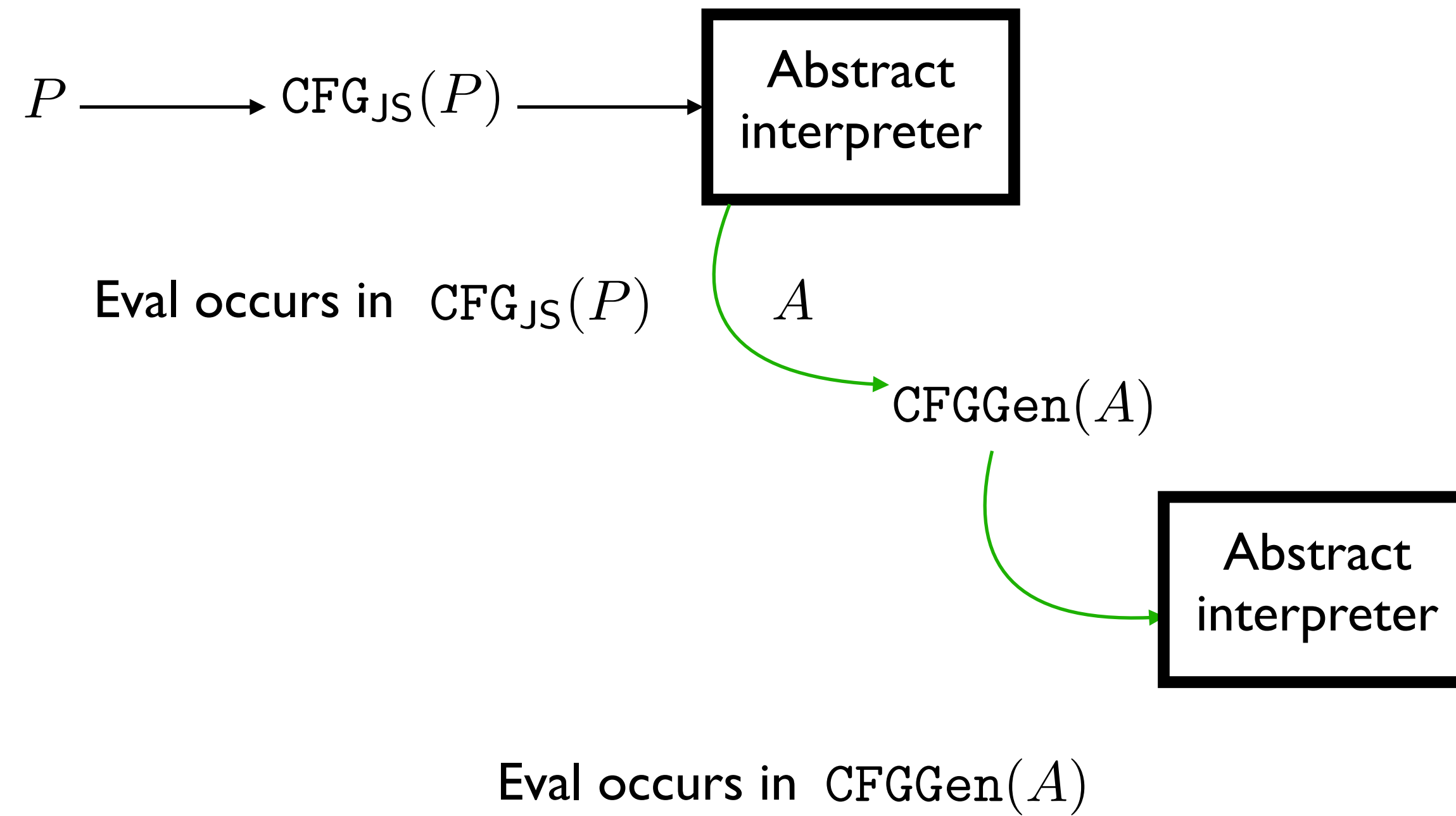
Architecture



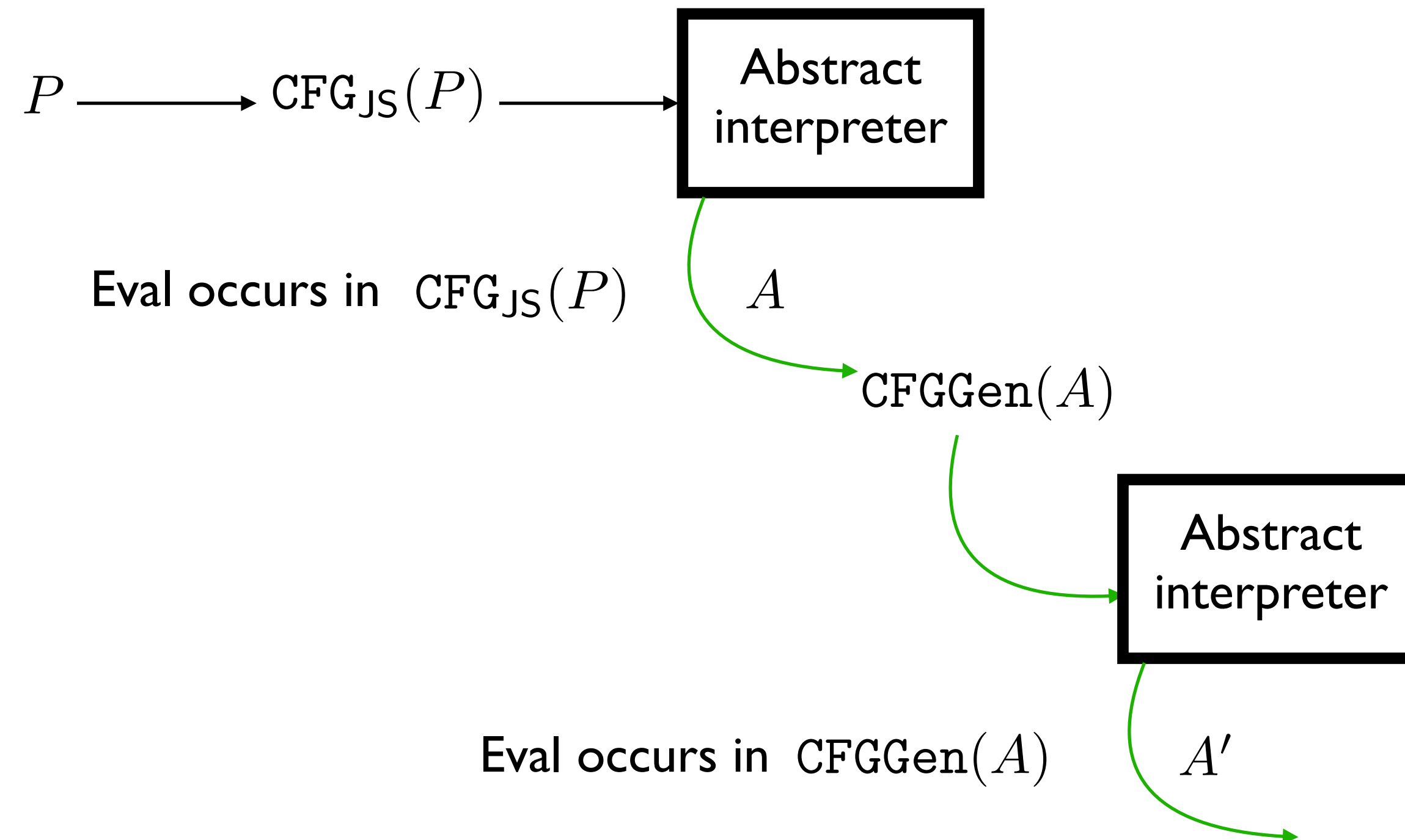
Architecture



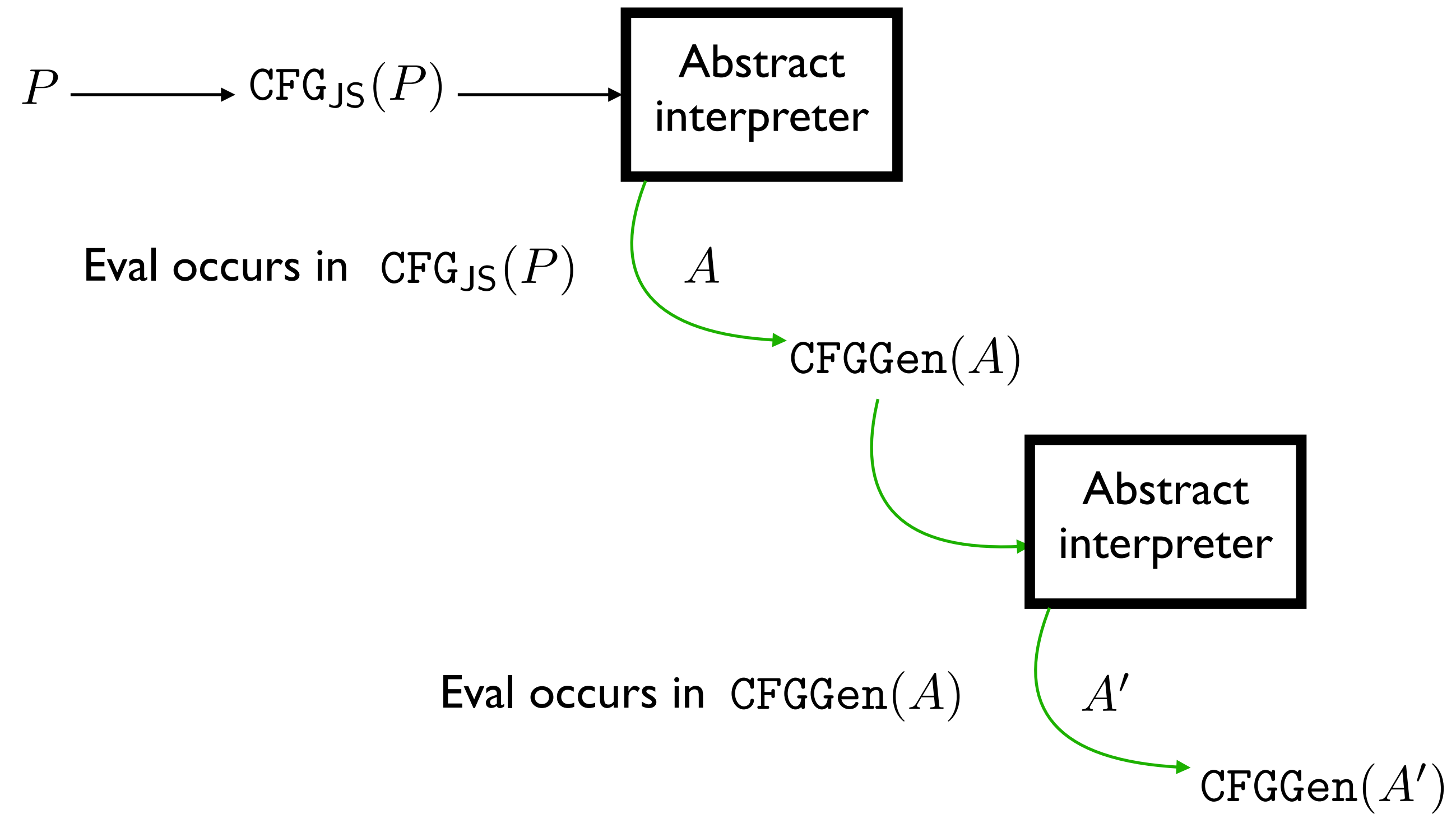
Architecture



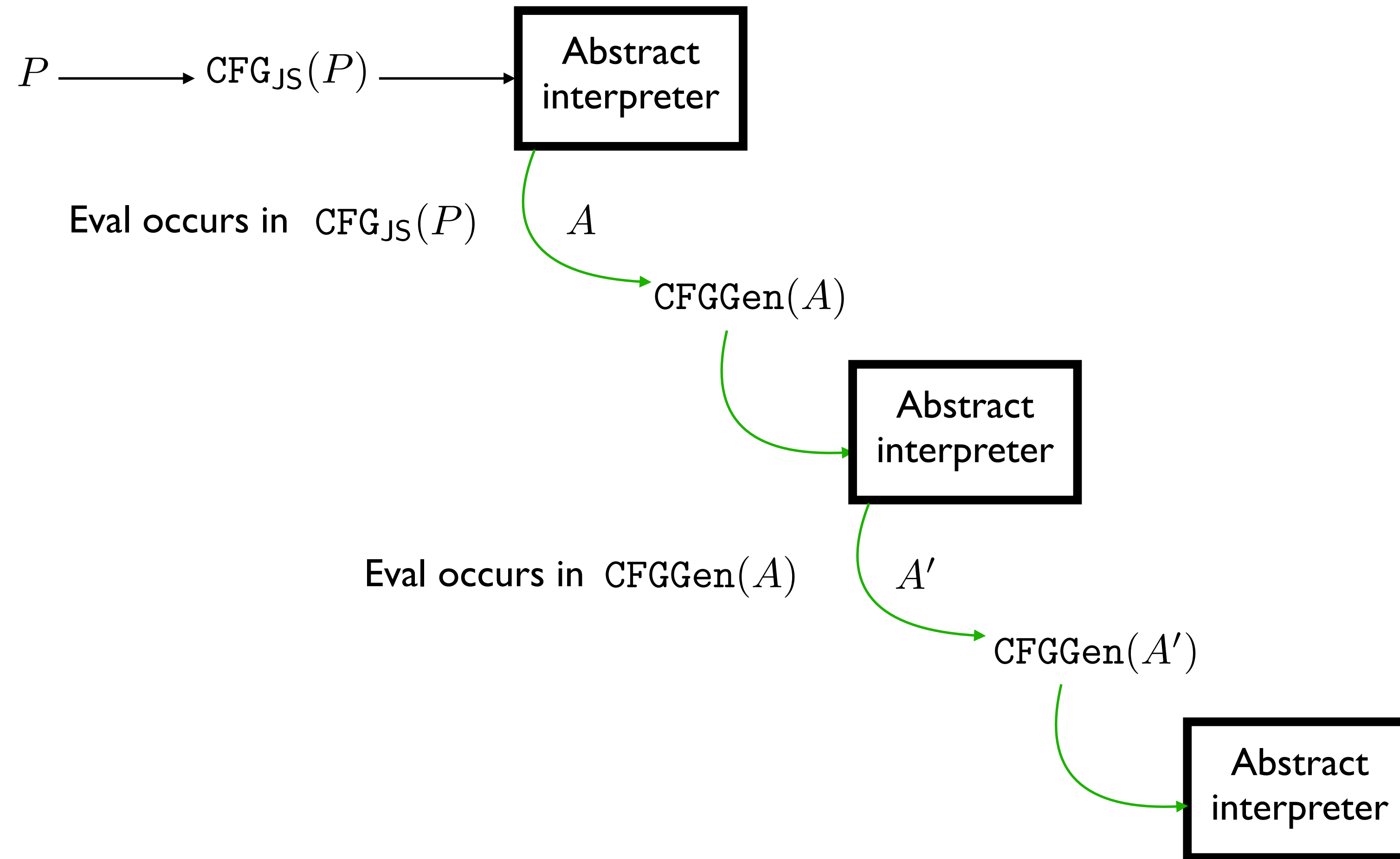
Architecture



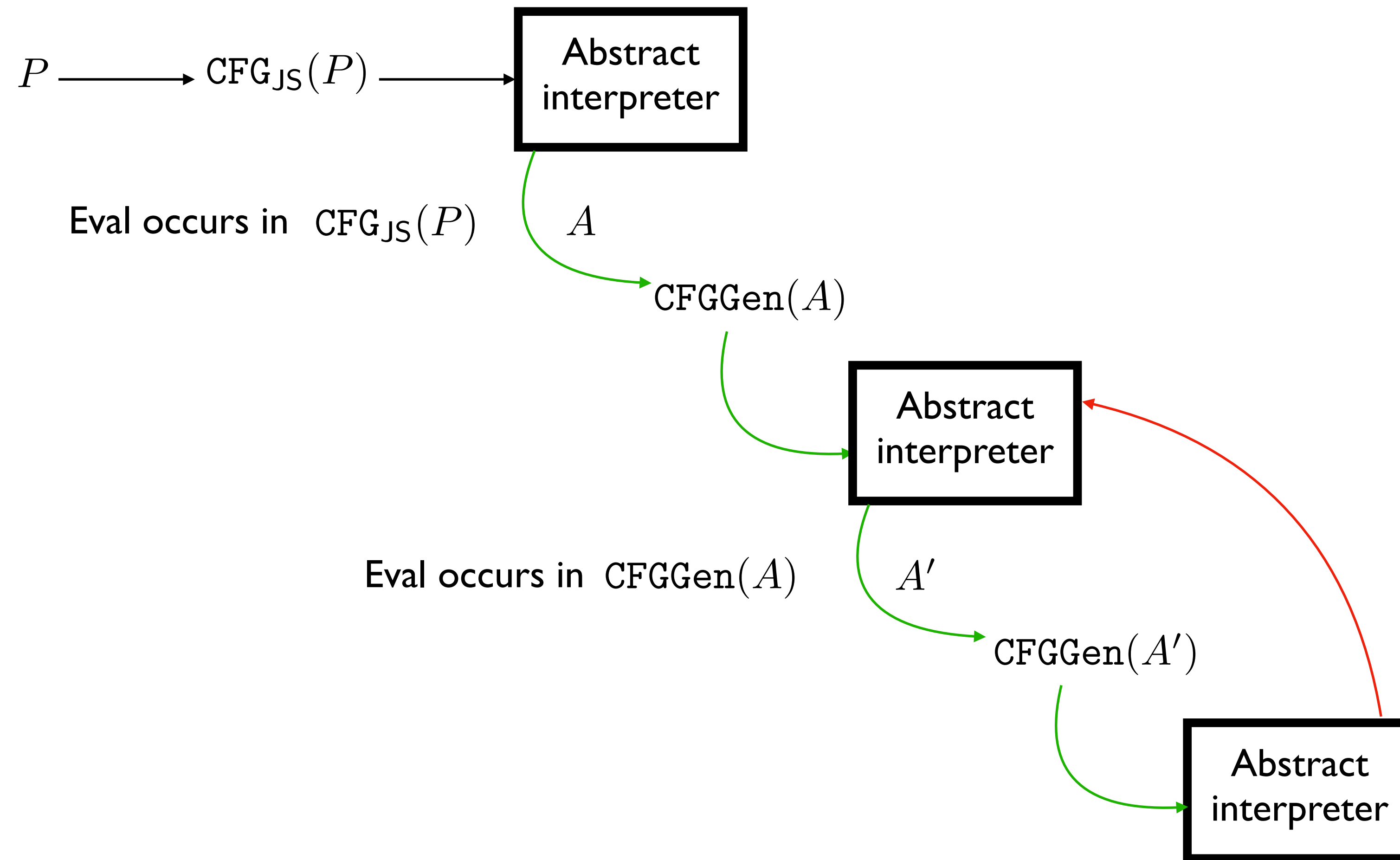
Architecture



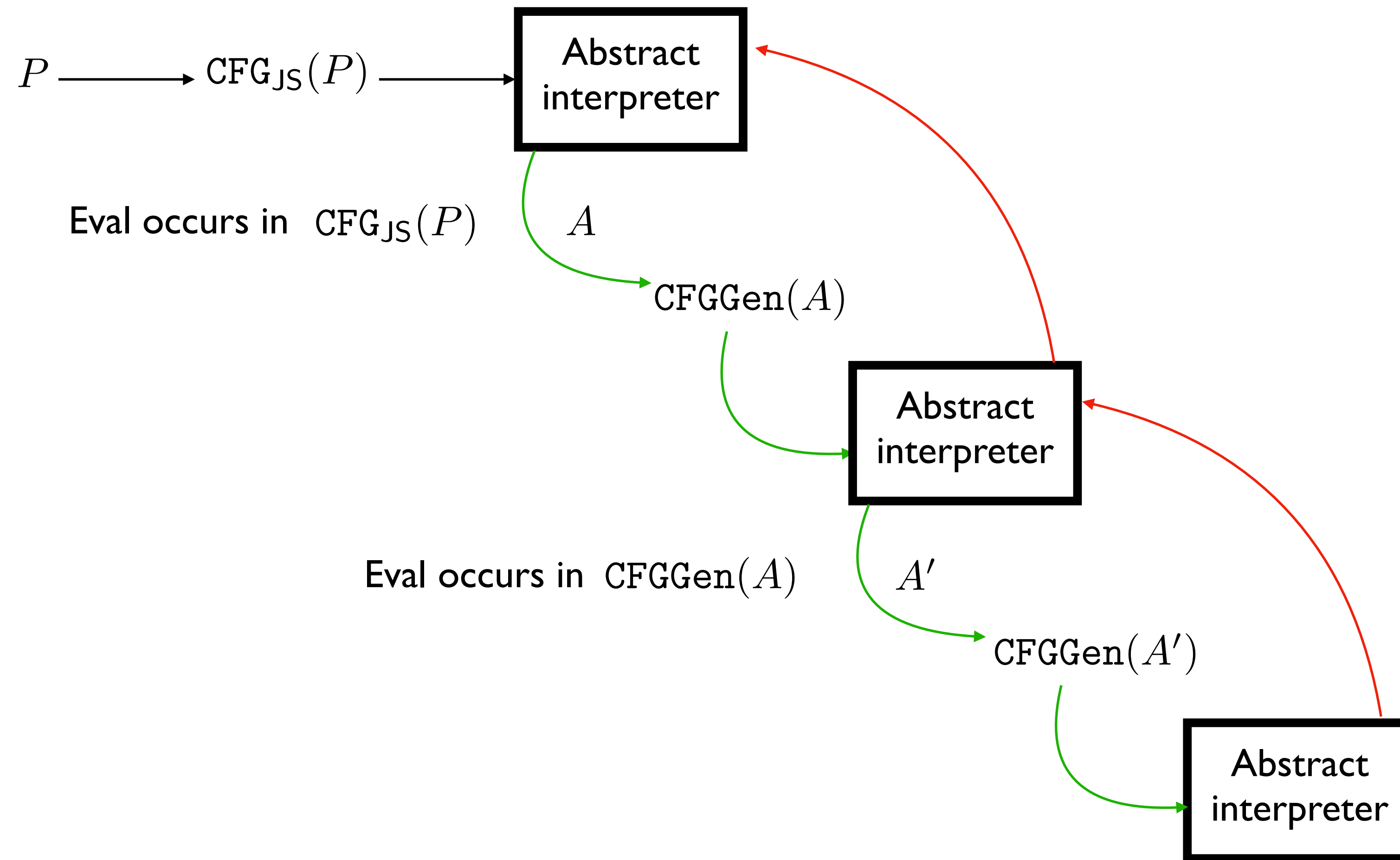
Architecture



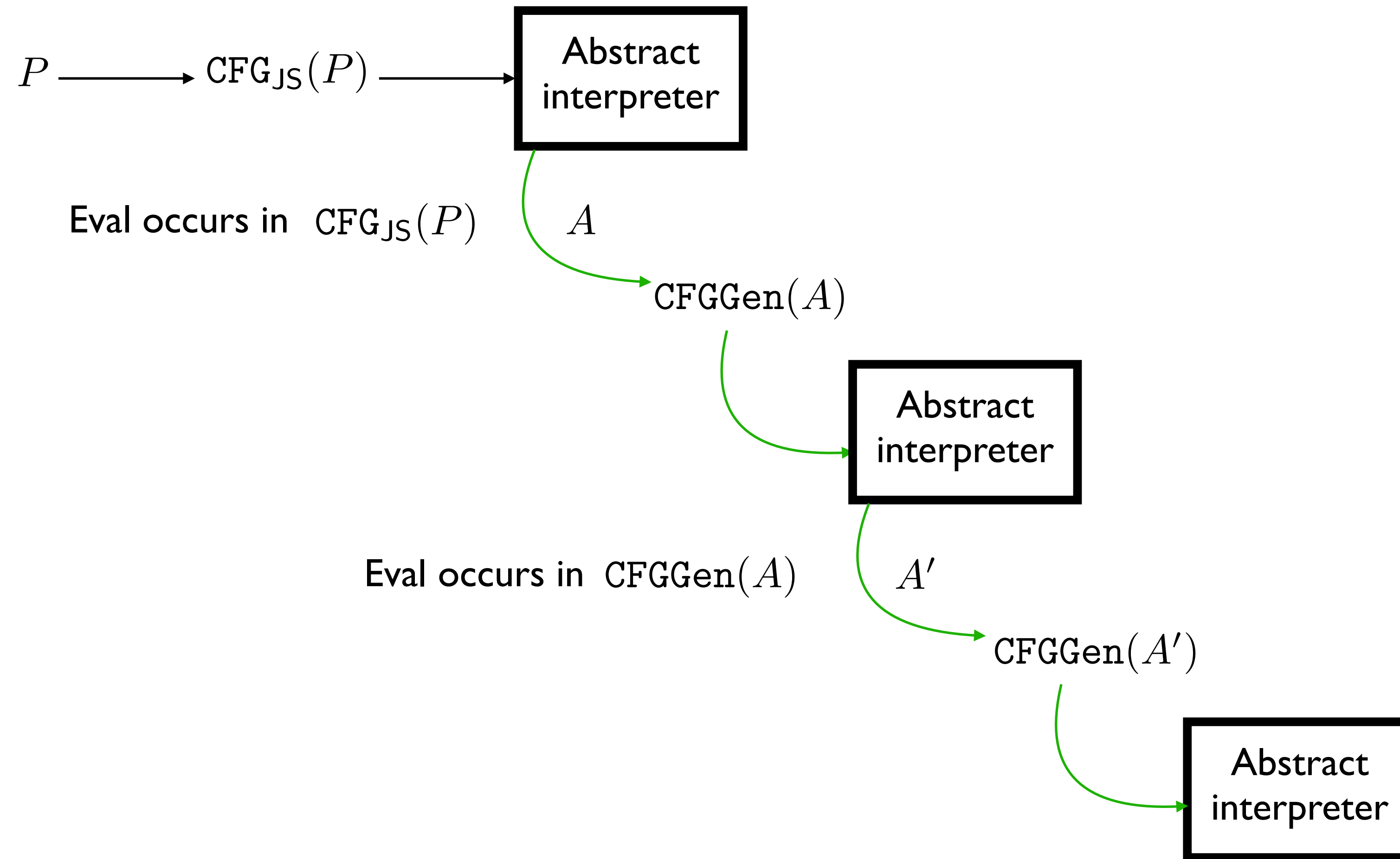
Architecture



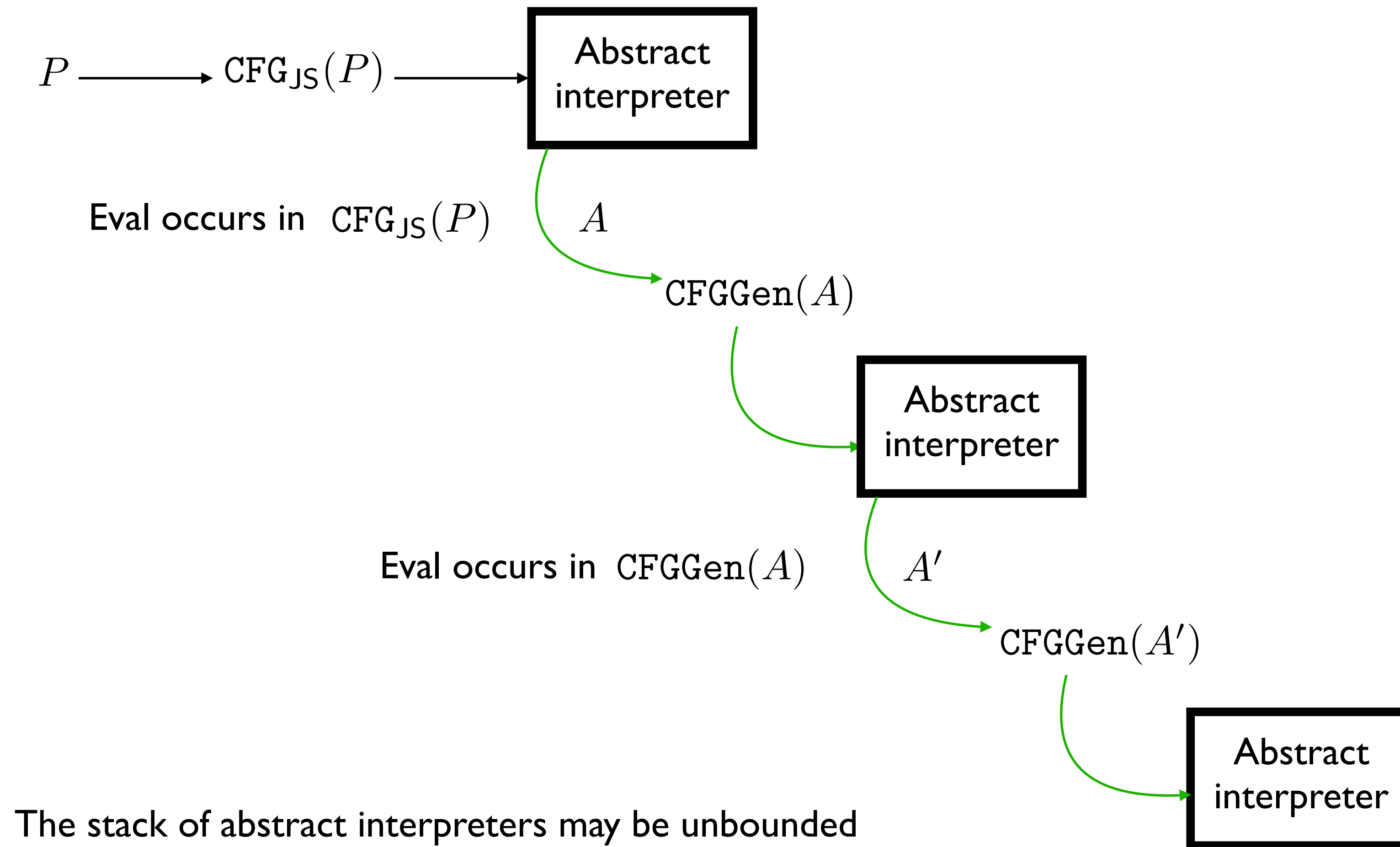
Architecture



Architecture

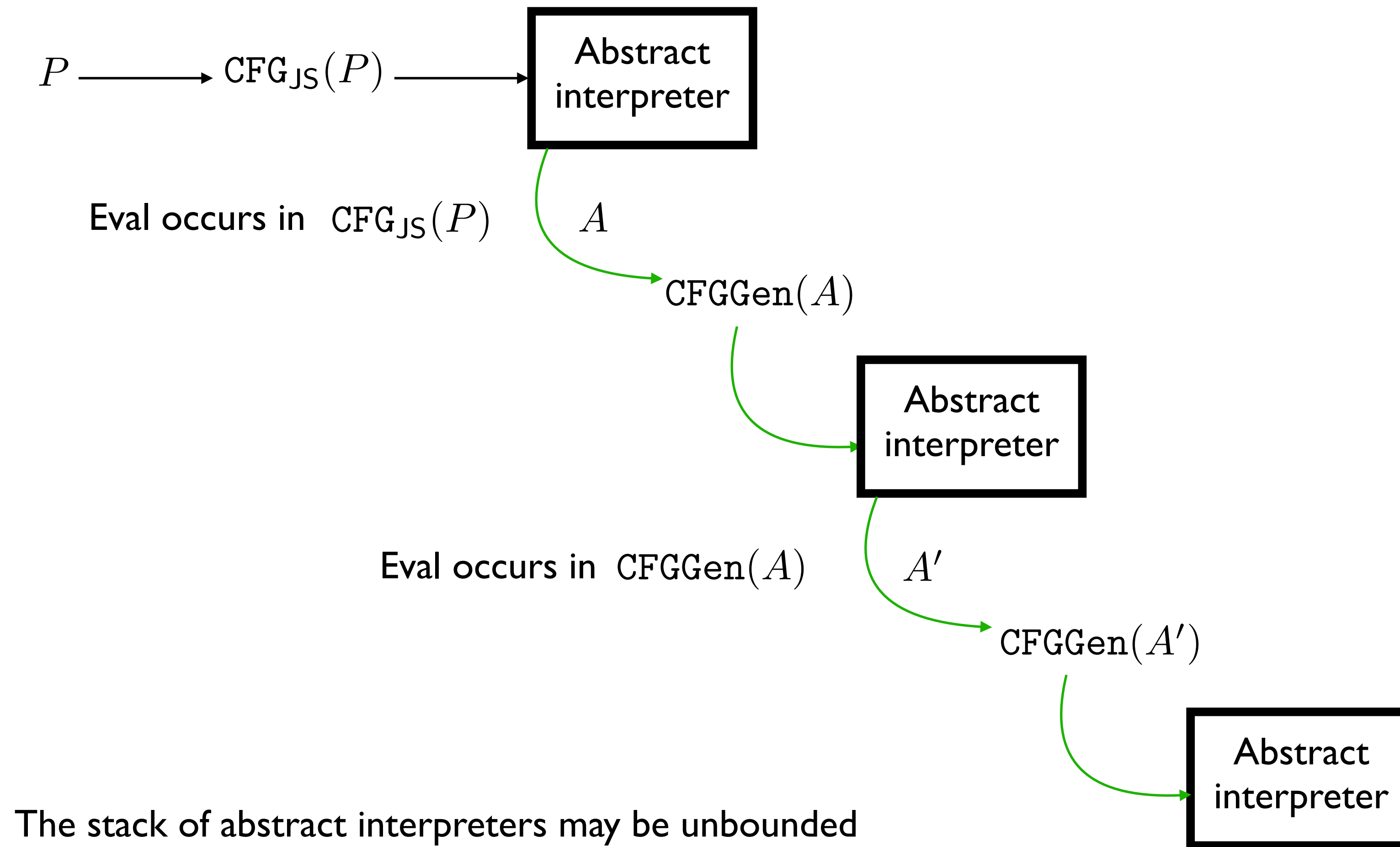


Architecture



```
x = "eval(x)";  
eval(x);
```

Architecture



```
x = "eval(x)";  
eval(x);
```

Stack height threshold: fix a maximal height of nested eval calls.

Soundness

Theorem 6.14 (Soundness). *Let $s \in \text{SE}$, let A_s be the FA recognizing the strings associated with s , then $\forall \sigma \in \mathcal{L}(A_s) \cap \mu\text{JS}, \forall m^\# \in \mathbb{M}^\#$*

$$\exists \Pi \subseteq \text{Paths}(G_s). G_s \triangleq \text{CFGGen}(\text{Regex}(\text{StmSyn}(A_s))). \llbracket \sigma \rrbracket^\# m^\# \subseteq \bigcup_{\pi \in \Pi} \llbracket \pi \rrbracket^\# m^\#$$

